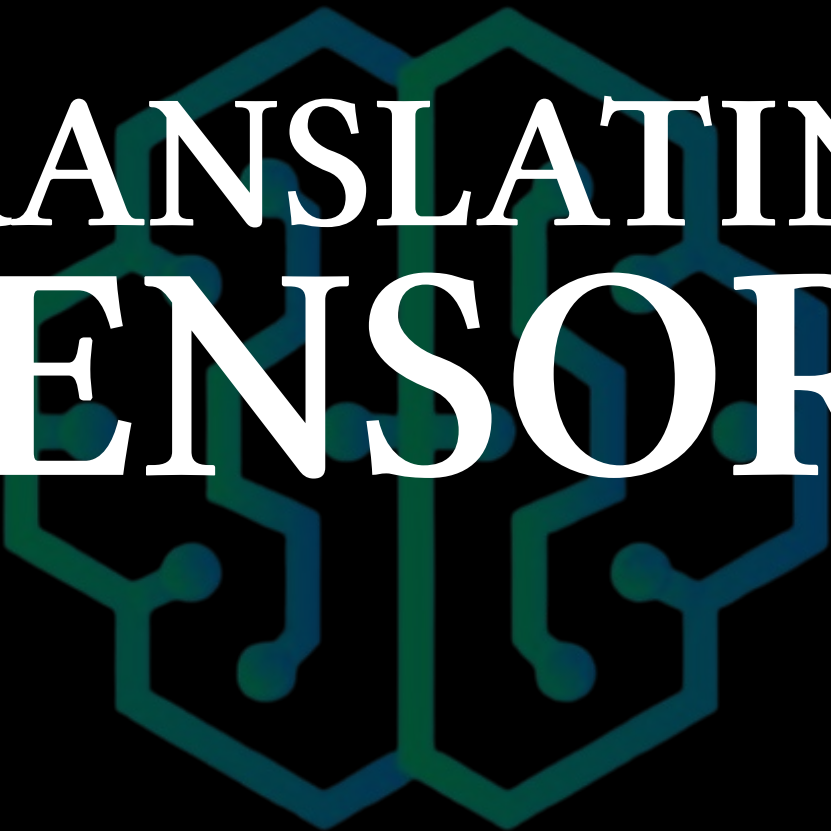




TRANSLATING TENSORS

AI Club, CFI
2026-27

Preface

We at AI Club are a community of students deeply interested in AI and its mathematical foundations. We work on research and implementation based projects, conduct events and sessions and organize hackathons to improve ourselves and help anybody interested in exploring the ever-growing domains in AI.

Every year during summer, the coordinators of AI Club conduct Knowledge Transfer Sessions (KTS) covering topics ranging from linear regression all the way up till complex neural network architectures, to ensure that the flow and quality of knowledge and expertise is sustained in the team.

This year, we present this book as a compilation of all our Knowledge Transfer Sessions, designed to be a useful guide for anybody looking forward to explore AI. We have also made the original slides and codes of the KTS available [here](#) and the recordings are available [here](#).

Disclaimer: Gemini 3.1 Pro was used to compile together all the KTS slides to generate the first draft of this book. However, the content has been proofread and highly refined to remove all instances of hallucinations and incompleteness.

On behalf of AI Club

Pakshal Nagda
Vaishali Rambabu

Team Leads
AI Club 2026-27

Contents

1	Linear Regression	<i>Pakshal</i>	1
1.1	Warmup: The Epic Story of Machine Learning		1
1.2	Ordinary Least Squares (OLS)		2
1.3	The Real World: Unknown Variance and Heteroscedasticity		5
1.4	The Bias-Variance Tradeoff & Regularization		7
1.5	A Probabilistic Lens		8
1.6	Binary Classification		9
1.7	Multi-class Classification		10
2	Classification Fundamentals	<i>Varnita & Rishi</i>	13
2.1	Warmup: The Art of Drawing Boundaries		13
2.2	K-Nearest Neighbours (KNN)		13
2.3	Naive Bayes: A Probabilistic Classifier		15
2.4	The Perceptron: The Foundation of Neural Networks		16
2.5	Support Vector Machines (SVM)		17
3	Tree-Based Models & Ensembles	<i>Shloka & Bhavana</i>	20
3.1	Warmup: The Wisdom of the Crowd		20
3.2	Decision Trees: The Universal Function Approximator		20
3.3	The Math of Splitting: Gini and Entropy		21
3.4	Ensemble Learning Fundamentals		23
3.5	Bagging & Random Forests		25
3.6	Boosting: AdaBoost to XGBoost		25
4	Unsupervised Learning	<i>Varnita & Hriday</i>	28
4.1	Warmup: Venturing into the Unknown		28
4.2	K-Means & The Centroid		29
4.3	Hierarchical Clustering		30
4.4	DBSCAN: A Density-Based Approach		31
4.5	Dimensionality Reduction & Principal Component Analysis		32
5	Deep Learning Foundations	<i>Mahir & Pranaav</i>	42
5.1	Warmup: The Limits of Linearity		42
5.2	The Artificial Neuron		42
5.3	Forward Propagation and Loss		44
5.4	Backpropagation: The Engine of Learning		44
5.5	The Pathology of Deep Networks: Gradients Strike Back		45
5.6	Optimizers: Navigating the Loss Landscape		45
5.7	Autoencoders		47
5.8	The Architecture of an Autoencoder		48
5.9	Flavors of Autoencoders		49

6	Natural Language Processing & Sequential Models	<i>Vishal & Rithvik</i>	51
6.1	Warmup: The Language Problem		51
6.2	Statistical NLP: N-Grams and HMMs		51
6.3	Neural NLP		53
6.4	The Challenge of Representing Words as Numbers		54
6.5	Dense Word Embeddings		55
6.6	Subword Tokenization & Byte-Pair Encoding (BPE)		57
6.7	Where is Our Model ? Autocomplete using ANN's		57
6.8	The Intuition : How we Autocomplete sentences		60
6.9	Recurrent Neural Networks : RNNs		61
6.10	LSTMs and GRUs: Gating the Memory		64
6.11	Sampling: Making the Model Creative		65
7	Transformers & State Space Models	<i>Shloka & Atharva</i>	66
7.1	Warmup: The Sequential Bottleneck		66
7.2	Attention is All You Need		66
7.3	The Transformer Architecture		68
7.4	The Sequence Modeling Crisis: Enter SSMs		69
7.5	State Space Models (SSMs)		70
7.6	Mamba: The Selective SSM		70
8	Computer Vision Fundamentals	<i>Madhura & Pranaav</i>	73
8.1	Warmup: The Visual World and the Flattening Problem		73
8.2	The Convolutional Operation		73
8.3	The Anatomy of a CNN Architecture		75
8.4	Data Augmentation		75
8.5	The Deep Training Crisis: A Gradients Recap		76
8.6	Transfer Learning: Standing on the Shoulders of Giants		76
9	Advanced Vision & Object Detection	<i>Abhinav & Krish</i>	79
9.1	Warmup: The Depth Conundrum		79
9.2	VGG Networks: The Power of Tiny Kernels		79
9.3	ResNets and The Degradation Problem		80
9.4	From Classification to Object Detection		82
9.5	YOLO: You Only Look Once		82
9.6	The Edge of Vision: UNet and ViT		83
9.7	Vision Transformers (ViT)		86
10	Reinforcement Learning	<i>Bhavana & Atharva</i>	89
10.1	Warmup: The RL Loop and Foresight		89
10.2	Markov Decision Processes (MDP)		89
10.3	Value Functions and The Bellman Equation		90
10.4	Dynamic Programming (DP)		93
10.5	Monte Carlo (MC) Methods		94
10.6	Temporal Difference (TD) Learning		94
A	Mathematical Toolkit		96
A.1	Linear Algebra: The Language of Data		96
A.2	Probability and Statistics: Handling Uncertainty		97
A.3	Calculus and Optimization: How Models Learn		99

Chapter 1

Linear Regression

Pakshal

[Link to session recording](#)

1.1 Warmup: The Epic Story of Machine Learning

Welcome to the world of Machine Learning (ML). Before we dive into matrices, gradients, and probability distributions, we need to ask a fundamental question: *Why do we need Machine Learning in the first place?*

Imagine there is some mysterious, hidden process out there in the universe generating data. Let's call the final output of this process y . This process generates y as a function of some other data x (which we call features) that it gathers from the environment. Mathematically, we can write this absolute truth as:

$$y = f(x) \tag{1.1}$$

where $x \in \mathbb{R}^d$ represents our d features, and $y \in \mathbb{R}$ is our target output.

Our job in machine learning is seemingly simple: given an arbitrary input x , predict the output $f(x)$. If we knew the exact mathematical form of f , all our problems would be solved. We wouldn't need ML at all; we would just write a simple program to calculate it!

But here is the hard reality:

- We have zero clue about the true form of f .
- We do not even know if we are measuring all the input features x needed for that function.
- We may not even be measuring the output y perfectly accurately.

All we have is a dataset of n historical input-output pairs (x, y) and nothing else. Thus, to make any progress, we must summon our two best friends: **Assumptions** and **Approximations**. This is where the epic story of ML begins.

1.1.1 The Assumptions

First, we assume that the outputs y we measure are inherently noisy. The universe isn't perfect, and our sensors aren't either. We assume the data was actually generated by:

$$y = f(x) + e \tag{1.2}$$

where e is a random variable denoting the error or noise in our measurement.

Second, since we have no idea what the true $f(x)$ looks like, we will approximate it with a function $\hat{f}(x)$ whose shape we get to decide. The absolute simplest shape we can choose is a straight line (or a flat plane in higher dimensions). This is our **Linearity Assumption**:

$$\hat{f}(x) = x^T \theta = x_1 \theta_1 + x_2 \theta_2 + \cdots + x_d \theta_d \quad (1.3)$$

where $\theta \in \mathbb{R}^d$ is the vector of **parameters** (or **weights**) that define our line. Our entire goal now reduces to finding the “best” possible θ given our dataset.

Note that technically we should also have an additional **bias** term θ_0 such that

$$\hat{f}(x) = \theta_0 + x^T \theta = \theta_0 + x_1 \theta_1 + x_2 \theta_2 + \cdots + x_d \theta_d \quad (1.4)$$

However we need not worry ourselves with a separate parameter, as we can add a new feature x_0 to our data which always equals 1. This way, we can treat θ_0 similar to just any other parameter in the weight vector.

$$\hat{f}(x) = \tilde{x}^T \tilde{\theta} \quad \text{where } \tilde{x}, \tilde{\theta} \in \mathbb{R}^{d+1} \quad (1.5)$$

To keep the math simple, we will just go with $\hat{f}(x) = x^T \theta$ assuming x already includes the additional dimension to incorporate the bias term.

1.2 Ordinary Least Squares (OLS)

Now that we have established our linear approximation, how do we find the “best” parameters θ ? To answer this, we need to mathematically define what “best” means.

Things may get mathematically heavy from now on. If you feel uncomfortable with the math at any point, please go through [Appendix A](#).

1.2.1 Matrix Formulation

Let’s look at all our n samples at once. We can pack all our input features into a massive matrix $X \in \mathbb{R}^{n \times d}$, where each row is a data sample and each column is a feature. We pack our targets into a vector $y \in \mathbb{R}^n$.

$$\underbrace{\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}}_{d \text{ Features / Columns}} \left\{ \begin{array}{l} \\ \\ \\ \end{array} \right\} n \text{ Samples} \left\{ \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{1 \text{ Column}} \quad \underbrace{\begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{bmatrix}}_{1 \text{ Column}} \right\} d \text{ Features}$$

Our model’s predictions for all n samples can be written elegantly using matrix multiplication:

$$\hat{y} = X\theta \quad (1.6)$$

Take a moment to digest the above statement. It is advisable to write down the expansion yourself and check that each term in $\hat{y}$ is $\hat{f}$ applied to every row of X .

We want to find a θ such that our predictions $\hat{y}$ are as close to the actual y as possible.

We define the difference between the truth and our prediction as the **Residual vector**: $r = y - \hat{y}$. Each term in r denotes the error in our predictions for the corresponding training data.

To penalize both positive and negative errors equally, and to strongly penalize massive errors, we define our objective function $J(\theta)$ as the Sum of Squared Errors (SSE):

$$J(\theta) = \sum_i r_i^2 = \|r\|_2^2 = r^T r \quad (1.7)$$

We wish to minimize this objective in order to minimize all errors in our predictions on the training data and the θ that achieves this will be the set of “best” parameters.

It is very right to question this specific choice of objective function. Why not sum of absolute errors or sum of errors to the power of 4? Strictly speaking, there is no restriction to use these alternatives too, but we will see soon that under certain assumptions about the errors e , the above formulation of objective is the most optimal.

1.2.2 The Calculus Derivation

Let's expand our objective function:

$$\begin{aligned} J(\theta) &= (y - \hat{y})^T (y - \hat{y}) \\ &= (y - X\theta)^T (y - X\theta) \\ &= y^T y - \theta^T X^T y - y^T X\theta + \theta^T X^T X\theta \end{aligned}$$

Since $\theta^T X^T y$ is a scalar, it is equal to its transpose $y^T X\theta$. Thus:

$$J(\theta) = y^T y - 2\theta^T X^T y + \theta^T X^T X\theta \quad (1.8)$$

This is a multivariable quadratic function. To find its global minimum, we take the gradient with respect to θ and set it to zero:

$$\nabla_{\theta} J(\theta^*) = -2X^T y + 2X^T X\theta^* = 0 \quad (1.9)$$

$$X^T X\theta^* = X^T y \quad (1.10)$$

Assuming $X^T X$ is invertible (which means our features are not perfectly correlated, i.e., no multicollinearity), we arrive at the legendary **Normal Equation**:

$$\theta^* = (X^T X)^{-1} X^T y \quad (1.11)$$

1.2.3 The Geometric Derivation

Calculus is great, but linear algebra gives us a beautiful visual intuition for what just happened.

Ideally, we want to solve the exact linear system $y = X\theta$.

Don't be tempted to directly jump to $\theta^ = X^{-1}y$! That can be true only if X is square and invertible, which is obviously not necessary here.*

From a linear algebra perspective, a perfect solution only exists if the vector y lies perfectly within the column space of X , denoted as $\text{col}(X)$, but because of noise, y almost never lies in this plane.

Instead, we seek a target vector $\hat{y} \in \text{col}(X)$ that minimizes the distance to y . The shortest path from a point to a plane is a straight, perpendicular drop. Therefore, the residual vector $r = y - \hat{y}$ must be strictly orthogonal to the column space of X .

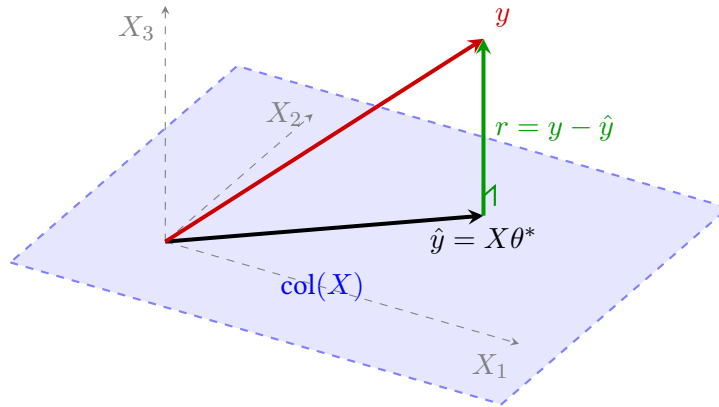


Figure 1.1: Geometric projection of y onto the column space of X .

If r is orthogonal to every column of X , then the dot product of X^T and r must be zero:

$$\begin{aligned} X^T r &= 0 \\ X^T (y - X\theta^*) &= 0 \\ X^T X\theta^* &= X^T y \end{aligned}$$

Which leads us to the exact same Normal Equation!

1.2.4 Code Implementation: Exact OLS

While libraries like `scikit-learn` handle this under the hood, implementing the closed-form solution in Python using NumPy is incredibly straightforward:

```

1 import numpy as np
2
3 def fit_ols(X, y):
4     # Add a column of ones to X to account for the bias (intercept) term
5     X_b = np.c_[np.ones((X.shape[0], 1)), X]
6
7     # Calculate theta* = (X^T * X)^-1 * X^T * y
8     theta_star = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
9
10    return theta_star

```

Listing 1.1: Exact OLS Implementation

*Note: Computing the inverse $(X^T X)^{-1}$ is computationally heavy $O(d^3)$ and can be unstable if features are highly correlated. This is why **gradient descent** (will be introduced in Section 5.6.1) is often preferred for large datasets.*

If you are reading this book for the first time or are not very comfortable playing around with linear algebra and probability, you can feel free to directly jump to Section 1.4.

1.2.5 The Gauss-Markov Theorem (BLUE)

We stated earlier that $\theta^* = (X^T X)^{-1} X^T y$. But how good is this estimate? Because our target y contains random noise e , our estimate θ^* is also a random vector. Every time we sample a dataset, we get a slightly different θ^* .

Under the assumptions that our noise has a mean of zero ($\mathbb{E}[e] = 0$) and a constant variance ($\text{Var}(e) = \sigma^2 I$), the Gauss-Markov Theorem states that OLS is the **Best Linear Unbiased Estimator (BLUE)**. Let us break down what this means.

1. Linear: Our estimate is a linear transformation of the target vector y , since $\theta^* = Cy$ where $C = (X^T X)^{-1} X^T$.

2. Unbiased: Over infinitely many samples, the average of our estimates will perfectly align with the true parameters θ . We can prove this using the linearity of expectation:

$$\begin{aligned}
 \mathbb{E}[\theta^*] &= \mathbb{E}[(X^T X)^{-1} X^T y] \\
 &= (X^T X)^{-1} X^T \mathbb{E}[y] \\
 &= (X^T X)^{-1} X^T \mathbb{E}[X\theta + e] \\
 &= (X^T X)^{-1} X^T (X\theta + \mathbb{E}[e]) \\
 &= (X^T X)^{-1} X^T X\theta + 0 \\
 &= I\theta = \theta
 \end{aligned}$$

3. Best (Minimum Variance): Among all possible linear, unbiased estimators, OLS has the tightest variance. That is, the variance of any linear combination of parameters θ is the minimum in case of OLS. Here is the proof:

Consider a linear unbiased estimator $\hat{\theta} = Cy$

Since it is unbiased, $\mathbb{E}[\hat{\theta}] = \theta \implies \mathbb{E}[Cy] = \mathbb{E}[C(X\theta + e)] = CX\theta + CX\mathbb{E}[e] = CX\theta = \theta$

$$CX = X^T C^T = I \tag{1.12}$$

Using the affine property of covariance ($\text{Cov}(Ay) = A\text{Cov}(y)A^T$), $\text{Cov}(\hat{\theta})$ can be written as

$$\begin{aligned}\text{Cov}(\hat{\theta}) &= \text{Cov}(Cy) \\ &= C\text{Cov}(y)C^T \\ &= C\text{Cov}(X\theta + e)C^T \\ &= C\text{Cov}(e)C^T \\ &= C\sigma^2 IC^T \\ &= \sigma^2 CC^T\end{aligned}$$

Similarly plugging in $C = (X^T X)^{-1} X^T$ for OLS,

$$\text{Cov}(\theta^*) = \sigma^2 (X^T X)^{-1} X^T X (X^T X)^{-1} = \sigma^2 (X^T X)^{-1} \quad (1.13)$$

Consider $D = C - (X^T X)^{-1} X^T$

$$\begin{aligned}DD^T &= (C - (X^T X)^{-1} X^T)(C^T - X(X^T X)^{-1}) \\ &= CC^T - CX(X^T X)^{-1} - (X^T X)^{-1} X^T C^T + (X^T X)^{-1} X^T X (X^T X)^{-1} \\ &= CC^T - (X^T X)^{-1} - (X^T X)^{-1} + (X^T X)^{-1} \\ &= CC^T - (X^T X)^{-1}\end{aligned}$$

Consider the variance of any linear combination of parameters, say $\hat{z} = \alpha_1 \hat{\theta}_1 + \alpha_2 \hat{\theta}_2 + \dots + \alpha_d \hat{\theta}_d = \alpha^T \hat{\theta}$. We can show that $\text{var}(z^*) \leq \text{var}(\hat{z}) \quad \forall \alpha \in \mathbb{R}^d$

$$\begin{aligned}\text{var}(\hat{z}) - \text{var}(z^*) &= \alpha^T \text{Cov}(\hat{\theta}) \alpha - \alpha^T \text{Cov}(\theta^*) \alpha \\ &= \alpha^T (\text{Cov}(\hat{\theta}) - \text{Cov}(\theta^*)) \alpha \\ &= \alpha^T (\sigma^2 CC^T - \sigma^2 (X^T X)^{-1}) \alpha \\ &= \sigma^2 \alpha^T (CC^T - (X^T X)^{-1}) \alpha \\ &= \sigma^2 \alpha^T DD^T \alpha \\ &= \sigma^2 (D^T \alpha)^T (D^T \alpha) \\ &\geq 0\end{aligned}$$

Hence, $\text{var}(z^*) \leq \text{var}(\hat{z}) \quad \forall \alpha \in \mathbb{R}^d$ and hence we can claim that OLS is the “best” possible linear unbiased estimator of θ . In simpler terms: OLS squeezes the maximum possible certainty out of your data.

1.3 The Real World: Unknown Variance and Heteroscedasticity

Up to this point, our probabilistic lens relied on two bold assumptions: first, that we magically knew the true error variance σ^2 , and second, that this variance was identical for every single data point (an assumption known as **Homoscedasticity**). In reality, neither of these is usually true.

1.3.1 Estimating the Error Variance (σ^2)

In practice, we must estimate the true error variance σ^2 using our sample residuals: $r = y - \hat{y} = y - X\theta^*$.

Let us express the residual vector r in terms of the true, unobservable errors e . Recall that $\theta^* = (X^T X)^{-1} X^T y$ and $y = X\theta + e$.

$$\begin{aligned}r &= y - X(X^T X)^{-1} X^T y \\ &= (I_n - H)y\end{aligned}$$

Here, $H = X(X^T X)^{-1} X^T$ is known as the **Hat Matrix** because it puts the "hat" on y (since $\hat{y} = Hy$). H has two special properties: it is symmetric ($H^T = H$) and idempotent ($H^2 = H$). Consequently, $(I_n - H)$ is also symmetric and idempotent.

Substituting $y = X\theta + e$, we get:

$$r = (I_n - H)(X\theta + e) = (I_n - H)X\theta + (I_n - H)e = (I_n - H)e \quad (1.14)$$

Note: $(I_n - H)X = X - X = 0$, which simplifies the equation.

To find the expected Sum of Squared Errors (SSE), we evaluate $\mathbb{E}[r^T r]$:

$$\mathbb{E}[r^T r] = \mathbb{E}[e^T (I_n - H)^T (I_n - H)e] = \mathbb{E}[e^T (I_n - H)e] \quad (1.15)$$

Because $e^T (I_n - H)e$ is a 1×1 scalar, it is equal to its own Trace. Using the cyclic property of the Trace operator ($\text{Tr}(ABC) = \text{Tr}(CAB)$) and the linearity of expectation, we can perform the famous "Trace Trick":

$$\begin{aligned} \mathbb{E}[r^T r] &= \mathbb{E}[\text{Tr}(e^T (I_n - H)e)] \\ &= \mathbb{E}[\text{Tr}((I_n - H)ee^T)] \\ &= \text{Tr}((I_n - H)\mathbb{E}[ee^T]) \\ &= \text{Tr}((I_n - H)\sigma^2 I_n) \\ &= \sigma^2 \text{Tr}(I_n - H) \end{aligned}$$

Using the properties of the trace, $\text{Tr}(I_n) = n$, and $\text{Tr}(X(X^T X)^{-1} X^T) = \text{Tr}((X^T X)^{-1} X^T X) = \text{Tr}(I_d) = d$. Therefore, $\text{Tr}(I_n - H) = n - d$. This gives us our final, unbiased estimator for the variance:

$$\hat{\sigma}^2 = \frac{r^T r}{n - d} = \frac{SSE}{n - d} \quad (1.16)$$

We elegantly divide by $n - d$ (instead of n) to account for the d degrees of freedom lost by estimating the d parameters of θ^* .

1.3.2 Weighted Least Squares (WLS)

What happens if the assumption of homoscedasticity completely fails? Imagine predicting house prices: the error variance for a \$10M mansion is likely much larger than the error variance for a \$100k apartment. If errors have different variances, their covariance matrix is no longer a simple scalar multiple of the identity matrix: $\text{Cov}(e) = \Sigma_e \neq \sigma^2 I_n$.

Standard OLS treats all points equally, which is suboptimal here. We want to give less weight to high-variance (noisy) points and more weight to low-variance (reliable) points.

To solve this, we can "whiten" the data. Since Σ_e is a valid covariance matrix, it is positive definite and can be decomposed (e.g., via Cholesky factorization) as $\Sigma_e = PP^T$. If we multiply our entire linear model by P^{-1} :

$$P^{-1}y = P^{-1}X\theta + P^{-1}e \implies y^* = X^*\theta + e^* \quad (1.17)$$

Let's check the covariance of our new transformed error vector e^* :

$$\text{Cov}(e^*) = \text{Cov}(P^{-1}e) = P^{-1}\text{Cov}(e)(P^{-1})^T = P^{-1}(PP^T)(P^{-1})^T = I_n \quad (1.18)$$

Brilliant! In this transformed space, the errors have an identity covariance matrix, satisfying standard OLS assumptions. Applying the OLS formula to our transformed data yields the **Weighted Least Squares (WLS)** estimator:

$$\begin{aligned} \theta_{WLS}^* &= ((X^*)^T X^*)^{-1} (X^*)^T y^* \\ &= (X^T (P^{-1})^T P^{-1} X)^{-1} X^T (P^{-1})^T P^{-1} y \\ &= (X^T \Sigma_e^{-1} X)^{-1} X^T \Sigma_e^{-1} y \end{aligned}$$

Note that still the issue that the covariance matrix Σ_e is unknown remains. It can be approximated using some known dynamics of the problem.

```

1 import numpy as np
2
3 def fit_wls(X, y, Sigma_e):
4     # Add a column of ones to X to account for the bias
5     X_b = np.c_[np.ones((X.shape[0], 1)), X]
6
7     # The inverse of the covariance matrix acts as our weights
8     W = np.linalg.inv(Sigma_e)
9
10    # Calculate theta_wls = (X^T * W * X)^-1 * X^T * W * y
11    theta_wls = np.linalg.inv(X_b.T @ W @ X_b) @ X_b.T @ W @ y
12
13    return theta_wls

```

Listing 1.2: Weighted Least Squares Implementation

1.4 The Bias-Variance Tradeoff & Regularization

Our OLS estimator is known as the Best Linear Unbiased Estimator (BLUE). However, being “unbiased” isn’t always perfect.

If we have limited data or highly correlated features, our model might become highly sensitive to the training data. It learns the noise instead of the signal. This is **high variance** (Overfitting). Conversely, if our model is too simple (like fitting a line to a parabola), it has **high bias** (Underfitting).

Our OLS estimator is known as the Best Linear Unbiased Estimator (BLUE). However, being unbiased isn’t always the ultimate goal. If we evaluate the Expected Mean Squared Error of our model on an unseen data point x_0 , the error mathematically decomposes into three distinct components:

$$\mathbb{E} \left[(y - \hat{f}(x_0))^2 \right] = \sigma^2 + \text{Var}(\hat{f}(x_0)) + \left[\text{Bias}(\hat{f}(x_0)) \right]^2 \quad (1.19)$$

1. **Irreducible Error (σ^2):** The inherent noise in the data generation process. No matter how good our model is, we can never eliminate this.
2. **Variance:** How much our prediction would jump around if we trained the model on a different sample of data. High variance implies the model is overly sensitive to the training data (**Overfitting**).
3. **Bias Squared:** The error introduced by approximating a highly complex real-world problem with a simpler model. High bias implies the model is too rigid to capture the underlying pattern (**Underfitting**).

If we have limited data or highly correlated features, our OLS model might suffer from massive variance. To minimize the *total* error, we are actually willing to introduce a little bit of bias if it dramatically reduces our variance.

1.4.1 Ridge and Lasso Regression

To mitigate high variance, we penalize the model for having excessively large parameters. We do this by adding a regularization term to our objective function.

Ridge Regression (L2 Penalty):

$$J(\theta) = (y - X\theta)^T (y - X\theta) + \lambda \|\theta\|_2^2 \quad (1.20)$$

This prevents any single feature from dominating the model. We can derive it similar to the Normal Equation using calculus. The closed-form solution becomes $\theta^* = (X^T X + \lambda I)^{-1} X^T y$.

This can be thought of as increasing the eigen values of the matrix to be inverted, thus effectively mitigating the case when the matrix is (nearly) singular due to (highly) correlated features.

LASSO (Least Absolute Shrinkage and Selection Operator) Regression (L1 Penalty):

$$J(\theta) = (y - X\theta)^T(y - X\theta) + \lambda\|\theta\|_1 \quad (1.21)$$

Lasso forces less important feature weights to exactly zero, effectively performing automatic feature selection. However, there isn't any closed form solution due to the non-differentiability of $\|\theta\|_1$ in certain points. It is typically formulated and solved as a quadratic programming problem.

Again if you are curious why specifically these penalties, why not L4 or L5 penalties, you will find your answer in the next section.

1.5 A Probabilistic Lens

We arbitrarily chose to minimize the Sum of Squared Errors. But was that the *right* choice? To answer this, we must look at our data through the lens of probability.

Recall our assumption: $y = X\theta + e$. Let us assume that the errors e are Gaussian (Normally distributed) with a mean of 0 and a variance of σ^2 . Because the error e is a random variable, our target y also becomes a random variable conditioned on X :

$$P(y_i|x_i; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^T\theta)^2}{2\sigma^2}\right) \quad (1.22)$$

1.5.1 Maximum Likelihood Estimation (MLE)

If we assume our n samples are independent, the Likelihood of observing our entire dataset is the product of their individual probabilities. **Maximum Likelihood Estimation (MLE)** asks: What parameter θ makes our observed dataset the most likely to have occurred?

$$\begin{aligned} \theta_{MLE}^* &= \arg \max_{\theta} \log L(\theta) \\ &= \arg \max_{\theta} \sum_{i=1}^n \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y_i - x_i^T\theta)^2 \right] \end{aligned}$$

To maximize this, we can ignore the constants and focus on minimizing the negative term:

$$\theta_{MLE}^* = \arg \min_{\theta} \sum_{i=1}^n (y_i - x_i^T\theta)^2 \quad (1.23)$$

This is a profound realization: **Minimizing the residual sum of squares is mathematically identical to maximizing the likelihood of the data assuming Gaussian noise!** Our seemingly “arbitrary” choice of squared error is actually perfectly justified.

1.5.2 The Bayesian Perspective (MAP)

Now coming to the next question: why L1 and L2 regularization? If MLE relies purely on data, **Maximum A Posteriori (MAP)** allows us to introduce a *prior belief* about what our weights should be before we even see the data.

- If we assume our weights θ follow a Gaussian distribution centered around zero, MAP mathematically derives the **Ridge Regression** objective.
- If we assume our weights follow a Laplace distribution (which encourages sparsity), MAP mathematically derives the **Lasso Regression** objective.

Regularization is simply enforcing a mathematical bias that our parameters should be close to zero!

1.6 Binary Classification

Linear regression is brilliant when predicting continuous numbers (like house prices). But what if we want to predict a binary category, like whether an email is "Spam" (1) or "Not Spam" (0)?

1.6.1 Logistic Regression

We can predict a target whose values can take any value in $(-\infty, \infty)$ using linear regression.

Predicting $y \in \{0, 1\}$ is not a good idea because it takes only discrete classes.

We can predict probability instead $P(y = 1|x) \in (0, 1)$, but this, too, takes only a limited range of values.

We can increase the range further by predicting the odds instead

$$\frac{P(y = 1|x)}{1 - P(y = 1|x)} \in (0, \infty)$$

However, still the range is not all real numbers. But, the log odds can take all real values.

$$\log \frac{P(y = 1|x)}{1 - P(y = 1|x)} \in (-\infty, \infty)$$

Note that now the range is the entire real line. We can fit a linear model to predict this.

Instead of predicting y directly, we predict the logarithm of the odds of $y = 1$:

$$\log \left(\frac{P(y = 1|x)}{1 - P(y = 1|x)} \right) = x^T \theta \quad (1.24)$$

Solving for $P(y = 1|x)$, we get the famous Sigmoid function:

$$\hat{y} = \frac{1}{1 + e^{-x^T \theta}} \quad (1.25)$$

Because our target is now a Bernoulli distribution (0 or 1), we can no longer use the Sum of Squared Errors. Instead, we return to our probabilistic lens: Maximum Likelihood Estimation (MLE).

Our model predicts the probability of the positive class: $\hat{y}_i = P(y_i = 1|x_i)$. We can write the probability mass function for a single true target y_i compactly using exponents:

$$P(y_i|x_i; \theta) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i} \quad (1.26)$$

Notice how if $y_i = 1$, the second term becomes 1, leaving just $\hat{y}_i$. If $y_i = 0$, the first term becomes 1, leaving $(1 - \hat{y}_i)$.

To find the parameters that maximize the likelihood of our entire dataset, we take the log-likelihood over all n independent samples:

$$\begin{aligned} \log L(\theta) &= \log \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i} \\ &= \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \end{aligned}$$

Optimization algorithms conventionally minimize functions. Therefore, we minimize the Negative Log-Likelihood, giving us the famous **Binary Cross Entropy (BCE) Loss**:

$$J(\theta) = - \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (1.27)$$

The BCE loss isn't an arbitrary choice; it naturally and strictly emerges from MLE for Bernoulli targets!

Unlike OLS, BCE does not have a closed-form solution. We absolutely must use iterative algorithms like **Gradient Descent** to find the parameters!

You may refer to Section 5.6.1 for more information regarding the gradient descent algorithm.

1.6.2 Code Implementation: Logistic Regression

Rather than relying on high-level wrappers like `scikit-learn`, implementing the training loop manually in Python ensures granular control over the forward and backward passes. This explicit mathematical mapping is especially useful if you plan to port the algorithm to C++ for tighter performance optimizations or custom hardware deployments later.

Here is the exact gradient descent implementation for Binary Cross-Entropy using NumPy:

```

1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 def compute_bce_loss(y, y_hat):
7     # Add epsilon to prevent log(0) domain errors
8     epsilon = 1e-15
9     y_hat = np.clip(y_hat, epsilon, 1 - epsilon)
10    return -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
11
12 def fit_logistic_regression(X, y, learning_rate=0.01, num_iterations=1000):
13     # Add a column of ones to X to account for the bias term
14     X_b = np.c_[np.ones((X.shape[0], 1)), X]
15     n_samples, n_features = X_b.shape
16
17     # Initialize parameters to zeros
18     theta = np.zeros(n_features)
19
20     for i in range(num_iterations):
21         # 1. Forward pass: compute linear combination and apply sigmoid
22         linear_model = X_b @ theta
23         y_hat = sigmoid(linear_model)
24
25         # 2. Compute gradient
26         # The gradient of BCE with respect to theta simplifies to (y_hat - y)
27         gradient = (1 / n_samples) * X_b.T @ (y_hat - y)
28
29         # 3. Update parameters
30         theta -= learning_rate * gradient
31
32         # Optional: Print loss every 100 iterations to monitor convergence
33         # if i % 100 == 0:
34         #     loss = compute_bce_loss(y, y_hat)
35         #     print(f"Iteration {i}: BCE Loss = {loss}")
36
37     return theta

```

Listing 1.3: Logistic Regression Implementation

1.7 Multi-class Classification

If we have k mutually exclusive classes instead of just 2, predicting a single probability is no longer sufficient. We need our model to output a valid probability distribution over all k classes. To do this, we generalize Logistic Regression into **Softmax Regression** (also known as Multinomial Logistic Regression).

1.7.1 Mathematical Formulation

First, we must change how we represent our target variable. Instead of a scalar $y \in \{0, 1\}$, we use **one-hot encoding**. If a sample belongs to class c , its target y is a k -dimensional vector where the c -th element is 1 and all others are 0.

Consequently, our parameter vector θ must expand into a parameter matrix $\Theta \in \mathbb{R}^{d \times k}$. Each column of Θ represents the weights for a specific class. For an input vector $x \in \mathbb{R}^d$, we compute a vector of raw scores (called **logits**), $z \in \mathbb{R}^k$:

$$z = \Theta^T x \quad (1.28)$$

To convert these unconstrained logits into a valid probability distribution (where all values are in $(0, 1)$ and sum to 1), we pass them through the **Softmax function**:

$$\hat{y}_c = P(y = c|x; \Theta) = \frac{e^{z_c}}{\sum_{j=1}^k e^{z_j}} = \frac{e^{\theta_c^T x}}{\sum_{j=1}^k e^{\theta_j^T x}} \quad (1.29)$$

1.7.2 Maximum Likelihood Estimation

Just as binary targets follow a Bernoulli distribution, one-hot encoded multi-class targets follow a **Categorical distribution** (or Multinoulli distribution). We can write the probability mass function for a single sample as:

$$P(y_i|x_i; \Theta) = \prod_{c=1}^k (\hat{y}_{ic})^{y_{ic}} \quad (1.30)$$

Notice that because y_i is one-hot, only the term corresponding to the true class c will have an exponent of 1. All other terms will have an exponent of 0 and evaluate to 1.

To find the optimal parameter matrix Θ^* , we maximize the likelihood of our entire dataset of n independent samples. Taking the log-likelihood:

$$\begin{aligned} \log L(\Theta) &= \log \prod_{i=1}^n \prod_{c=1}^k (\hat{y}_{ic})^{y_{ic}} \\ &= \sum_{i=1}^n \sum_{c=1}^k y_{ic} \log(\hat{y}_{ic}) \end{aligned}$$

Because optimization algorithms conventionally minimize functions, we minimize the negative log-likelihood. This gives us the **Categorical Cross-Entropy Loss**:

$$J(\Theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^k y_{ic} \log \left(\frac{e^{\theta_c^T x_i}}{\sum_{j=1}^k e^{\theta_j^T x_i}} \right) \quad (1.31)$$

Once again, the loss function isn't invented out of thin air; it is the direct, unavoidable mathematical consequence of applying MLE to a Categorical distribution!

1.7.3 The Gradient and Custom Implementations

Because there is no closed-form solution to minimizing Cross-Entropy, we must use an iterative algorithm like Gradient Descent to solve the optimization problem.

```

1 import numpy as np
2
3 def softmax(Z):
4     exp_Z = np.exp(Z)
5     return exp_Z / np.sum(exp_Z, axis=1, keepdims=True)
6
7 def compute_categorical_crossentropy(Y_one_hot, Y_hat):
8     # Add epsilon to prevent log(0) domain errors
9     epsilon = 1e-15
10    Y_hat = np.clip(Y_hat, epsilon, 1 - epsilon)
11    # Sum over classes, mean over samples
12    return -np.mean(np.sum(Y_one_hot * np.log(Y_hat), axis=1))

```

```

13
14 def fit_softmax_regression(X, Y_one_hot, learning_rate=0.01, num_iterations=1000):
15     # Add a column of ones to X to account for the bias term
16     X_b = np.c_[np.ones((X.shape[0], 1)), X]
17     n_samples, n_features = X_b.shape
18     n_classes = Y_one_hot.shape[1]
19
20     # Initialize the parameter matrix to zeros
21     Theta = np.zeros((n_features, n_classes))
22
23     for i in range(num_iterations):
24         # 1. Forward pass: compute logits and probabilities
25         Z = X_b @ Theta
26         Y_hat = softmax(Z)
27
28         # 2. Compute gradient
29         # The gradient of CE with respect to Theta simplifies beautifully
30         gradient = (1 / n_samples) * X_b.T @ (Y_hat - Y_one_hot)
31
32         # 3. Update parameters
33         Theta -= learning_rate * gradient
34
35         # Optional: Print loss every 100 iterations to monitor convergence
36         # if i % 100 == 0:
37         #     loss = compute_categorical_crossentropy(Y_one_hot, Y_hat)
38         #     print(f"Iteration {i}: Cross-Entropy Loss = {loss}")
39
40     return Theta

```

Listing 1.4: Softmax Regression Implementation

1.7.4 The Identifiability Quirk (Overparameterization)

An interesting theoretical quirk of Softmax regression is that its parameters are **not uniquely identifiable**.

Suppose we take our parameter matrix Θ and add an arbitrary constant vector $v \in \mathbb{R}^d$ to every single class's parameter vector (i.e., $\theta_c \rightarrow \theta_c + v$). Let's see what happens to our logits:

$$z_c = (\theta_c + v)^T x = \theta_c^T x + v^T x \quad (1.32)$$

When we plug this shifted logit into the Softmax function:

$$\hat{y}_c = \frac{e^{\theta_c^T x + v^T x}}{\sum_{j=1}^k e^{\theta_j^T x + v^T x}} = \frac{e^{\theta_c^T x} e^{v^T x}}{e^{v^T x} \sum_{j=1}^k e^{\theta_j^T x}} = \frac{e^{\theta_c^T x}}{\sum_{j=1}^k e^{\theta_j^T x}} \quad (1.33)$$

The $e^{v^T x}$ term completely cancels out in the numerator and denominator! This means there are infinitely many parameter matrices Θ that result in the exact same probability predictions. Because the probabilities must sum to 1, the model has $k - 1$ degrees of freedom, not k .

While standard gradient descent will simply converge to one of these valid solutions (which is perfectly sufficient for making predictions), this structural redundancy can be eliminated. We can apply L2 Regularization to strictly penalize the magnitudes of Θ (forcing a unique minimum), or we can hardcode one of the class parameter vectors to be exactly zero ($\theta_k = \vec{0}$), turning it into a baseline pivot class against which all other classes are measured.

Conclusion of Chapter 1: We have journeyed from the naive assumption of a straight line through the rigorous landscapes of calculus, geometry, and probability. We've seen how assumptions about noise lead us to squared errors, how prior beliefs lead us to regularization, and how changing our data distribution leads us to classification. In the next chapter, we will leave the strict confines of linear boundaries and explore a diverse toolkit of classification algorithms!

Chapter 2

Classification Fundamentals

Varnita & Rishi

[Link to session recording](#)

2.1 Warmup: The Art of Drawing Boundaries

In the previous chapter, we focused primarily on regression—predicting a continuous numeric value. But the real world frequently demands discrete decisions. Is this email spam or not? Is this patient healthy or diseased? Is the animal in this image a cat, a dog, or a cow?

This is the domain of **Classification**. Depending on the task, we can categorize these problems into three types:

- **Binary Classification:** Two mutually exclusive classes (e.g., Spam vs. Not Spam).
- **Multiclass Classification:** Three or more mutually exclusive classes (e.g., Cat vs. Dog vs. Cow).
- **Multilabel Classification:** Multiple non-exclusive classes (e.g., a movie tagged as both "Action" and "Comedy").

To solve these problems, we deploy **Classifiers**. At their core, classifiers attempt to draw decision boundaries in our feature space. Linear classifiers (like Logistic Regression or standard SVMs) slice the space with straight hyperplanes. Non-linear classifiers (like K-Nearest Neighbours or Kernel SVMs) can bend and curve to capture highly complex relationships.

In this chapter, we will explore a menagerie of classification algorithms. We will view them through two distinct lenses: the **Probabilistic Lens** (K-Nearest Neighbours, Naive Bayes), which outputs a probability $p \in (0, 1)$, and the **Margin-Based Lens** (Perceptron, SVM), which draws geometric boundaries to maximize the separation between classes.

2.2 K-Nearest Neighbours (KNN)

Imagine that you are trying to estimate the price of a house. You would look at the recently sold houses in your immediate neighbourhood to estimate this. Let us take another example; suppose you moved to a new location. You would choose a restaurant to dine in by asking nearby residents.

This is the principle of **similarity** and forms the foundation of K-Nearest Neighbours (KNN). KNN makes a very simple assumption: *Data points that are similar in their features are likely to share the same label.*

KNN can be used for both classification and regression. In the examples given above, the house price one comes under regression, and the restaurant one under classification.

2.2.1 The Lazy Learner

Unlike regression models or neural networks, KNN does absolutely **zero training**. During the "training" phase, it simply memorizes the entire dataset. It is an *instance-based*, non-parametric model.

All the heavy lifting happens during **inference** (prediction time):

1. Calculate the distance between the unseen query point and *every single point* in the training dataset.
2. Sort the training points by this distance and pick the K closest points.
3. For classification, the predicted class is the majority vote among these K neighbours. For regression, the predicted value is the mean of it's K neighbours.

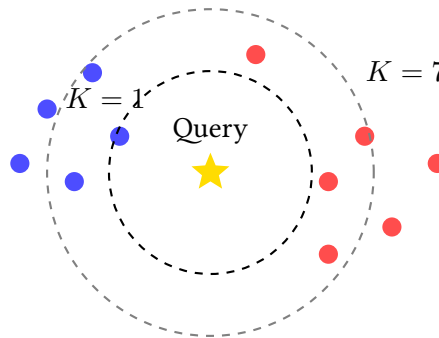


Figure 2.1: In a $K = 1$ scenario, the query point is classified as Blue. In a $K = 7$ scenario, the four Red points outvote three Blue points, changing the prediction to Red.

2.2.2 The Three Hyperparameters of KNN

Because KNN has no internal parameters (like weights or biases), it relies entirely on three user-defined hyperparameters:

1. Choosing 'K': The Bias-Variance Tradeoff

- **Small K (e.g., $K = 1$):** The model trusts the single closest point blindly. It captures tight local structures but is extremely sensitive to noise and outliers. This leads to a jagged decision boundary (**Low Bias, High Variance / Overfitting**).
- **Large K (e.g., $K = 30$):** The model takes a broader consensus. It smooths over noise but might ignore important local patterns, classifying everything as the most frequent class globally (**High Bias, Low Variance / Underfitting**).

Pro Tip: We typically choose an odd value for K to prevent voting ties in binary classification. The optimal K is often found using the Elbow Method via cross-validation.

2. Distance Metrics How do we define "closest"?

- **Euclidean Distance (L_2 Norm):** $\sqrt{\sum (x_i - y_i)^2}$. The default straight-line distance. Works well for continuous features.
- **Manhattan Distance (L_1 Norm):** $\sum |x_i - y_i|$. City-block distance. Highly robust to outliers.
- **Hamming Distance:** $\sum \mathbb{I}(x_i \neq y_i)$. Used for categorical or binary features to count the number of mismatches.

3. **Weighting Schemes** By default, KNN uses **Uniform Weighting**: every neighbor gets exactly 1 vote, regardless of whether it is 1 unit away or 100 units away. In **Distance Weighting**, a neighbor's vote is inversely proportional to its distance ($w_i = 1/d_i$ or $w_i = 1/d_i^2$). A neighbor right next to the query point has a massive influence, while a neighbor barely inside the K -radius has minimal say.

```

1 import numpy as np
2 from collections import Counter
3
4 def predict_knn(X_train, y_train, x_query, k=3):
5     # 1. Calculate Euclidean distances from the query to all training points
6     distances = np.sqrt(np.sum((X_train - x_query)**2, axis=1))
7
8     # 2. Get the indices of the K nearest neighbors
9     nearest_indices = np.argsort(distances)[:k]
10
11    # 3. Extract their labels
12    nearest_labels = y_train[nearest_indices]
13
14    # 4. Return the most common label (Majority Vote)
15    most_common = Counter(nearest_labels).most_common(1)
16    return most_common[0][0]

```

Listing 2.1: KNN Prediction from Scratch using NumPy

2.3 Naive Bayes: A Probabilistic Classifier

KNN relies on geometric distance. But what if we approach classification purely through the mathematics of probability? Enter the Naive Bayes classifier, rooted in the legendary Bayes' Theorem.

Recall Bayes' Theorem:

$$P(H|E) = \frac{P(E|H) \times P(H)}{P(E)} \quad (2.1)$$

Where H is our Hypothesis (the Class) and E is our Evidence (the Features).

- $P(H|E)$ is the **Posterior**: The probability of the class given the features. (What we want to find).
- $P(E|H)$ is the **Likelihood**: The probability of seeing these features if the class is true.
- $P(H)$ is the **Prior**: The baseline probability of the class before seeing any evidence.
- $P(E)$ is the **Evidence**: The total probability of the features occurring. (Often ignored during prediction since it's constant for all classes).

The algorithm is called "Naive" because it makes a massive, mathematically convenient, but logically flawed assumption: *Feature Independence*. It assumes that every feature contributes to the class prediction independently of every other feature. Even though this is rarely true in the real world, the classifier performs exceptionally well in practice.

2.3.1 Multinomial Naive Bayes (Text Classification)

Multinomial Naive Bayes is famously used for text data, like classifying emails as Spam or Not Spam. Here, features are counts of words (Bag of Words).

To predict the class of a new email containing words $x_1, x_2, \dots, x_n$, we multiply the likelihoods of seeing each word in a given class:

$$P(Class_k|x) \propto P(Class_k) \prod_{i=1}^n P(x_i|Class_k) \quad (2.2)$$

The Zero-Frequency Problem & Laplace Smoothing

Imagine the word "dinner" appears in our query email. But what if "dinner" *never* appeared in the Spam training dataset? The likelihood $P(\text{"dinner"}|Spam)$ becomes exactly 0. Because we multiply all probabilities together, this single 0 obliterates the entire equation, forcing the final probability of Spam to be 0, regardless of how many other "spammy" words were present.

To fix this, we apply **Laplace Smoothing**. We add a small pseudo-count (usually $\alpha = 1$) to every word count, ensuring no probability is ever zero:

$$\hat{\theta} = \frac{x_i + \alpha}{N + \alpha d} \quad (2.3)$$

Where x_i is the word count, N is the total words in the class, and d is the number of unique words in the entire vocabulary.

2.3.2 Gaussian Naive Bayes (Continuous Features)

If our features are continuous numbers (like flower petal lengths or human heights), we cannot count them like words. Instead, we assume the features follow a Normal (Gaussian) distribution.

For each class, we calculate the mean μ and variance σ^2 of that feature. The likelihood of a new continuous value x belonging to that class is found by plugging x into the Gaussian Probability Density Function (PDF):

$$P(x|Class_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right) \quad (2.4)$$

2.3.3 The Numeric Underflow Fix: Log Probabilities

A critical computational bottleneck in Naive Bayes is multiplying many tiny probabilities together. E.g., $0.01 \times 0.005 \times 0.0001 \dots$. Very quickly, computers will round this to absolute 0.0, a hardware limitation known as **Numeric Underflow**.

We solve this using logarithms. By taking the log of both sides, we convert multiplications into additions ($\log(a \times b) = \log(a) + \log(b)$). Instead of maximizing the raw probability, we maximize the Log Probability:

$$\log P(Class_k|x) \propto \log P(Class_k) + \sum_{i=1}^n \log P(x_i|Class_k) \quad (2.5)$$

Because the log function is strictly increasing, whichever class has the highest log-probability also has the highest actual probability.

2.4 The Perceptron: The Foundation of Neural Networks

Probabilistic classifiers are excellent, but sometimes we want a rigorous geometric boundary. The Perceptron is the absolute simplest form of a neural network, designed purely for binary classification (labels $y \in \{+1, -1\}$).

2.4.1 The Hyperplane

The Perceptron seeks a flat decision boundary called a **Hyperplane**. In 2D, this is a line; in 3D, a flat plane. The equation for this boundary is:

$$w^T x + b = 0 \quad (2.6)$$

Here, x is our input vector, w is the weight vector, and b is the bias.

Geometric Intuition: The weight vector w is the *normal vector* to the decision boundary. It always points strictly perpendicular to the hyperplane, pointing toward the positive class. When a new point x_i arrives, we compute the dot product $w^T x_i + b$.

- If the result is > 0 , the point is on the positive side of the boundary. Predict $+1$.
- If the result is < 0 , the point is on the negative side. Predict -1 .

2.4.2 The Learning Rule

The perceptron initializes w and b randomly. It iterates through the training data, predicting the class for each point. It *only updates* its weights when it makes a mistake.

If the perceptron predicts $\hat{y}$ but the true label is y , and a mistake was made ($y \neq \hat{y}$), it rotates the boundary by adding or subtracting the misclassified vector x from the weight vector w :

$$w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x \quad (2.7)$$

$$b \leftarrow b + \eta \cdot (y - \hat{y}) \quad (2.8)$$

Where η is the learning rate. Geometrically, if it misclassified a $+1$ point as -1 , it literally pulls the normal vector w closer to x , shifting the hyperplane until x falls on the correct side!

```

1 import numpy as np
2
3 def perceptron_train_step(X, y, w, b, learning_rate=0.1):
4     for i in range(len(X)):
5         # Compute prediction: +1 if w^T x + b >= 0 else -1
6         z = np.dot(w, X[i]) + b
7         y_hat = 1 if z >= 0 else -1
8
9         # Update weights ONLY if a mistake is made
10        if y[i] != y_hat:
11            w = w + learning_rate * (y[i] - y_hat) * X[i]
12            b = b + learning_rate * (y[i] - y_hat)
13
14    return w, b

```

Listing 2.2: Perceptron Update Step

2.5 Support Vector Machines (SVM)

The Perceptron is mathematically guaranteed to find a separating hyperplane if the data is linearly separable in finite iterations. (Proof beyond the scope of this book.) However, there are infinitely many lines that can separate two clusters. The perceptron stops the moment it finds *any* working line, which often leaves the boundary dangerously close to the data points.

What makes a good boundary? A boundary shouldn't just separate classes; it should separate them with maximum confidence. A Support Vector Machine (SVM) finds the **optimal** hyperplane that maximizes the **Margin**—the empty space between the decision boundary and the closest data points on either side.

2.5.1 Hard Margin SVM

These closest points are the only ones that matter; they are the **Support Vectors**. If you delete all other data points, the SVM boundary will not change.

Let our decision boundary be $w^T x + b = 0$. We want the closest positive point to lie on the line $w^T x + b = 1$, and the closest negative point to lie on $w^T x + b = -1$.

Through vector geometry, the total perpendicular distance (margin) between these two lines is precisely $\frac{2}{\|w\|}$. To maximize this margin width, we must minimize $\|w\|$.

This gives us the Hard Margin SVM optimization objective:

$$\min_{w,b} \quad \frac{1}{2} \|w\|_2^2 \quad (2.9)$$

$$\text{subject to} \quad y_i(w^T x_i + b) \geq 1 \quad \forall i \quad (2.10)$$

Take a moment to look at the above formulation and convince yourself that it is exactly what we want.

This is a constrained optimization problem. Unlike our previous models, this is explicitly solved using **Quadratic Programming**, not Gradient Descent!

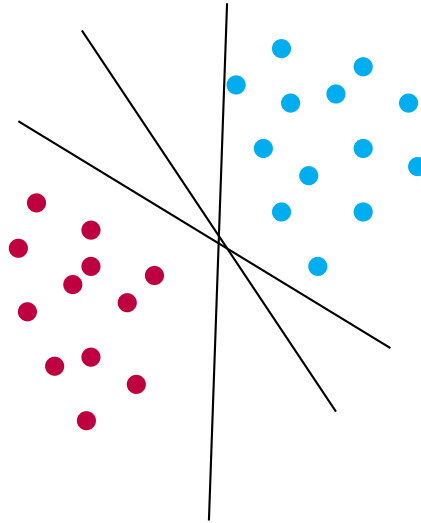


Figure 2.2: Comparison of possible decision boundaries for separating two classes, illustrating how an SVM selects the optimal hyperplane by maximizing the margin between the boundary and the closest data points.

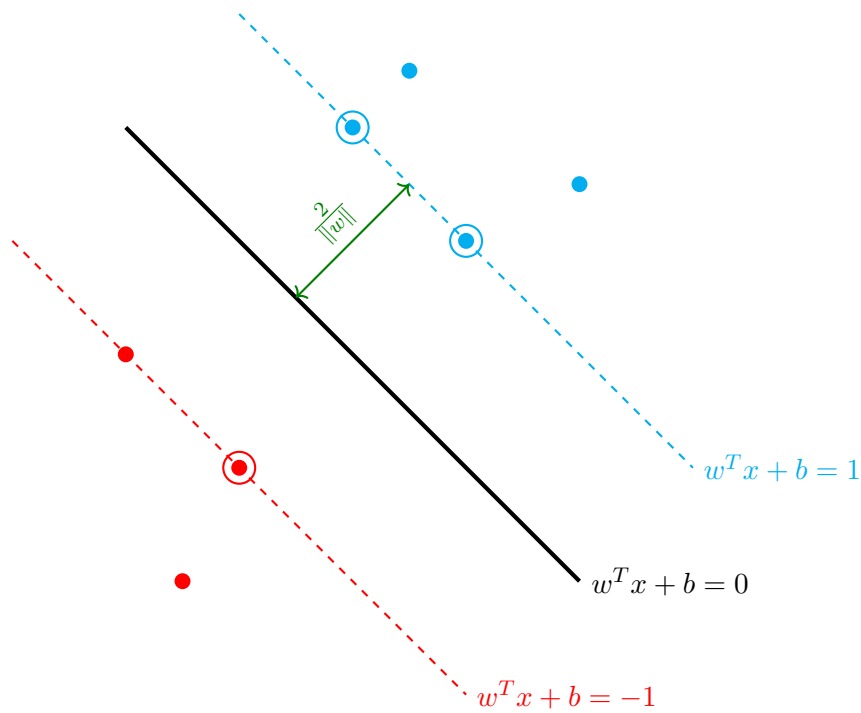


Figure 2.3: The Maximum Margin Hyperplane. The circled points are the Support Vectors.

2.5.2 Soft Margin SVM & The Hinge Loss

Hard Margin SVM strictly assumes the data is perfectly separable. If there is a single outlier crossing the boundary, the optimization becomes mathematically infeasible (no w can satisfy all constraints).

Real-world data is messy. **Soft Margin SVM** relaxes the constraints, allowing some points to violate the margin or even cross the boundary, but penalizing them for doing so. We introduce the **Hinge Loss**:

$$L_i = \max(0, 1 - y_i(w^T x_i + b)) \quad (2.11)$$

If a point is correctly classified and safely outside the margin, its loss is exactly 0. If it violates the margin, the loss increases linearly the further it trespasses.

The new Soft Margin objective function balances maximizing the margin width with minimizing these violations:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b)) \quad (2.12)$$

The C **Hyperparameter** controls the strictness:

- **High C:** The model is terrified of violations. It forces a very narrow margin to classify everything correctly (Risk of Overfitting).
- **Low C:** The model is relaxed. It allows many margin violations to maintain a wide, generalized margin (Better Generalization).

2.5.3 The Kernel Trick

What if the data is fundamentally non-linear, like a circle of red points surrounded by a ring of blue points? No straight line can separate them.

Instead of drawing a curved boundary, SVMs use the **Kernel Trick**. They mathematically project the original 2D data into a higher-dimensional space (e.g., 3D) where a flat plane *can* cleanly slice through it.

The genius of the Kernel Trick is that computing this mapping $\phi(x)$ explicitly is computationally brutal (or impossible for infinite dimensions). However, because the SVM math only ever relies on the *dot product* between pairs of vectors, we can replace the standard dot product with a Kernel Function $K(x, x') = \phi(x) \cdot \phi(x')$. This computes the high-dimensional dot product directly, without ever actually mapping the data!

Common Kernels:

- **Linear Kernel:** Standard SVM. Good for high-dimensional, sparse data (like text).
- **Polynomial Kernel:** $K(u, v) = (u \cdot v + c)^d$. Captures feature interactions up to degree d .
- **RBF (Gaussian) Kernel:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. Measures similarity by distance. Points close together are "similar". It mathematically maps data to an infinite-dimensional space! This is the most popular default kernel.

Conclusion of Chapter 2: We have explored several popular classification algorithms like kNN, Naive Bayes, Perceptron and SVM, what assumptions they work under and what their advantages and disadvantages are. In the next chapter, we will look at a new approach that can be used for both classification or regression: trees.

Chapter 3

Tree-Based Models & Ensembles

Shloka & Bhavana

[Link to session recording](#)

3.1 Warmup: The Wisdom of the Crowd

In the previous chapter, we spent a lot of time trying to find the perfect mathematical line or plane to divide our data. But what if we stopped trying to draw complex geometric shapes and instead just started asking simple "yes/no" questions?

Furthermore, what if we didn't just ask one person to make predictions, but instead gathered a room full of 100 people? Imagine trying to predict the outcome of a massive football tournament. If you rely on just one expert, their prediction will likely be skewed by personal biases or an over-reliance on specific statistics. However, if you gather 100 people (some who analyze defense, some who look at historical trends, and some who just guess based on recent form) and average their predictions together, the collective guess will almost always be far more accurate and stable than the single smartest person in the room.

This is the core philosophy behind **Ensemble Learning**: combining multiple "weak learners" (simple models) to create one incredibly accurate "strong learner." But before we can build a forest, we must first understand how to grow a single tree.

3.2 Decision Trees: The Universal Function Approximator

A Decision Tree is exactly what it sounds like: a hierarchical structure of nested if-else statements. It evaluates a feature, asks a question (e.g., "Is Age < 30?"), and routes the data down a branch based on whether the answer is True or False.

3.2.1 Why Decision Trees?

Decision Trees are beloved in Machine Learning for several profound reasons:

- **High Interpretability:** A decision tree produces a simple, visual, human-readable path. We can trace exactly why a model made a specific prediction.
- **Non-parametric:** They make absolutely no assumptions about the distribution of the data. They don't care if the underlying relationship is linear, polynomial, or completely bizarre.
- **Minimal Pre-processing:** Unlike Neural Networks or Support Vector Machines, Decision Trees do not require data normalization or scaling. Because a tree simply splits data at specific thresholds (e.g., $X_1 > 10$), the scale of X_1 is irrelevant. They can also natively handle categorical variables without One-Hot Encoding and are robust to missing values.

3.2.2 The Geometric View: Axis-Aligned Splits

While SVMs draw diagonal lines, Decision Trees can only ever produce **axis-aligned rectangular regions**. Every time a tree asks a question about a single feature, it is drawing a straight line perpendicular to that feature's axis.

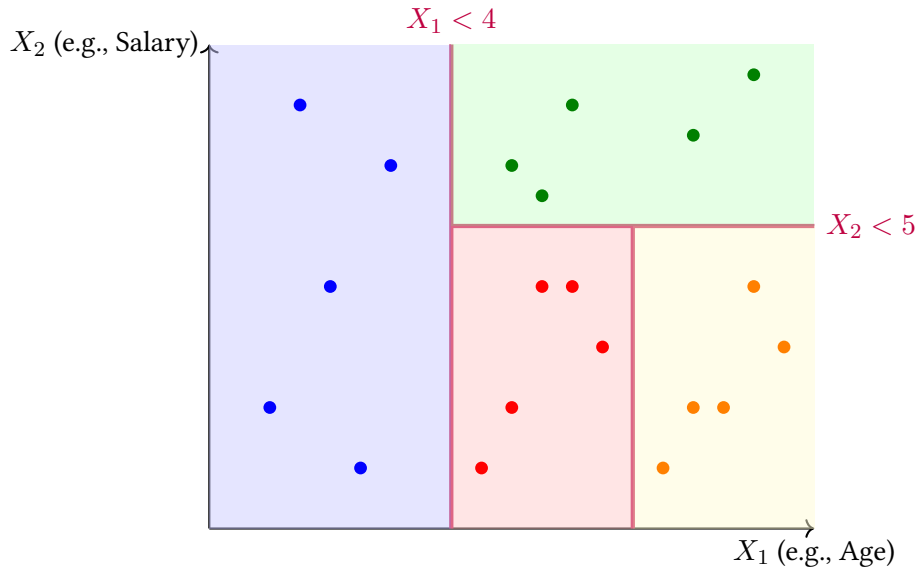


Figure 3.1: A Decision Tree recursively dividing the feature space into axis-aligned boxes.

This is a direct geometric consequence of looking at only one feature at a time. It is exactly why a single Decision Tree struggles with data where the true boundary is diagonal (it would need a "staircase" of many tiny splits to approximate a 45-degree line).

3.3 The Math of Splitting: Gini and Entropy

The entire algorithm for building a Decision Tree is:

1. Figure out which feature (and what threshold) is the "best" to split the data.
2. Split the data into two smaller datasets (Left and Right children).
3. Repeat recursively until a stopping condition is met (e.g., maximum depth or a perfectly pure leaf).

But how do we define the "best" split? We cannot simply use *Accuracy*. A node containing 51% Class A and 49% Class B has the exact same accuracy as a node containing 99% Class A and 1% Class B, because both will simply output a majority vote of "Class A". We need a metric that rewards the algorithm for untangling the data into *purier* subsets step-by-step.

3.3.1 Gini Impurity

Gini Impurity measures the diversity or chaos of a dataset. Specifically, it asks: *If I randomly pick two elements from this dataset, what is the probability that they are of different classes?*

For a dataset with n classes, where p_i is the proportion of elements belonging to class i , the probability of picking two elements of the *same* class is $\sum_{i=1}^n p_i^2$. Therefore, the probability of picking two elements of *different* classes (the Impurity) is:

$$Gini = 1 - \sum_{i=1}^n p_i^2 \quad (3.1)$$

- A perfectly pure node (e.g., 100% Class A) has a Gini of $1 - (1^2 + 0^2) = 0$.
- A perfectly mixed binary node (50% Class A, 50% Class B) has a Gini of $1 - (0.5^2 + 0.5^2) = 0.5$.

To evaluate a split, we calculate the **Weighted Average Gini Impurity** of the child nodes. We multiply each child's impurity by the fraction of the total data points it received. The split that results in the lowest weighted Gini Impurity is chosen.

Note: Gini is the default metric in libraries like Scikit-Learn because it does not require computing computationally expensive logarithms.

3.3.2 Entropy and Information Gain

Entropy approaches the same problem from the perspective of Information Theory. It measures the amount of "surprise" or uncertainty in the data.

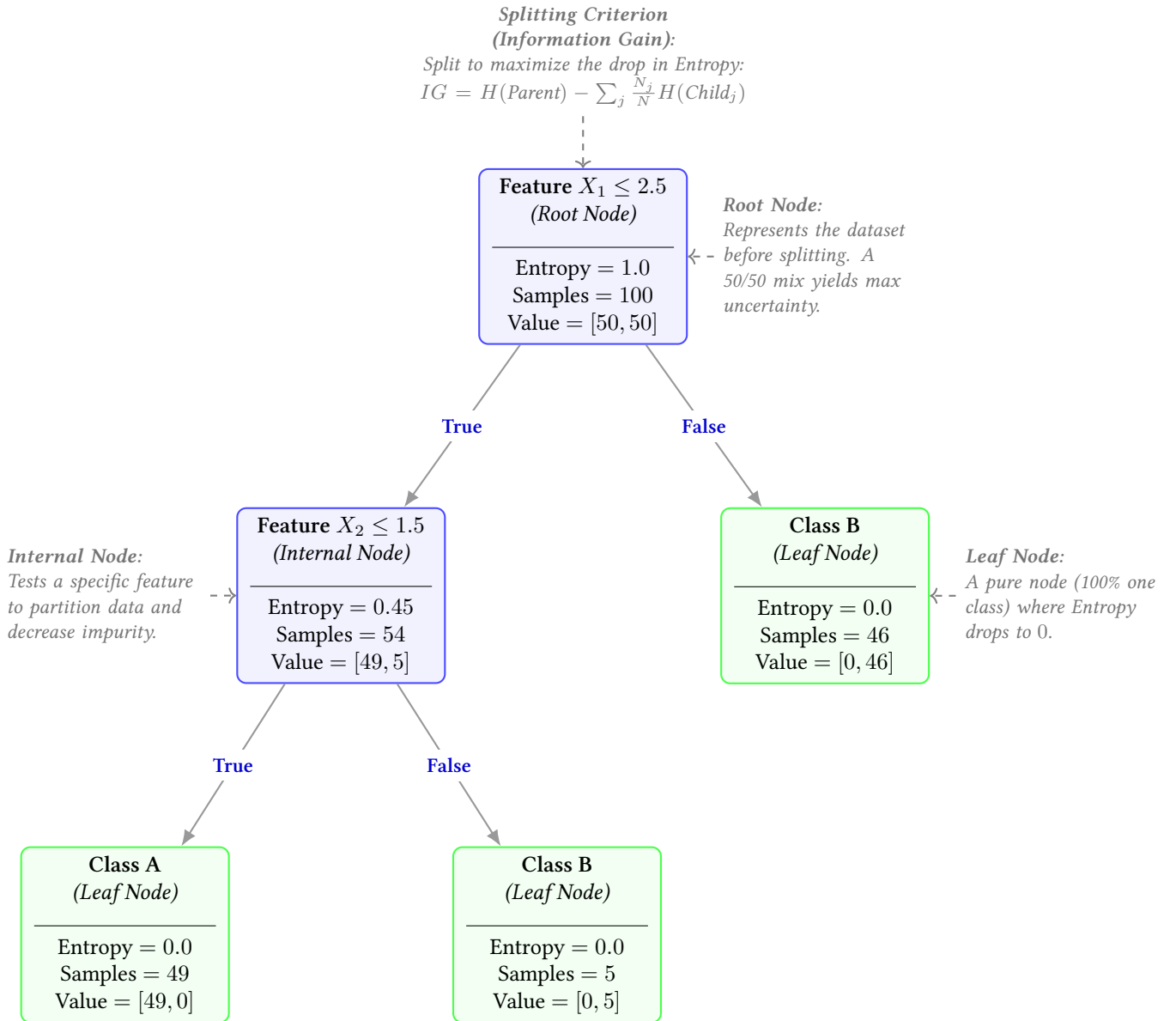
If a dataset is 100% Class A, there is no surprise when we draw an element. If it's a 50/50 split, surprise is maximized. We quantify this using the negative logarithm of the probability:

$$Entropy = H(X) = - \sum_{i=1}^n p_i \log_2(p_i) \quad (3.2)$$

When we split a parent node into child nodes, we want to maximize the drop in Entropy. This drop is called **Information Gain (IG)**:

$$IG = H(\text{Parent}) - \sum_j \frac{N_j}{N} H(\text{Child}_j) \quad (3.3)$$

3.3.3 An example of a decision tree



3.3.4 Regression Trees

What if our target y is a continuous number (e.g., house price) instead of a category? The tree becomes a **Regression Tree**. Instead of calculating Gini or Entropy, the tree evaluates splits by minimizing the **Mean Squared Error (MSE)**.

For a proposed split, the prediction for a leaf is simply the average of all the continuous values in that leaf. The tree computes the weighted sum of the MSE for the Left and Right splits:

$$J(s) = \frac{n_{left}}{n} MSE_{left} + \frac{n_{right}}{n} MSE_{right} \quad (3.4)$$

It then chooses the split point s that minimizes $J(s)$.

3.4 Ensemble Learning Fundamentals

A single Decision Tree is inherently prone to **overfitting** (high variance). Left unconstrained, a decision tree recursively partitions the feature space until it perfectly memorizes the training set, fitting both the

underlying signal and the stochastic noise. To mitigate this high variance while preserving predictive capacity, we combine multiple base models into an **Ensemble**.

An ensemble aggregates predictions from multiple base learners (e.g., individual decision trees) to form a composite model whose collective prediction routinely outperforms any individual constituent.

3.4.1 Error Decomposition: The Bias-Variance Tradeoff

As we saw earlier in Chapter 1, the total expected generalization error of a machine learning model decomposes into three additive terms:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \sigma^2 \quad (3.5)$$

where σ^2 is the irreducible error inherent in the data generation process.

- **Bias:** Systematic error resulting from overly simplistic assumptions in the model family (underfitting).
- **Variance:** Error stemming from sensitivity to minor fluctuations or noise in the training set (overfitting).

Different ensemble strategies target distinct components of this error: **Bagging** primarily reduces variance by averaging uncorrelated models, whereas **Boosting** systematically reduces bias by sequentially correcting prediction errors.

- **Arithmetic Average (Regression):** For continuous target values, the ensemble prediction $\hat{y}$ is the simple mean of M individual model outputs:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m(x) \quad (3.6)$$

- **Weighted Average (Regression):** When models possess varying levels of reliability or validation accuracy, predictions are weighted by a trust factor $w_m \geq 0$ (where $\sum w_m = 1$):

$$\hat{y} = \sum_{m=1}^M w_m \cdot \hat{y}_m(x) \quad (3.7)$$

- **Hard Voting / Majority Rule (Classification):** Each base learner casts a discrete vote for a class label $c \in \{1, \dots, C\}$. The class receiving the absolute majority wins:

$$\hat{y} = \arg \max_c \sum_{m=1}^M \mathbb{I}(\hat{y}_m(x) = c) \quad (3.8)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

- **Soft Voting / Probability Averaging (Classification):** Base learners output predicted class probabilities $P_m(y = c | x)$. The ensemble computes the average probability per class and selects the highest:

$$\hat{y} = \arg \max_c \frac{1}{M} \sum_{m=1}^M P_m(y = c | x) \quad (3.9)$$

Soft voting is generally preferred over hard voting because it weights high-confidence predictions more heavily than marginal ones.

Ensemble methods generally fall into two distinct families: **Bagging** (Parallel) and **Boosting** (Sequential).

3.5 Bagging & Random Forests

Bootstrap Aggregating (Bagging) reduces variance by training multiple models independently in parallel, and then averaging their predictions.

3.5.1 Bootstrapping

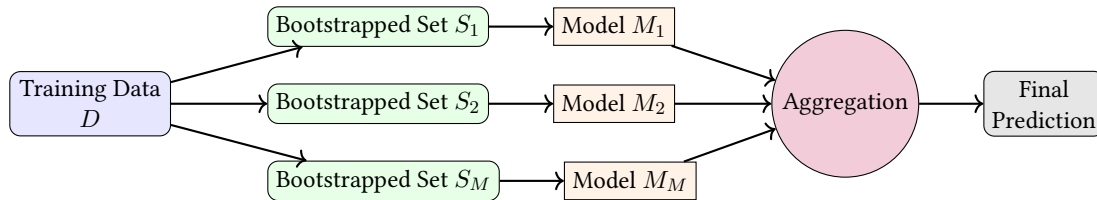


Figure 3.2: Parallel Ensemble Architecture (Bagging): Models are trained independently on distinct bootstrapped sub-datasets.

3.5.2 Bootstrapping and Out-Of-Bag (OOB) Estimation

For each of our M models, we create a new training dataset by randomly drawing N samples from the original dataset. Because we replace the sample after drawing it, some original data points will appear multiple times in a single bootstrapped dataset, while others (roughly 37%) will be left out entirely.

Mathematically, the probability that a specific data point is *not* selected in a single draw is $1 - \frac{1}{N}$. For N independent draws, the probability that a sample is excluded from a bootstrapped set is:

$$P(\text{Excluded}) = \left(1 - \frac{1}{N}\right)^N \xrightarrow{N \rightarrow \infty} \frac{1}{e} \approx 0.368 \quad (36.8\%) \quad (3.10)$$

Thus, roughly 63.2% of unique samples end up in any given bootstrapped dataset, while the remaining $\sim 36.8\%$ are left out.

These left-out points are called **Out-Of-Bag (OOB)** samples and can be used as a built-in validation set to test the model without needing a separate hold-out set!

3.5.3 Random Forests

A Random Forest is a Bagged Ensemble of Decision Trees with one additional trick: **Feature Subsetting**.

Even with Bootstrapping, if there is one highly dominant feature in the dataset, almost every tree will choose to split on that feature first. The trees will become highly correlated. To force the trees to be truly diverse, a Random Forest restricts the features available at each split. When evaluating a node, the tree is only allowed to look at a random subset (usually $\sqrt{d}$) of the features.

This ensures that the forest is composed of highly diverse, uncorrelated trees. When you average their predictions, the variance drops dramatically. *Note: Bagging reduces variance, but it does not fix high bias. If the base models are too simple, the ensemble will still underfit.*

3.6 Boosting: AdaBoost to XGBoost

While Bagging trains models in parallel independently, **Boosting** trains models *sequentially*. Each new model in the sequence specifically attempts to correct the errors made by the models before it.

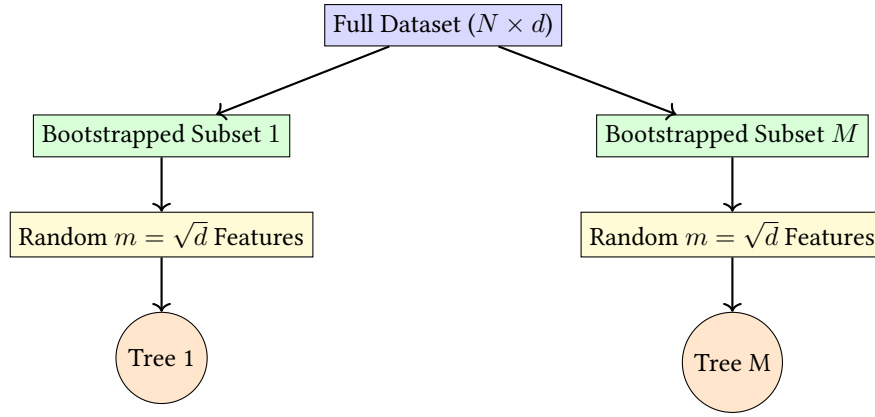


Figure 3.3: Random Forest Feature Subsetting: Forcing trees to consider only $m \ll d$ random features at each split guarantees tree diversity.

3.6.1 AdaBoost (Adaptive Boosting)

AdaBoost typically uses **Decision Stumps** (a tree with a depth of exactly 1; just a single root node and two leaves) as its weak learners.

1. Initially, all N data samples are given an equal weight of $1/N$.
2. A stump is trained on the data.
3. We calculate the stump's **Amount of Say** (α) based on its Total Error (the sum of the weights of the misclassified samples):

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right) \quad (3.11)$$

4. We **Update the Sample Weights**. We exponentially increase the weights of the misclassified samples, and exponentially decrease the weights of correctly classified samples.

$$w_{new} = w_{old} \times e^{\pm\alpha} \quad (3.12)$$

5. We normalize the weights so they sum to 1, and train the next stump. Because the misclassified points now have massive weights, the next stump is forced to focus heavily on getting them right!

3.6.2 Gradient Boosting Machines (GBM)

Instead of adjusting sample weights, Gradient Boosting fits subsequent models directly to the **Residual Errors** (the difference between the prediction and the truth) of the previous models.

1. Start with a naive initial prediction, F_0 (usually the mean of the target variable).
2. Calculate the residual errors: $r_1 = y - F_0$.
3. Train a new Regression Tree to predict these residuals (not the original targets!).
4. Update the overall model by adding the new tree's predictions, scaled by a **Learning Rate** ($\nu \in (0, 1]$) to prevent immediate overfitting:

$$F_1 = F_0 + \nu \cdot \text{Tree}_1(x) \quad (3.13)$$

5. Calculate new residuals $r_2 = y - F_1$ and repeat!

3.6.3 XGBoost (eXtreme Gradient Boosting)

XGBoost is an immensely powerful, highly optimized implementation of Gradient Boosting. It dominated Kaggle competitions for years due to its speed and mathematical refinements.

Crucially, XGBoost introduces **Regularization** (L1 and L2 penalties) directly into the tree-building process to heavily restrict overfitting. It evaluates the quality of a leaf using a **Similarity Score**:

$$\text{Similarity Score} = \frac{(\sum \text{residuals})^2}{\text{Number of Residuals} + \lambda} \quad (3.14)$$

Here, λ is the L2 Regularization parameter. If a leaf contains conflicting residuals (e.g., +5 and -5), they cancel out, resulting in a low score. The algorithm wants leaves where the residuals are homogeneous.

To decide if a split is worth making, XGBoost calculates the **Gain**:

$$\text{Gain} = \text{Similarity}_{\text{Left}} + \text{Similarity}_{\text{Right}} - \text{Similarity}_{\text{Root}} \quad (3.15)$$

If the Gain is negative (or less than a required parameter γ), the tree is pruned back.

Listing 3.1 demonstrates how to instantiate and fit Random Forest (Bagging) and Gradient Boosting models using Python's `scikit-learn` library.

```

1 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4
5 # Generate dummy dataset
6 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
8
9 # 1. Random Forest Classifier (Bagging with Feature Subsetting)
10 rf_model = RandomForestClassifier(
11     n_estimators=100,
12     max_features='sqrt',
13     oob_score=True,
14     random_state=42
15 )
16 rf_model.fit(X_train, y_train)
17
18 print(f"Random Forest Training Accuracy: {rf_model.score(X_train, y_train):.4f}")
19 print(f"Out-Of-Bag (OOB) Accuracy Score: {rf_model.oob_score_: .4f}")
20
21 # 2. Gradient Boosting Classifier (Sequential Boosting)
22 gbm_model = GradientBoostingClassifier(
23     n_estimators=100,
24     learning_rate=0.1,
25     max_depth=3,
26     random_state=42
27 )
28 gbm_model.fit(X_train, y_train)
29
30 print(f"GBM Test Accuracy Score: {gbm_model.score(X_test, y_test):.4f}")

```

Listing 3.1: Implementation of Ensemble Models in Scikit-Learn

Conclusion of Chapter 3: We have evolved from single, highly-interpretable decision trees into massive, mathematically complex forests. We've seen how Bagging reduces variance through democracy and random subsets, and how Boosting reduces bias by relentlessly learning from its past mistakes. But what happens when we don't have the answers? In the next chapter, we will strip away our labels entirely and venture into the wilderness of Unsupervised Learning!

Chapter 4

Unsupervised Learning

Varnita & Hriday

[Link to session recording](#)

4.1 Warmup: Venturing into the Unknown

Up until this point, we have operated with a safety net. Whether predicting continuous house prices or classifying spam emails, we always had a target variable y to guide our models. We fed the algorithm the data X , let it make a guess, and then corrected it using the "right answers" y . This is Supervised Learning.

But the real world is messy. Labeling data is expensive, time-consuming, and often impossible at a massive scale. In fact, over 90% of the world's data is completely unlabelled. There is no y .

When we drop the y , we enter the realm of **Unsupervised Learning**. We can no longer ask the model to "Predict y ." Instead, we ask it to "Find the hidden structure inside X ." Unsupervised learning is the pure art of discovery. It generally falls into three massive categories:

1. **Clustering**: Grouping data points so that points in the same group are more similar to each other than to those in other groups (e.g., Customer Segmentation).
2. **Dimensionality Reduction**: Compressing the data by reducing the number of features while retaining maximum information (e.g., Data Visualization, Noise Removal).
3. **Association Rule Learning**: Discovering interesting co-occurrences (e.g., "People who buy diapers also tend to buy beer").

4.1.1 The Golden Rule of Clustering

To group data, we must first mathematically define what makes two points "similar." The absolute foundation of clustering is distance which determines the similarity of two points.

- **Euclidean Distance (L_2)**: The straight-line distance, heavily sensitive to scale. $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$
- **Manhattan Distance (L_1)**: City-block distance, robust to outliers. $d(x, y) = \sum |x_i - y_i|$
- **Cosine Similarity**: Measures the angle between vectors, entirely ignoring magnitude. Highly effective for sparse data like text. $S_c(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$

Regardless of the metric chosen, every good clustering algorithm strives to satisfy **The Golden Rule of Clustering**:

Maximize Intra-cluster Similarity (Tight Groups)
Minimize Inter-cluster Similarity (Far Apart Groups)

4.2 K-Means & The Centroid

K-Means is the most famous centroid-based clustering algorithm. It seeks to partition n observations into k clusters, where each observation belongs to the cluster with the nearest mean (centroid).

4.2.1 The Objective: Inertia

K-Means mathematically enforces the "Tight Groups" half of the Golden Rule by minimizing **Inertia**, also known as the Within-Cluster Sum of Squares (WCSS). Let μ_k be the centroid of cluster C_k :

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2 \quad (4.1)$$

While Inertia is intuitive, it has severe geometric limitations. It inherently assumes that clusters are convex and isotropic (spherical). It performs terribly on elongated clusters or irregular manifolds. Furthermore, because it relies on non-normalized Euclidean distances, it is highly susceptible to the *Curse of Dimensionality*.

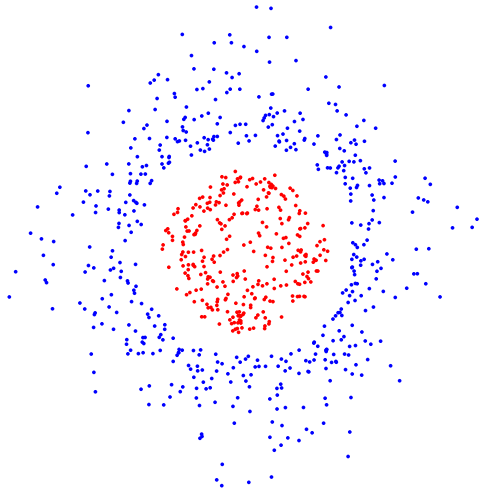


Figure 4.1: K-Means assumes convex, spherical clusters, struggling to capture the non-convex structure of the blue cluster surrounding the red cluster.

4.2.2 Lloyd's Algorithm

Minimizing WCSS is an NP-hard problem. Therefore, K-Means uses an elegant heuristic approach known as Lloyd's Algorithm, which guarantees convergence to a local minimum:

1. **Initialization:** Randomly select k data points to act as the initial centroids.
2. **Assignment:** Calculate the distance from every data point to every centroid. Assign each data point to the cluster of its closest centroid.
3. **Update:** For each new cluster, compute the mean of all its assigned data points. Move the centroid to this new mean.
4. **Repeat:** Repeat Steps 2 and 3 until the centroids stop moving (convergence).

4.2.3 The Initialization Problem & K-Means++

If Lloyd's Algorithm starts with a bad set of random centroids (e.g., two centroids initialized right next to each other), it will get trapped in a terrible local minimum.

K-Means++ solves this by spreading out the initial centroids mathematically.

1. Pick the first centroid μ_1 completely at random from the data.
2. For every other data point x_i , compute its distance $D(x_i)$ to the *nearest* already-chosen centroid.
3. Choose the next centroid randomly from the data points, but weigh the probability of choosing point x_i proportional to $D(x_i)^2$.

$$P(x_i) = \frac{D(x_i)^2}{\sum_{j=1}^N D(x_j)^2} \quad (4.2)$$

This brilliantly ensures that points furthest away from existing centroids have an exponentially higher chance of being selected as the next centroid, leading to drastically faster and more stable convergence.

4.2.4 Choosing the Optimal 'K'

K-Means forces you to guess k upfront. How do we know if we guessed right?

- **The Elbow Method:** Plot k on the x-axis and WCSS on the y-axis. WCSS will always decrease as k increases. The optimal k is at the "elbow" of the curve, where the rate of decrease shifts from rapid to slow.
- **Silhouette Score:** Evaluates cohesion vs. separation. For a point i , let $a(i)$ be the average distance to points in its own cluster, and $b(i)$ be the average distance to points in the nearest neighboring cluster.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4.3)$$

Scores approach +1 for perfect clustering, 0 for overlapping boundaries, and -1 for incorrect assignments.

4.3 Hierarchical Clustering

What if we don't want to guess k at all? Hierarchical clustering builds a massive tree of connections, known as a **Dendrogram**, allowing us to see the entire relational structure of the data and choose k *after* the fact.

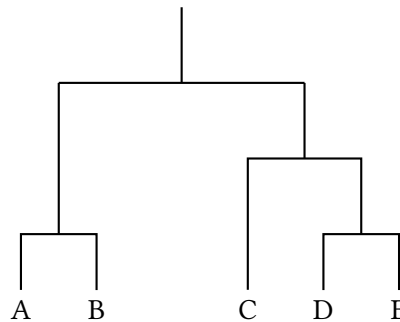


Figure 4.2: A Dendrogram.

4.3.1 Agglomerative vs. Divisive

There are two ways to build the tree:

- **Agglomerative (Bottom-Up):** Start with every single data point as its own cluster. Iteratively find the two most similar clusters and merge them, repeating until only one massive cluster remains. (Standard approach).
- **Divisive (Top-Down):** Start with all data in one massive cluster. Recursively split it into smaller groups by finding the most dissimilar points, stopping when every point is isolated.

4.3.2 Linkage Criteria

To merge clusters in the Agglomerative approach, we must compute the distances between them. But how do you measure the distance between two amorphous blobs of points, Cluster A and Cluster B?

- **Single Linkage (Min):** The distance between A and B is the shortest distance between *any* point in A and *any* point in B. Highly efficient for non-globular data but prone to "chaining" (creating long, snake-like clusters).
- **Complete Linkage (Max):** The distance is the maximum distance between any point in A and any point in B. Favors tight, spherical clusters.
- **Average Linkage:** The arithmetic mean of all pairwise distances between points in A and points in B.

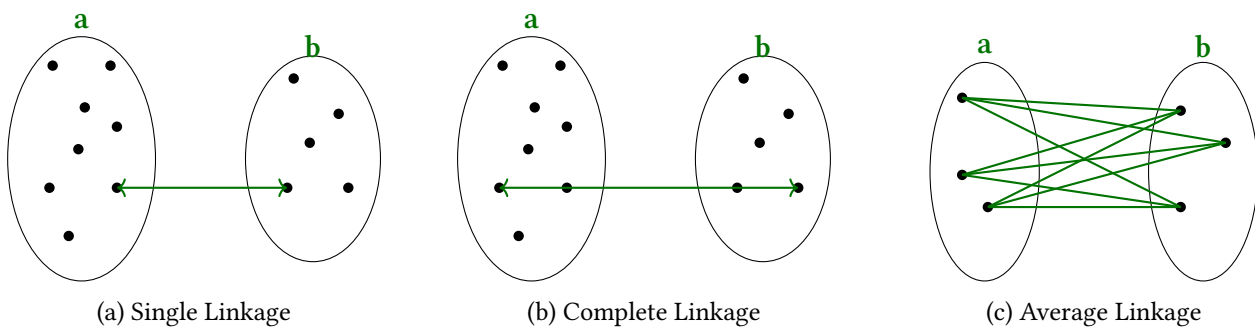


Figure 4.3: Illustration of the three linkage criteria between clusters a and b .

The brilliance of the dendrogram is the **Threshold Cut**. A horizontal line drawn at any height explicitly dictates the number of clusters (based on how many vertical lines it intersects). A massive vertical gap before a merge indicates highly distinct natural clusters.

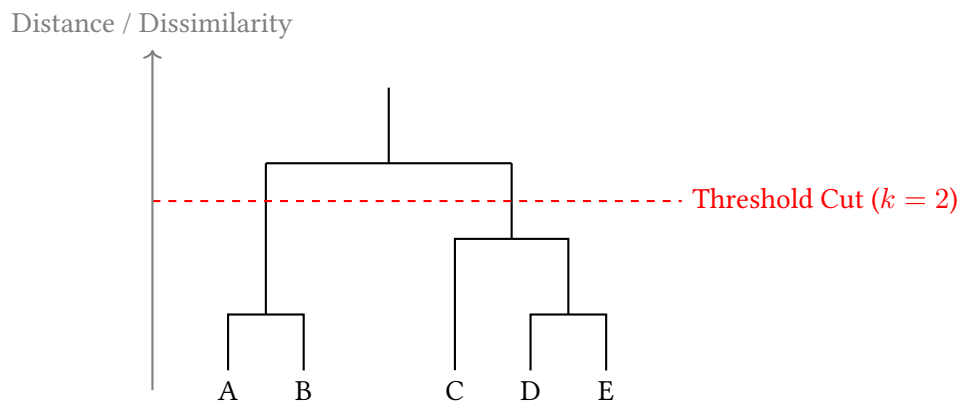


Figure 4.4: A Dendrogram. Cutting at the red dashed line results in exactly two clusters: $\{A, B\}$ and $\{C, D, E\}$.

4.4 DBSCAN: A Density-Based Approach

Both K-Means and Hierarchical clustering struggle severely with noise and non-spherical clusters (like two interlocking half-moons). DBSCAN (Density-Based Spatial Clustering of Applications with Noise) throws away centroids entirely and views clusters as dense regions of space separated by empty voids.

4.4.1 The Two Parameters

DBSCAN relies on just two hyperparameters:

1. ϵ (**Epsilon**): The radius of the neighborhood around a point.
2. **MinPts**: The minimum number of points required within the ϵ -radius to consider that region "dense."

4.4.2 The Three Types of Points

Based on these parameters, DBSCAN strictly categorizes every single point in the dataset:

- **Core Points**: Points that have at least *MinPts* neighbors within their ϵ -radius. They are the dense hearts of the clusters.
- **Border Points**: Points that have fewer than *MinPts* neighbors, but fall within the ϵ -radius of a Core Point. They form the outer edges of a cluster.
- **Noise (Outliers)**: Any point that is neither a Core Point nor a Border Point. DBSCAN explicitly isolates them, refusing to force them into a cluster!

The Algorithm Workflow: The algorithm begins at a random Core Point, expanding the cluster by linking to all density-reachable Core points (other Core points within ϵ). Then expands to all density-reachable Non-Core points. Once a cluster cannot expand further, it moves to the next unvisited Core Point. Any point left untouched at the end is branded as an outlier.

If ϵ (the radius) is too small, your data becomes heavily fragmented with most points incorrectly flagged as noise, but if it is too large, distinct groups will merge together into a single cluster that swallows up actual outliers. Conversely, if your *MinPts* (minimum points) threshold is too small, the algorithm is easily fooled into building clusters out of sparse, random noise; however, if *MinPts* is too large, the algorithm becomes overly strict, completely ignoring legitimate but less-dense clusters and treating them as noise instead.

```

1 from sklearn.cluster import KMeans, DBSCAN
2 import numpy as np
3
4 # X is our unlabelled dataset
5 X = np.random.rand(100, 2)
6
7 # K-Means requires specifying k upfront
8 kmeans = KMeans(n_clusters=3, init='k-means++')
9 kmeans_labels = kmeans.fit_predict(X)
10
11 # DBSCAN discovers k dynamically and identifies outliers (labelled as -1)
12 dbscan = DBSCAN(eps=0.1, min_samples=5)
13 dbscan_labels = dbscan.fit_predict(X)

```

Listing 4.1: Comparing K-Means and DBSCAN in Scikit-Learn

4.5 Dimensionality Reduction & Principal Component Analysis

If clustering is the art of grouping data, dimensionality reduction is the art of *compressing* it.

In the real world, datasets often contain hundreds or even thousands of features. As the number of features grows, the data space grows exponentially, causing data to become sparse and making patterns harder to find. This phenomenon is known as the **Curse of Dimensionality**. High-dimensional data requires massive processing power and memory, and it severely increases the chance of overfitting. Furthermore, humans can only interpret data in two or three dimensions; reduction allows us to graph, visually inspect, and understand complex datasets. Finally, stripping away the least important variables helps eliminate random noise and redundancies, revealing the true underlying structure of the data.

Methods to reduce dimensionality include Principal Component Analysis (PCA), autoencoders, Singular Value Decomposition (SVD), and others. In this section, we focus on PCA—the cornerstone of linear dimensionality reduction.

4.5.1 What is PCA? The “Best Camera Angle” Analogy

Imagine you are photographing a highly complex three-dimensional sculpture. If you take a photo from the side, it might look like a single flat line. If you take it from the front, you might only see a circle. **Principal Component Analysis (PCA)** is a mathematical algorithm designed to find the absolute *best camera angle*—the one specific projection that captures the maximum “spread” or variance of the object in a single two-dimensional snapshot, eliminating redundant information.

Formally, PCA finds a new coordinate system (a rotation of the original axes) that best summarizes the information without unnecessary redundancy. The axes of this new system are called **Principal Components**.

PCA finds the axes of maximum variance in 3D space

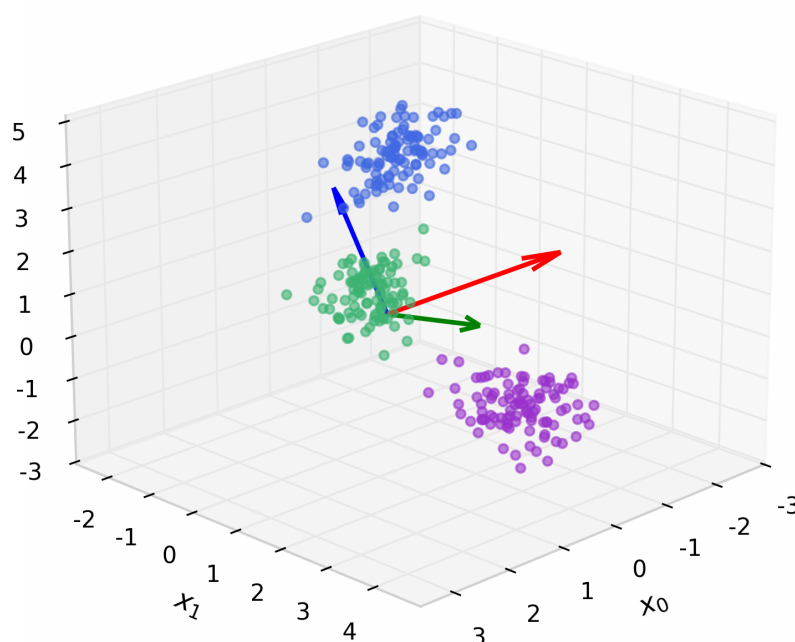


Figure 4.5: PCA finds the axes (arrows) along which the data varies the most. The blue arrow represents the first principal component, capturing the direction of maximum variance.

4.5.2 A Worked Example: Midterm and Endterm Marks

To build intuition, let us begin with a simple two-dimensional dataset. We have measured the midterm and endterm marks of six different students, shown in Table 4.1.

Student	A	B	C	D	E	F
Midterm (M)	35	45	28	88	78	92
Endterm (E)	30	22	42	75	95	80

Table 4.1: Midterm and endterm marks for six students.

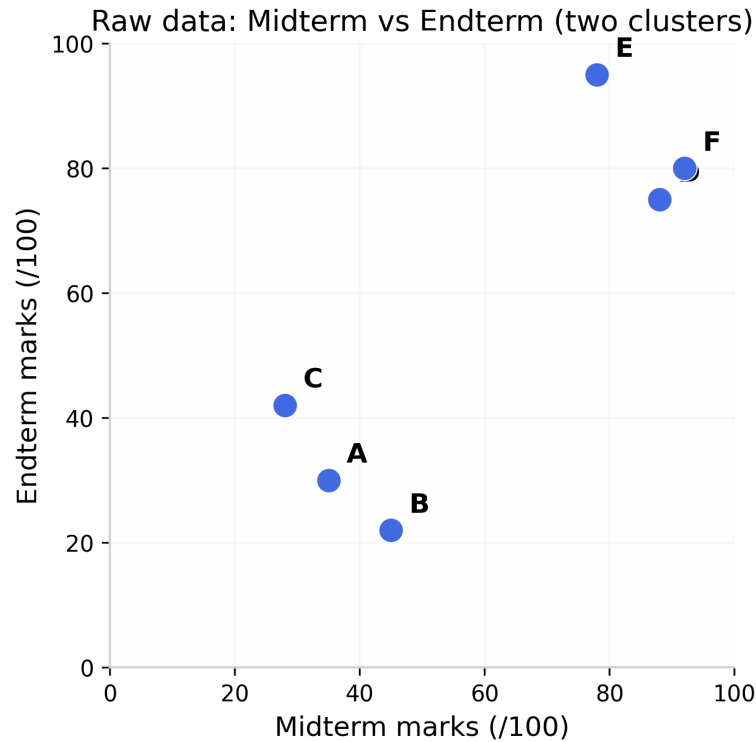


Figure 4.6: Raw data plotted in the original feature space. Two natural clusters are visible: students who scored poorly on both exams (A, B, C) and students who scored well (D, E, F).

Step 1: Centering the Data

The first step in PCA is strictly required: we must center the data. We calculate the mean of each feature and shift the entire dataset so that the mean sits exactly at the origin (0, 0).

For our data, the means are:

$$\bar{M} = \frac{35 + 45 + 28 + 88 + 78 + 92}{6} = 61, \quad \bar{E} = \frac{30 + 22 + 42 + 75 + 95 + 80}{6} \approx 57.3.$$

Subtracting these means from every observation gives the centered coordinates shown in Figure 4.7. Notice that centering does not change the relative distances between points; it merely shifts the coordinate system.

Step 2: Scaling (When Necessary)

In our current example, we have not scaled the data because both midterm and endterm are measured on the same 100-point scale. However, in cases where feature units differ—such as height in centimeters versus weight in kilograms, or age versus salary—scaling is absolutely crucial.

PCA fundamentally relies on variance. If one feature has vastly larger raw numbers than another, it will mathematically dominate the principal components simply because its variance is larger, not because it is more important. Standardizing features to unit variance forces all features onto a level playing field before PCA is applied.

Step 3: Finding the Best-Fit Line

After centering, PCA seeks to draw a straight line through the origin that “fits” the data best. But what does “best” mean?

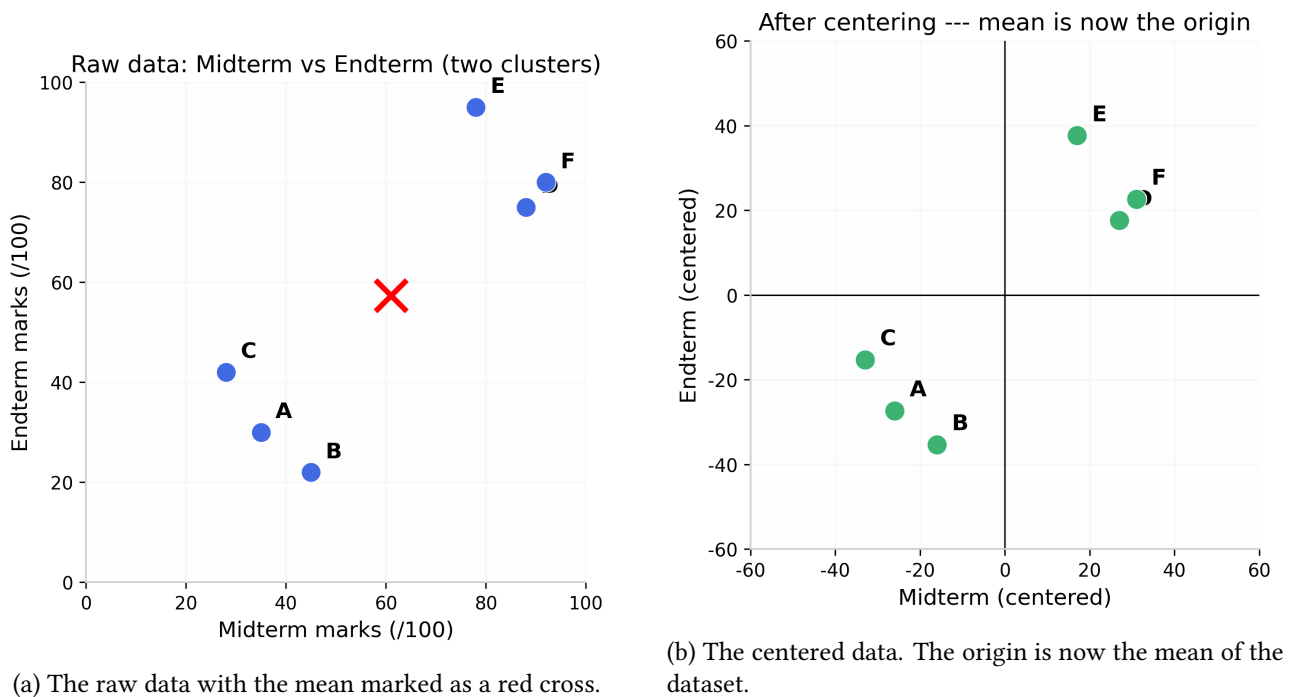


Figure 4.7: Centering transforms the data without distorting its internal geometry.

Consider any candidate line passing through the origin. We project every data point perpendicularly onto this line, as shown in Figure 4.8. PCA seeks the specific line that **maximizes the sum of the squared distances from the projected points to the origin**.

At first glance, one might think we should minimize the perpendicular error distances from the points to the line. Remarkably, these two objectives are identical. Consider a single data point, as illustrated in Figure 4.9:

- Let a be the fixed distance from the data point to the origin.
- Let b be the perpendicular distance from the data point to the line.
- Let c be the distance from the projected point to the origin.

By the Pythagorean theorem, $a^2 = b^2 + c^2$. Since a is fixed for any given point, maximizing c is mathematically identical to minimizing b .

Principal Component 1

The optimal line is called **Principal Component 1 (PC1)**. For our student marks dataset, PC1 has a slope of approximately 1.07, as shown in Figure 4.10. This slope near 1.0 indicates that the data is almost evenly spread with respect to midterm and endterm marks—which is expected, since students who do well on one exam tend to do well on the other.

We can think of PC1 as a new composite axis: a linear combination of the original midterm and endterm axes. Instead of tracking two separate (and highly correlated) variables, PC1 captures the single underlying dimension of “overall academic performance.”

Eigenvectors and Eigenvalues

The unit vector pointing along PC1—scaled to have length exactly 1—is called the **Eigenvector** of the first principal component. Its components are known as **loading scores**. For our dataset, the eigenvector is approximately (0.683, 0.730).

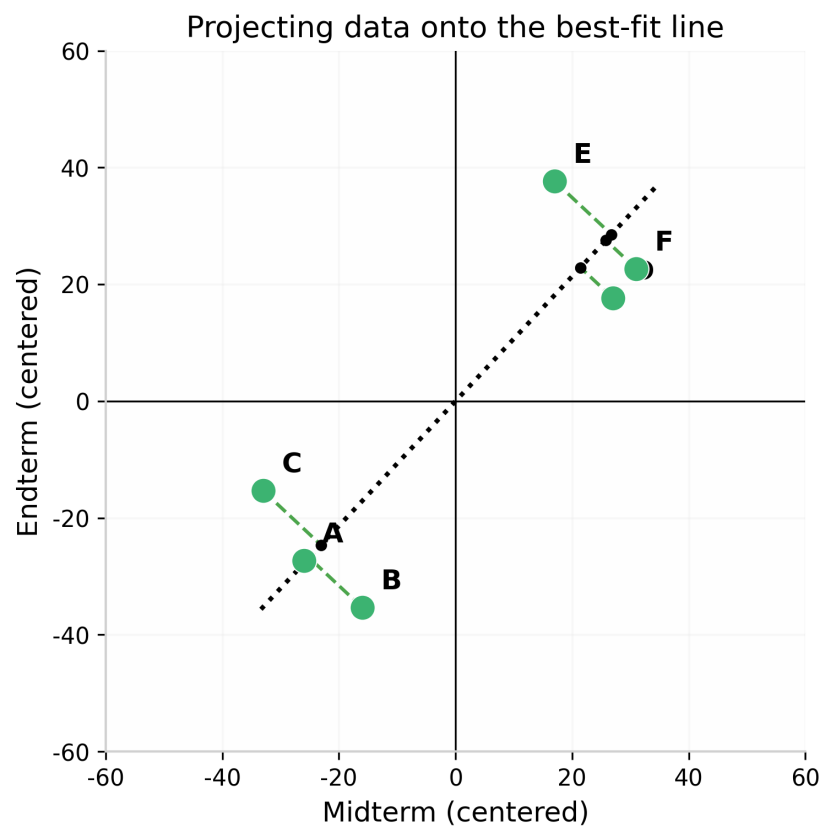


Figure 4.8: PCA projects each data point perpendicularly onto a candidate line. The optimal line is the one that maximizes the sum of squared distances from the projected points (black dots) to the origin.

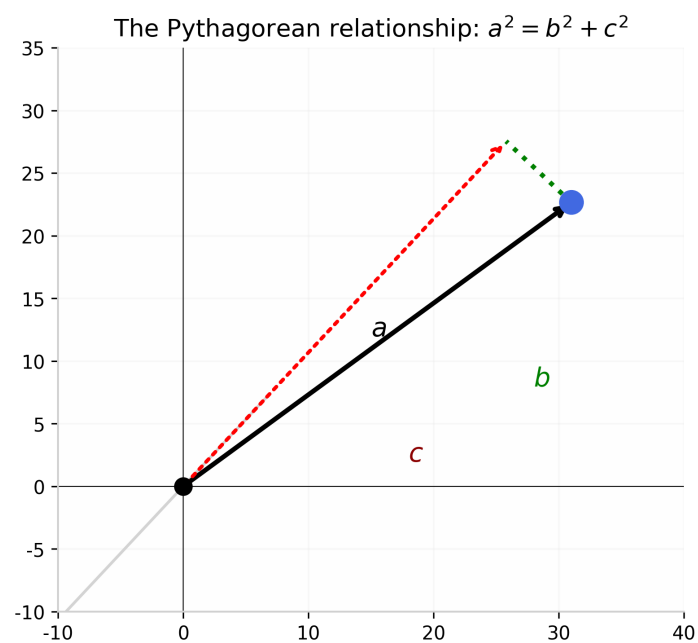


Figure 4.9: For any point, the distance to the origin a is fixed. Therefore, maximizing the projection distance c (red) is equivalent to minimizing the perpendicular error b (green).

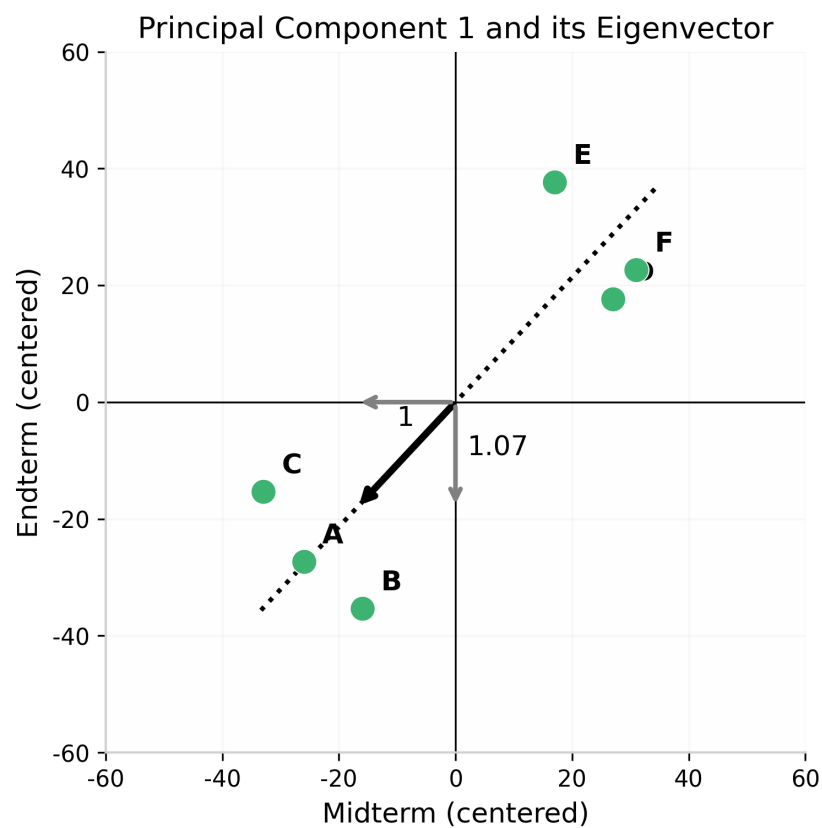


Figure 4.10: PC1 (dotted line) captures the direction of maximum variance. The black arrow is the unit eigenvector, whose components (loading scores) tell us how much each original feature contributes to PC1.

You cannot simply use any vector along the best-fit line. The eigenvector is special because it satisfies three critical properties simultaneously:

1. It is standardized to unit length.
2. It is orthogonal (perpendicular) to all other principal components.
3. It is ranked by how much information (variance) it holds.

The **Eigenvalue** associated with PC1 quantifies how much variance is captured along this direction. It is computed as the average of the sum of squared projection distances. Using $n - 1$ in the denominator (Bessel's correction) gives an unbiased estimate:

$$\lambda_1 = \frac{\sum_{i=1}^n c_i^2}{n - 1}. \quad (4.4)$$

The square root of the eigenvalue is called the **singular value**. In summary: the eigenvector tells you the *direction*, and the eigenvalue tells you the *amount of information* spread along that direction.

Principal Component 2

Because our example lives in two dimensions, PC2 is fully determined once PC1 is found: it is simply the line through the origin that is strictly orthogonal (perpendicular) to PC1, as shown in Figure 4.11.

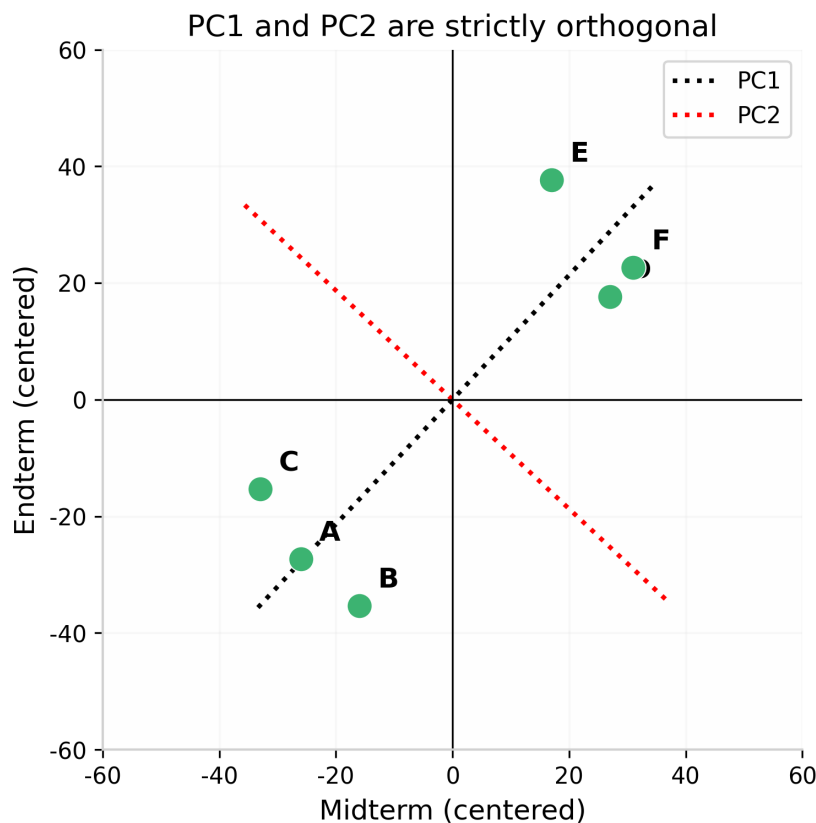


Figure 4.11: PC2 (red dotted line) is orthogonal to PC1. Together, they form a new rotated coordinate system. In higher dimensions, every subsequent PC is orthogonal to all previous ones.

Once both components are found, we simply rotate the entire dataset so that PC1 becomes horizontal and PC2 becomes vertical. We then use the projected coordinates to plot the data in this new, uncorrelated space. This is how PCA is performed in two dimensions.

4.5.3 Beyond Geometry: The Linear Algebra Perspective

Until now, every idea in PCA has been explained using pictures. In two dimensions, we can draw the data. In three dimensions, we can still (barely) visualize it. But once we have 4, 40, or 400 features, our geometric intuition runs out of room. Fortunately, the mathematics scales effortlessly.

Step 1: Represent the Data as a Matrix

Suppose each student now has d features instead of 2. The entire centered dataset can be written as a single matrix $X \in \mathbb{R}^{n \times d}$, where each row is an observation and each column is a feature. The data has already been centered, so each column has mean zero.

Step 2: Project the Entire Dataset

Suppose we choose a candidate direction vector v . Projecting every observation onto this direction is accomplished by the matrix-vector product Xv . The i -th entry of this resulting vector is the dot product $x_i \cdot v$, giving the projected coordinate of that observation.

Step 3: Maximize the Spread

We want to choose v such that the variance of the projected data is maximized. The variance of the projections is proportional to the sum of squared projections:

$$\text{Var}(Xv) \propto (Xv)^T(Xv) = v^T X^T X v. \quad (4.5)$$

Thus, our objective is to maximize $v^T X^T X v$.

Step 4: Constrain the Direction

If longer vectors were allowed, the objective could grow without bound—simply scaling v by 100 would increase the variance by 10,000. To prevent this, we restrict v to be a unit vector:

$$v^T v = 1. \quad (4.6)$$

Our optimization problem is now:

$$\max_v v^T X^T X v \quad \text{subject to} \quad v^T v = 1. \quad (4.7)$$

Step 5: Lagrange Multipliers

We combine the objective and the constraint into a single Lagrangian function:

$$\mathcal{L}(v, \lambda) = v^T X^T X v - \lambda(v^T v - 1). \quad (4.8)$$

Taking the derivative with respect to v and setting it to zero yields:

$$\frac{\partial \mathcal{L}}{\partial v} = 2X^T X v - 2\lambda v = 0 \implies X^T X v = \lambda v. \quad (4.9)$$

Step 6: PCA Meets Linear Algebra

The equation $X^T X v = \lambda v$ is exactly the **eigenvalue equation**. This gives PCA an elegant interpretation that requires no geometry:

- The **Eigenvectors** of the covariance matrix $X^T X$ are exactly the Principal Component directions.
- The **Eigenvalues** are exactly the amount of variance captured along those directions.

We did not start by looking for eigenvectors. We simply asked, “Which direction captures the most variance?” The mathematics gave us the answer.

4.5.4 Finding the Principal Components in Practice

Given the covariance matrix $X^T X$, one could find its eigenvalues by solving the characteristic polynomial $\det(X^T X - \lambda I) = 0$. For four features, this is still manageable. For five or more, the polynomial becomes degree five or higher. The Abel–Ruffini Theorem tells us there is no general algebraic formula for solving arbitrary degree-5 (or higher) polynomials.

Furthermore, explicitly forming $X^T X$ amplifies numerical errors, requires extra computation, and becomes inefficient for large datasets. Modern PCA takes a smarter route.

4.5.5 The Singular Value Decomposition (SVD) Trick

Instead of decomposing $X^T X$, modern PCA works directly with the data matrix using **Singular Value Decomposition (SVD)**:

$$X = U \Sigma V^T, \quad (4.10)$$

where:

- U contains the left singular vectors.
- Σ is a diagonal matrix containing the singular values (the square roots of the eigenvalues).
- V contains the right singular vectors—**these are exactly our Principal Components**.

Compared to eigendecomposition of $X^T X$, SVD offers decisive advantages:

- **Better numerical stability:** It avoids squaring the condition number of the matrix.
- **Computational efficiency:** It works directly on the original data matrix.
- **Handles rank-deficient data:** It works naturally even when the number of features exceeds the number of samples.

Because of these advantages, almost every modern PCA implementation—including scikit-learn, MATLAB, NumPy, and R—uses SVD internally rather than explicitly computing $X^T X$.

4.5.6 Limitations of PCA

While PCA is a legendary tool, it has two massive blind spots that every practitioner must understand.

Limitation 1: High Variance $\neq$ High Importance

PCA is completely *unsupervised*. It looks only at how much the data varies; it has no knowledge of any target variable or prediction task. As a result, features with high variance may be retained even if they are irrelevant, and features with low variance may be discarded even if they are highly predictive.

Feature	Variance	Actually Useful?
Family Income	High	✓ Yes
Age	Low	× Maybe
Student ID	Very High	× No

Table 4.2: PCA may assign a large weight to “Student ID” simply because the numbers vary wildly, even though the feature carries no meaningful information for prediction.

PCA optimizes variance—not predictive power.

Limitation 2: PCA Assumes Linear Structure

PCA looks for directions that are linear combinations of the original features. This works well when the underlying structure is approximately linear. However, many real-world datasets are inherently non-linear: concentric circles, spirals, and curved manifolds. For these datasets, no straight line can properly capture the true structure.

PCA can rotate the coordinate system; it cannot bend it. To compress non-linear data, we must look toward other techniques.

4.5.7 When Should We Use Something Else?

The best dimensionality reduction technique depends on your data and your objective. Table 4.3 summarizes the most common alternatives to PCA.

Method	When to Use
LDA	When class labels are available and class separation is the goal.
Kernel PCA	Captures non-linear patterns by implicitly mapping data to a higher-dimensional feature space.
t-SNE	Excellent for visualizing clusters in 2D or 3D. Preserves local neighborhood structure.
UMAP	Similar to t-SNE but generally faster and better at preserving both local and global structure.

Table 4.3: Alternatives to PCA for different data characteristics and objectives.

4.5.8 Key Takeaways

Question	Answer
Why PCA?	To reduce dimensionality while preserving as much variance as possible.
How does it work?	Find orthogonal directions that maximize variance.
How do we solve it?	Project data $\rightarrow$ Maximize variance $\rightarrow$ Solve the eigenvalue equation.
How is it implemented?	Modern libraries use Singular Value Decomposition (SVD).

Table 4.4: PCA in one picture.

4.5.9 Before You Use PCA: Best Practices

Do	Don't
Center the data.	Forget preprocessing.
Scale when units differ.	Compare centimeters with kilograms.
Choose components carefully using explained variance.	Keep components blindly.
Use a scree plot to guide your choice.	Assume “more PCs = better.”
Remember PCA is unsupervised.	Expect PCA to improve every ML model.

Table 4.5: Practical dos and don'ts for applying PCA.

“PCA does not understand your data—it simply finds the directions where your data varies the most.”

Conclusion of Chapter 4: In this chapter, we explored the class of ML algorithms that do not require labeled data, including clustering techniques like K-Means, Hierarchical clustering and DBSCAN, and a dimensionality reduction technique, PCA. Having equipped ourselves with supervised and unsupervised machine learning techniques, we shall enter the realm of deep learning.

Chapter 5

Deep Learning Foundations

Mahir & Pranaav

[Link to session recording](#)

5.1 Warmup: The Limits of Linearity

In the previous chapters, we relied heavily on linear models. Even when we used the Kernel Trick in SVMs to warp our data, the underlying mathematics still fundamentally sought a straight, flat hyperplane. But the real world is almost never linearly separable. Imagine a dataset where the target points form a spiral, or where identifying a "cat" requires recognizing curved ears, furry textures, and two eyes arranged in a specific spatial hierarchy. A single straight line is utterly useless here.

To capture these highly non-linear, complex patterns, we cannot rely on a single equation. Instead, we must stack multiple simple equations together, allowing them to bend, fold, and warp the decision boundary layer by layer. This is the core philosophy behind **Neural Networks** (also known as Multi-Layer Perceptrons, or MLPs).

5.2 The Artificial Neuron

The fundamental building block of a neural network is the **Neuron** (or Node). Interestingly, a single artificial neuron does exactly the same math as the Perceptron we studied in Chapter 2, with one critical upgrade.

For a given input vector x and a neuron with weight vector w and bias b :

1. **Weighted Sum:** It calculates the linear combination $z = w^T x + b$.
2. **Activation:** It passes this sum z through a non-linear **Activation Function** $g(z)$ to produce the final output $a = g(z)$.

5.2.1 Why Activation Functions Matter

If we do not use an activation function (or if we use a linear one like $g(z) = z$), stacking multiple layers is completely mathematically useless. The composition of multiple linear functions is just another linear function!

Activation functions inject the crucial **non-linearity** that allows the network to learn curves and complex boundaries. They also help control the magnitude of information passing through the network.

1. **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$

- Squeezes numbers smoothly between 0 and 1. Useful when the output needs to represent a probability.

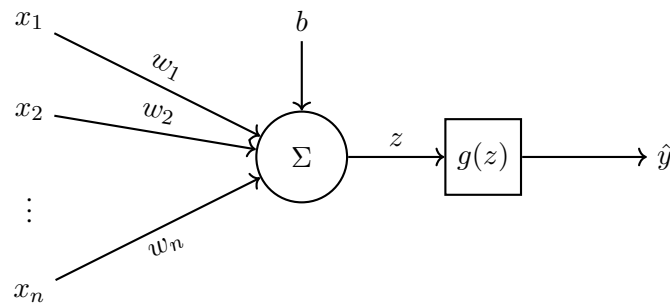


Figure 5.1: An Artificial Neuron. It computes $z = \sum w_i x_i + b$, then outputs $a = g(z)$.

- **Drawback:** The maximum derivative of a Sigmoid is 0.25. As we will see later, this causes the catastrophic *Vanishing Gradient Problem* in deep networks.
2. **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
 - Squeezes numbers between -1 and 1. Because it is zero-centered, it generally trains faster and more stably than Sigmoid.
 - **Drawback:** Still suffers from vanishing gradients at extreme z values.
 3. **ReLU (Rectified Linear Unit):** $ReLU(z) = \max(0, z)$
 - The modern default for hidden layers. It is computationally incredibly cheap (no exponents). Crucially, for any $z > 0$, the gradient is exactly 1, completely solving the vanishing gradient problem for positive inputs!
 - **Drawback (Dying ReLU):** If $z < 0$, the gradient is exactly 0. If a neuron's weights update in a way that causes it to only output negative numbers, it will never learn again. It is "dead".
 4. **Leaky ReLU & ELU:**
 - Designed to fix Dying ReLU by allowing a small, non-zero gradient for negative inputs. E.g., Leaky ReLU: $g(z) = \max(0.01z, z)$.
 5. **Softmax:** $g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$
 - Used exclusively in the output layer for multi-class classification tasks.
 - Converts raw scores into a probability distribution: all outputs lie between 0 and 1 and sum to exactly 1.
 - Pairs naturally with Cross-Entropy Loss, making it the standard choice for classification output layers.

5.2.2 Network Architecture: Depth vs. Width

Neurons are stacked into **Layers**. In a Fully Connected (Dense) layer, every neuron in the current layer connects to every single neuron in the next layer.

- **Wide Networks** (many neurons in a single hidden layer) have high capacity but tend to memorize patterns rather than generalize them.
- **Deep Networks** (many sequential layers) allow the network to learn hierarchical representations (e.g., Layer 1 learns edges $\rightarrow$ Layer 2 learns shapes $\rightarrow$ Layer 3 learns objects). Deep networks generally represent complex functions far more efficiently.

5.3 Forward Propagation and Loss

Forward Propagation is the process of feeding input data x through the network layer by layer until it produces a final prediction $\hat{y}$.

Once we have our prediction $\hat{y}$, we must evaluate how wrong it is compared to the true label y using a **Loss Function** $L(y, \hat{y})$.

- For regression tasks, we typically use **Mean Squared Error (MSE)**.
- For classification tasks, we typically use **Cross-Entropy Loss** (as derived in Chapter 1 via Maximum Likelihood Estimation).

5.4 Backpropagation: The Engine of Learning

Forward propagation gives us the error. But how do we fix it? We must figure out exactly how much every single weight and bias in the entire network contributed to that final error.

We do this using **Backpropagation** (Backward Propagation of Errors), which is simply an elegant application of the **Chain Rule of Calculus**.

5.4.1 The Chain Rule Applied

Let's zoom in on a single weight w_{13} connecting an input x_1 to a hidden neuron H_3 . The hidden neuron's output y_3 is then passed via weight w_{35} to an output neuron O_5 , which produces the final prediction $\hat{y}$.

We want to find $\frac{\partial L}{\partial w_{13}}$. The error must flow backward from the Loss L , through the output neuron, through the hidden neuron, down to the specific weight.

$$\frac{\partial L}{\partial w_{13}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_5} \cdot \frac{\partial z_5}{\partial y_3} \cdot \frac{\partial y_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{13}} \quad (5.1)$$

Let's break down exactly what these terms are:

1. $\frac{\partial L}{\partial \hat{y}}$: The derivative of the Loss function. How sensitive is the loss to our final prediction?
2. $\frac{\partial \hat{y}}{\partial z_5}$: The derivative of the output activation function $g'(z_5)$.
3. $\frac{\partial z_5}{\partial y_3}$: Since $z_5 = w_{35}y_3 + b_5$, the derivative with respect to y_3 is simply the connecting weight w_{35} .
4. $\frac{\partial y_3}{\partial z_3}$: The derivative of the hidden activation function $g'(z_3)$.
5. $\frac{\partial z_3}{\partial w_{13}}$: Since $z_3 = w_{13}x_1 + b_3$, the derivative with respect to w_{13} is simply the input x_1 .

Substituting these back in, we get the exact mathematical gradient for our weight:

$$\frac{\partial L}{\partial w_{13}} = \frac{\partial L}{\partial \hat{y}} \cdot g'(z_5) \cdot w_{35} \cdot g'(z_3) \cdot x_1 \quad (5.2)$$

While this scalar perspective is great for intuition, in practice we compute gradients for entire layers at once using matrix calculus. For a given layer l with weights $W^{(l)}$, activations $a^{(l-1)}$, and an error term $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$, the vectorized gradient is elegantly computed as the outer product:

$$\nabla_{W^{(l)}} L = \delta^{(l)} (a^{(l-1)})^T \quad (5.3)$$

By computing these gradients for every single parameter, we know exactly how to adjust the network to decrease the loss!

5.5 The Pathology of Deep Networks: Gradients Strike Back

Look closely at the Backpropagation equation above. It is a long chain of multiplications. If we have a network with 50 layers, we will multiply 50 activation function derivatives together. This mathematical reality creates the two greatest enemies of deep learning.

5.5.1 The Vanishing Gradient Problem

Recall that the maximum derivative of the Sigmoid function is 0.25. If our network has 10 layers using Sigmoid, the gradient at the very first layer will be multiplied by at best $0.25^{10} \approx 0.00000095$. This assumes every activation happens to be at its steepest point simultaneously, which is an optimistic bound. In practice, gradients vanish even faster.

The gradient becomes effectively zero. The early layers (which are supposed to learn foundational features like edges) receive absolutely no signal and stop learning completely. The network “underfits” despite having massive capacity.

The Fix: Use ReLU activations (derivative is 1, so it doesn’t shrink), use **Batch Normalization** to keep inputs in healthy ranges, or use **Residual Connections** (ResNets, covered in Chapter 9) to bypass layers entirely.

5.5.2 The Exploding Gradient Problem

The mirror-image problem. If the weights are initialized too large, the local derivatives might be greater than 1 (e.g., 1.5). Across 50 layers, $1.5^{50} \approx 6.3 \times 10^8$. The gradient spirals out of control. Weight updates become so massive that the loss oscillates wildly, eventually turning into ‘NaN’ (Not a Number), crashing the training process.

The Fix: **Gradient Clipping** (forcing the gradient magnitude to not exceed a specific threshold, e.g., 5.0) and proper **Weight Initialization**.

5.5.3 Smart Initialization (Xavier & He)

Starting with random weights is dangerous. If they are too small, signals vanish. If they are too large, signals explode. We must initialize weights such that the variance of the activations stays perfectly balanced from the first layer to the last.

- **Xavier (Glorot) Initialization:** Designed for Sigmoid/Tanh networks. Weights are drawn from a uniform distribution over the range $\left[-\sqrt{\frac{6}{n_{in}+n_{out}}}, +\sqrt{\frac{6}{n_{in}+n_{out}}}\right]$, ensuring the variance of activations remains stable across layers.
- **He Initialization:** Designed specifically for ReLU networks (accounting for the fact that ReLU zeroes out half the inputs). Weights are drawn from a normal distribution with variance $\frac{2}{n_{in}}$.

5.6 Optimizers: Navigating the Loss Landscape

Now that Backpropagation has given us the gradients, how do we use them to update our weights? We are trying to find the global minimum of a massive, multidimensional, non-convex loss function landscape.

5.6.1 Gradient Descent (GD)

The simplest approach: take a step in the direction of the steepest descent.

$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} J(\theta) \quad (5.4)$$

Where α is the **Learning Rate**.

- **Too low:** The model takes agonizingly tiny steps and takes forever to converge.

- **Too high:** The model takes massive steps, overshoots the minimum, and diverges.

How much data do we use to calculate $\nabla_{\theta} J(\theta)$?

- **Batch GD:** Uses the entire dataset. Highly accurate, but computationally brutal and slow.
- **Stochastic GD (SGD):** Uses a single random data point. Very fast, but the path is highly noisy and erratic.
- **Mini-Batch GD:** The industry standard. Uses a small chunk of data (e.g., 32 or 64 samples). Balances computational speed with stable gradient estimation.

Parameter	Batch GD	Mini-Batch GD	Stochastic GD
Accuracy	High	Moderate	Low
Computation Time	High	Moderate	Low
Noise Sensitivity	Low	Moderate	High
Memory Usage	High	Moderate	Low

Table 5.1: Comparison of Gradient Descent variants (This is for a single step).

5.6.2 SGD with Momentum

Standard SGD gets stuck easily in shallow local minima and saddle points. To fix this, we introduce the concept of physics.

Instead of updating weights based purely on the *current* gradient, we maintain a “velocity” vector v_t . It is an exponentially weighted moving average of all past gradients.

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla L(\theta_t) \quad (5.5)$$

$$\theta_{t+1} = \theta_t - \alpha v_t \quad (5.6)$$

Where β is the momentum coefficient (usually 0.9). If the gradient points in the same direction for many steps, velocity builds up, allowing the model to blast right through small local minima!

5.6.3 RMSProp

What if our loss landscape is shaped like a steep ravine? The gradient will oscillate wildly up and down the steep walls, making very little progress along the actual floor of the ravine.

RMSProp solves this by giving every single parameter its own custom learning rate. It maintains a moving average of the *squared* gradients (E_t):

$$E_{t+1} = \beta E_t + (1 - \beta) g_t^2 \quad (5.7)$$

We then divide the current gradient by the square root of this average:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E_{t+1} + \epsilon}} g_t \quad (5.8)$$

If a parameter has been jumping around with huge gradients, E_t will be massive. Dividing by it forces that parameter to take tiny, cautious steps. If a parameter has small gradients, it gets a relative speed boost. It brutally dampens oscillations!

5.6.4 Adam (Adaptive Moment Estimation)

Why choose between Momentum and RMSProp when you can have both? **Adam** is the undisputed king of modern optimizers. It tracks both the first moment (momentum, m_t) and the second moment (squared gradients, v_t).

First, we update the two moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (5.9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5.10)$$

Because m_0 and v_0 are initialized at 0, the early estimates are biased toward zero. Adam applies a clever **Bias Correction** to fix this:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (5.11)$$

And the final parameter update utilizes both:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (5.12)$$

```

1 def adam_step(theta, gradient, m, v, t, alpha=0.001, beta1=0.9, beta2=0.999, epsilon=1e
  -8):
2     # 1. Update biased first moment estimate
3     m = beta1 * m + (1 - beta1) * gradient
4
5     # 2. Update biased second raw moment estimate
6     v = beta2 * v + (1 - beta2) * (gradient ** 2)
7
8     # 3. Compute bias-corrected first moment estimate
9     m_hat = m / (1 - beta1 ** t)
10
11    # 4. Compute bias-corrected second raw moment estimate
12    v_hat = v / (1 - beta2 ** t)
13
14    # 5. Update parameters
15    theta = theta - (alpha / (np.sqrt(v_hat) + epsilon)) * m_hat
16
17    return theta, m, v

```

Listing 5.1: Basic Implementation of Adam Optimizer

5.7 Autoencoders

In Chapter 4, we used Principal Component Analysis (PCA) to compress high-dimensional data down into a smaller space while retaining maximum variance. But we also noted its fatal flaw: PCA is strictly a linear operation. It cannot fold or bend coordinate systems to compress complex, non-linear manifolds.

What if we could use the extreme non-linear power of Neural Networks to compress data?

Imagine giving a student a massive, 1000-word essay and telling them to memorize it perfectly. If they have unlimited brainpower, they will just memorize every single word verbatim. But what if we force them to write a 10-word summary on a sticky note, and then we ask a second student to reconstruct the original 1000-word essay using *only* those 10 words? To succeed, the first student cannot just memorize words; they must extract the absolute core, fundamental *meaning* and *structure* of the essay.

This is the exact philosophy behind an **Autoencoder**. It is an unsupervised neural network that simply tries to copy its input to its output, but it is forced to push the data through a tight mathematical bottleneck.

5.8 The Architecture of an Autoencoder

An Autoencoder consists of three distinct parts:

1. **The Encoder:** A standard neural network that takes high-dimensional input X and systematically compresses it into smaller and smaller hidden layers.
2. **The Bottleneck (Latent Space):** The smallest layer in the absolute middle of the network. The activations of these neurons represent the compressed, low-dimensional "code" or "latent representation" of the data.
3. **The Decoder:** A neural network that takes the compressed latent code and attempts to upscale and reconstruct it back into the original input dimension, outputting $\hat{X}$.

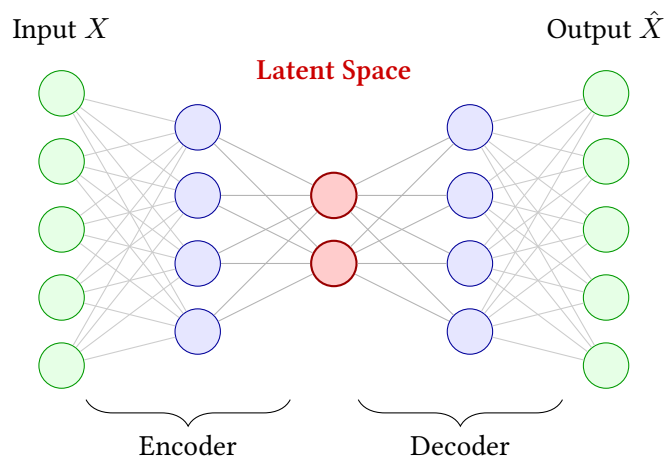


Figure 5.2: A classic Autoencoder architecture forcing data through a low-dimensional bottleneck.

The **Loss Function** is simply the reconstruction error. For continuous data, this is the Mean Squared Error: $L = \|X - \hat{X}\|_2^2$. For binary data, we use Binary Cross Entropy.

5.8.1 The Pre-Training Paradigm

Training deep autoencoders end-to-end using standard backpropagation is notoriously difficult due to vanishing gradients. Moreover, the early encoder layers often fail to learn useful features if training is only done from end to end fully at one pass

To solve this, we can use **Unsupervised Layer-by-Layer Pre-Training**.

1. Take only the input layer and the first hidden layer. Treat them as a shallow, single-layer autoencoder and train it to reconstruct the input.
2. Once trained, freeze the weights of this hidden layer.
3. Now, use the outputs of this frozen hidden layer as the "input" to train the next hidden layer in isolation.
4. Repeat this process layer by layer. Finally, assemble all the pre-trained layers together and do a final full-network fine-tuning pass.

This ensures every single layer is guaranteed to capture robust, meaningful information from the layer immediately preceding it.

5.8.2 Effects of Pre-Training

Layer-by-Layer pre-training is, by design, a greedy approach and the final fine tuning is essential to achieve complete accuracy. However, this method of training, is still more accurate than full end-to-end training and the final step is less computationally expensive since we start from good enough weights. This method also ensures better convergence, but doesn't solve identity-learning

5.9 Flavors of Autoencoders

If the latent space is of high dimensionality, the model might just learn the "Identity Function"—a useless mapping where X is perfectly passed to $\hat{X}$ without the model learning any underlying features. This is completely useless in applications where the compressed data is also part of the output. To prevent this, we introduce various regularization strategies, resulting in different types of Autoencoders.

5.9.1 Denoising Autoencoders

Instead of changing the architecture, we change the data. We take the original input X and aggressively corrupt it by adding Gaussian noise or masking out random pixels, creating $\tilde{X}$.

We feed the corrupted $\tilde{X}$ into the encoder, but we calculate the loss against the original, clean X : $L = \|X - \text{Decoder}(\text{Encoder}(\tilde{X}))\|^2$. Because the network must output a clean image from a noisy input, learning a simple identity function is mathematically penalized. The model is now forced to learn robust and clean features to reconstruct the original image without any noise.

5.9.2 Sparse Autoencoders

What if we want a wide latent space (more neurons than the input) but still want compression? We use a **Sparse Autoencoder**. We add a regularization penalty to the loss function that strictly limits the number of neurons allowed to "fire" (activate) at any given time.

If a neuron's activation is near 1, it is "active". If it is near 0, it is "inactive". We define a sparsity hyperparameter ρ (e.g., 0.05), dictating that on average, a neuron should only fire 5% of the time. We calculate the actual average activation $\hat{\rho}_j$ for a neuron j over a batch of data.

To force $\hat{\rho}_j$ to match our target ρ , we add the **Kullback-Leibler (KL) Divergence** penalty to our loss function. KL Divergence measures how much one probability distribution diverges from another:

$$J_{\text{sparse}}(W, b) = J_{\text{recon}}(W, b) + \beta \sum_j KL(\rho \parallel \hat{\rho}_j) \quad (5.13)$$

$$KL(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \quad (5.14)$$

If the actual activation $\hat{\rho}_j$ deviates from 0.05, the KL divergence explodes toward infinity, brutally punishing the network.

By forcing sparsity, each neuron is forced to only activate for highly specific, distinct features. The network untangles overlapping concepts, leading to highly interpretable, distinct feature representations.

5.9.3 Variational Autoencoders (VAEs)

Standard autoencoders are deterministic; a specific input always maps to one specific latent point. If you pick a random point in the latent space and pass it to the decoder, you will likely get absolute garbage out, because the latent space is not continuous.

Variational Autoencoders (VAEs) solve this by encoding the input not as a single point, but as a *probability distribution*. The encoder outputs two vectors: a vector of means (μ) and a vector of standard deviations (σ). Corresponding elements of the mean vector and the std dev vector map to the probability distribution

for corresponding features. The bottleneck then samples a random point z from the Gaussian distribution defined by $\mathcal{N}(\mu, \sigma^2)$, and passes z to the decoder.

However, we cannot backpropagate gradients through a purely random sampling node. To solve this, VAEs use the **Reparameterization Trick**. We sample a completely independent random noise vector $\epsilon \sim \mathcal{N}(0, I)$ and shift it by our learned parameters:

$$z = \mu + \sigma \odot \epsilon \quad (5.15)$$

Because ϵ handles all the randomness, z is now a deterministic function of μ and σ , allowing backpropagation to flow perfectly!

The VAE loss function balances the reconstruction error with a KL Divergence term that forces all these learned distributions to closely match a standard Normal distribution $\mathcal{N}(0, 1)$.

$$L_{VAE} = \|X - \hat{X}\|^2 + \sum KL(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) \quad (5.16)$$

Because the latent space is now a continuous, structured probability landscape, VAEs are incredibly powerful **Generative Models**. You can sample random coordinates from the latent space, pass them through the decoder, and generate entirely new, realistic data that never existed in the training set!

Conclusion of Chapter 5: We have seen how neural networks can be repurposed from supervised prediction engines into unsupervised discovery engines. Autoencoders allow us to compress dimensionality, denoise signals, and even generate entirely new data. But so far, our networks have only looked at isolated, static snapshots of data. In the real world—whether it’s predicting the next word in a sentence, forecasting stock prices, or tracking a moving object—data is sequential. Time matters. In the next chapter, we will evolve our architecture to give neural networks the power of memory!

Chapter 6

Natural Language Processing & Sequential Models

Vishal & Rithvik

[Link to session recording](#)

6.1 Warmup: The Language Problem

So far, our models have processed numbers, continuous coordinates, and categorical labels. But human language is vastly different. It is composed of discrete, arbitrary symbols (words) arranged in sequences of highly variable lengths, where the exact order of the symbols dictates the entire meaning of the sequence.

The quest to make machines understand language began long before deep learning. In 1966, MIT researcher Joseph Weizenbaum created **ELIZA**, widely considered the first chatbot. ELIZA simulated a psychotherapist. If a user typed "I am feeling sad," ELIZA would respond, "Why do you say you are feeling sad?"

While revolutionary, ELIZA possessed absolutely zero understanding of language. It operated on rigid pattern matching and scripted keyword substitution. If a user employed novel sentence structures, typos, or unexpected phrasing, ELIZA would produce nonsensical or generic replies. It was a static rulebook that could not learn or adapt.

To truly "understand" language, we must move away from static rules and allow models to infer the statistical and semantic properties of words from massive corpora of text.

6.2 Statistical NLP: N-Grams and HMMs

Before neural networks dominated NLP, the field relied heavily on statistics and probability distributions.

6.2.1 The N-Gram Model

An N-gram model predicts the next word in a sequence based purely on the previous $n - 1$ words.

$$P(w_t | w_1, w_2, \dots, w_{t-1}) \approx P(w_t | w_{t-n+1}, \dots, w_{t-1}) \quad (6.1)$$

This is built upon the **Markov Assumption**: the probability of a future state depends only on the most recent state, not the entire history.

- **Unigram (1 word)**: Assumes words occur independently.
- **Bigram (2 words)**: Predicts based on the 1 preceding word.

- **Trigram (3 words):** Predicts based on the 2 preceding words.

N-grams are simple frequency counts. We count how often a specific sequence appears in a training corpus and convert those counts to probabilities. They have several practical use cases:

1. **Text Generation:** Start with a seed phrase, sample the most probable next word, append it, and slide the context window forward.
2. **Document Retrieval:** Search queries and documents are represented as sets of N-grams. We score relevance using **Jaccard Similarity**: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$.
3. **Spelling Checkers:** Break a misspelled word ("recieve") into character-level trigrams (rec, eci, civ, iev, eve) and compare Jaccard similarity against trigrams of dictionary words ("receive").

Drawbacks of N-Grams: They possess no real understanding of meaning. "The food was good, not bad" and "The food was bad, not good" output identical unigram counts. They also suffer from severe domain bias and limited context windows.

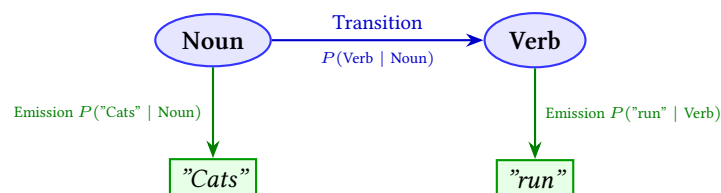
6.2.2 Hidden Markov Models (HMMs)

While N-grams look only at observed words, **Hidden Markov Models (HMMs)** assume there is an underlying, unobserved (**hidden**) structure generating those words—such as Parts of Speech (POS tags like Noun, Verb, Adjective).

An HMM relies on two core probabilities:

- **Transition Probabilities** $P(\text{Verb} \mid \text{Noun})$: The probability of moving from one hidden state (tag) to the next.
- **Emission Probabilities** $P(\text{"run"} \mid \text{Verb})$: The probability of observing a specific word given a hidden state.

Visualizing How HMMs Work



Limitations

While powerful for early POS tagging, HMMs have three key drawbacks:

- **Markov Assumption:** Assumes the next tag depends *only* on the current tag (ignores long-distance context).
- **Observation Independence:** Assumes words depend only on their own tag, ignoring nearby words.
- **Data Heavy:** Requires large amounts of manually annotated training data.

6.3 Neural NLP

Historical Context: The Evolution from Autocomplete to AI

Early neural NLP researchers (2003–2013) were not attempting to design conversational assistants like ChatGPT or Gemini. The primary goal was **autocomplete** : predicting the most probable next word given a sequence of prior words.

Two decades later, scaling neural networks to hundreds of billions of parameters revealed an emergent phenomenon:
optimizing for next-token prediction at massive scale forces the network to acquire general reasoning, translation, and code-synthesis capabilities.

Modern chatbots are fundamentally next-word predictors that have been instruction-tuned to converse rather than merely complete sentences.

TLDR : Modern LLM's are really good auto completion agents.

To understand how they work, we must first learn auto-completion. Similar to how we must study **Thompson's, Rutherford's, Bohr's Atomic model** before studying **Schrodinger's model** and **Quantum Mechanical model**.

The previous section dealt with auto-completion using statistical data (Statistical NLP). The following section will introduce us to **Deep Learning methods for autocomplete**

6.3.1 The Need for Deep Learning/Neural Networks in Auto-Completion

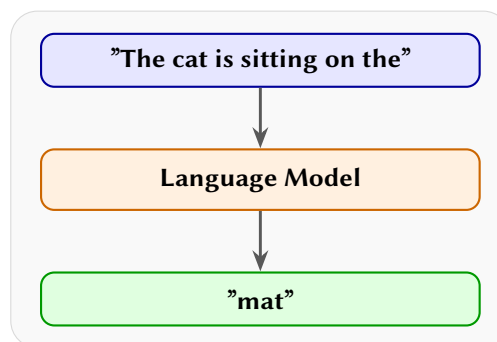


Figure 6.1: The fundamental language modeling objective: predicting the next token.

To formalize this objective:

- Given an input sequence of words (the context), our goal is for a **Language Model** to predict the next word or even better, output a probability distribution over the vocabulary for the next word.
- Mathematically, we seek a target function $f : \text{Context} \rightarrow \text{Next Word}$, which capable of generalizing across all natural language context combinations. In other words, we seek a function f which takes as **input** : $\text{context} \in \text{English}$ and **outputs** : probability distribution of the next word.
- Hence, our goal is simply to find this function f
- Because the underlying function f is far too complex to be rule-crafted by hand, we utilize a **Neural Network** (It's job is to find functions) to approximate it from corpus data. So, our idea to put the context and the correct prediction into the network and train it until it learns the underlying function.
- Sounds easy right? This has some issues. Lets look at them one by one.

6.4 The Challenge of Representing Words as Numbers

Our program (Neural network) operates strictly on mathematical vectors and matrices in other words, our program doesn't understand words. I.E., raw text strings cannot be directly fed into a model. We need a way to convert words to numbers so that they can be fed into our program

Motivation: Capturing Semantic Proximity

Consider two context phrases:

"Today's **test** was extremely _____"
 "Today's **exam** was extremely _____"

Intuitively, both the above contexts should yield nearly identical output predictions. If the representations of the words **test** and **exam** are similar to each other then, it would be easier for the network to learn. So whatever mathematical representation that we choose, we'd prefer that it has the property that similar words have similar meanings.

6.4.1 Integer Encoding and Its Flaws

The most naive approach to numerical conversion is assigning each unique word in a vocabulary $\mathcal{V}$ an arbitrary scalar integer index:

$$\mathcal{V} = \{\text{"cat"} : 1, \text{"dog"} : 2, \text{"apple"} : 3, \dots\} \quad (6.2)$$

Why Integer Encoding Fails

Assigning arbitrary scalar integers creates severe optimization roadblocks for neural networks:

1. **Artificial Ordinality:** The network interprets scalar values as continuous measurements ($1 < 2 < 3$), incorrectly imposing an ordinal ranking where "dog" is treated as quantitatively larger than "cat".
2. **Spurious Arithmetic Relationships:** Linear transformations on indices yield nonsensical operations ($1 + 2 = 3 \implies \text{"cat"} + \text{"dog"} = \text{"apple"}$).

Core Takeaway: To feed words into neural networks without imposing artificial ordinal constraints, we must map discrete tokens into multi-dimensional vectors rather than scalar integers.

6.4.2 One-Hot Encoding and the Orthogonality Barrier

To eliminate artificial ordinality, we can place each word on its own independent, orthogonal axis in high-dimensional space.

One-Hot Vectors

For a vocabulary of size $N = |\mathcal{V}|$, word i is represented as an N -dimensional basis vector $v_i \in \{0, 1\}^N$, where the i -th dimension is 1 and all other dimensions are 0:

$$v_{\text{cat}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \end{bmatrix}, \quad v_{\text{dog}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix}, \quad v_{\text{apple}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \end{bmatrix} \quad (6.3)$$

While One-Hot Encoding successfully removes ordinal biases, it introduces two catastrophic failure modes at scale:

The Orthogonality and Sparsity Bottleneck

1. **Zero Semantic Proximity (Orthogonality):** The dot product of any two distinct one-hot vectors is strictly zero:

$$v_{\text{test}} \cdot v_{\text{exam}} = 0 \quad \text{and} \quad v_{\text{test}} \cdot v_{\text{apple}} = 0 \quad (6.4)$$

Additionally, the Euclidean distance between any two distinct one-hot vectors is constant ($\|v_i - v_j\| = \sqrt{2}$). The network has no geometric way to recognize that "test" is semantically closer to "exam" than to "apple".

2. **The Curse of Dimensionality (Sparsity):** For a vocabulary size of $N = 100,000$, every single token requires a 100,000-dimensional vector containing 99,999 zeros. Multiplying these sparse vectors by weight matrices wastes memory and creates extreme computational inefficiencies.

ENTER EMBEDDINGS

Translating Discrete Symbols into Dense Semantic Geometry

6.5 Dense Word Embeddings

An **Embedding** is a continuous, dense vector representation $v \in \mathbb{R}^d$, where $d \ll |\mathcal{V}|$ (typical dimensions range from $d = 100$ to $d = 768$). Unlike sparse one-hot vectors, embeddings map words into a low-dimensional continuous manifold where geometric distance directly mirrors semantic meaning.

The Distributional Hypothesis

"You shall know a word by the company it keeps." — J.R. Firth (1957)

We need to assign words to vectors. The issue is that I can't manually decide what vector to be assigned to a particular word as there are 200,000 words.

Also, the word **similar** is pretty vague. Hence,

We need a **mathematical rule / algorithm** which assigns words to vectors or in other words, finds the embedding vectors for all the words in our vocab.

Below are a few algorithms which find the embeddings.

6.5.1 Shallow Window-Based Models: Word2Vec

Developed by Mikolov et al. (2013), **Word2Vec** trains a shallow two-layer neural network using a sliding context window across text corpora. It comes in two primary architectural variants:

- **Skip-Gram:** Given a central target word x_t , the model predicts the surrounding context tokens within window radius k .
- **CBOW (Continuous Bag-of-Words):** Given the surrounding context tokens within window radius k , the model predicts the central target word x_t .

Why CBOW Forces Semantic Clustering:

Consider two sentences: "*I **hope** to pass the exam*" and "*I **wish** to pass the exam*".

In CBOW, the model averages the embedding vectors of context words to predict the center token:

$$\text{Avg}_1 = \frac{v_I + v_{to} + v_{pass} + v_{the} + v_{exam}}{5} \implies \text{Predicts "hope"} \quad (6.5)$$

$$\text{Avg}_2 = \frac{v_I + v_{to} + v_{pass} + v_{the} + v_{exam}}{5} \implies \text{Predicts "wish"} \quad (6.6)$$

Because nearly identical context averages generate predictions for both "*hope*" and "*wish*", backpropagation pulls the embedding vectors v_{hope} and v_{wish} closer together in parameter space.

[Click Here](#) for understanding CBoW, Skipgram in detail. [Click here](#) for embeddings visualization

After looking at this the embedding vectors, you will realize that similar words get similar vectors and, vector relations such as "*King*" - "*Man*" + "*Women*" = "*Queen*" emerge.

6.5.2 Global Matrix Factorization: GloVe

While Word2Vec scans local sliding windows, **GloVe (Global Vectors)** by Pennington et al. (2014) combines local context modeling with global corpus co-occurrence statistics.

GloVe constructs a global **Co-occurrence Matrix** X , where entry X_{ij} stores how frequently target word i appears within the context window of word j across the entire corpus.

The Objective Function

GloVe enforces that the dot product of two word vectors equals the logarithm of their global co-occurrence count, minimizing a weighted least-squares loss:

$$J = \sum_{i,j=1}^{|\mathcal{V}|} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (6.7)$$

- $w_i \in \mathbb{R}^d$: Target word vector for word i .
- $\tilde{w}_j \in \mathbb{R}^d$: Context word vector for word j .
- $b_i, \tilde{b}_j \in \mathbb{R}$: Scalar bias terms accounting for standalone word frequencies.
- $f(X_{ij})$: Weighting function to cap stop-word dominance and eliminate zero-count evaluation errors.

The Weighting Function $f(X_{ij})$

To stabilize optimization, GloVe defines $f(x)$ as:

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}} \right)^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases} \quad (6.8)$$

Typically setting $x_{\max} = 100$ and $\alpha = 0.75$, this function achieves three goals:

1. **Handles Zero Counts:** Ensures $f(0) = 0$, avoiding evaluation errors with $\log(0)$.
2. **Attenuates Noise:** Assigns minimal weight to rare co-occurrences (e.g., typos).
3. **Caps Stop-Words:** Limits the influence of extremely frequent stop-words (e.g., "*the*", "*is*").

Final Word Vector Construction

Because X is symmetric, target vectors w_i and context vectors $\tilde{w}_i$ encode complementary geometric information. The final embedding for word i is calculated as their sum:

$$w_{\text{final}} = w_i + \tilde{w}_i \quad (6.9)$$

6.6 Subword Tokenization & Byte-Pair Encoding (BPE)

Word-level embeddings face a major drawback: the **Out-of-Vocabulary (OOV)** problem. If a model encounters a word during inference that was absent from its training vocabulary (e.g., medical terms or typos), it fails or defaults to a generic `<UNK>` token. Conversely, character-level modeling eliminates OOV but dramatically increases sequence lengths, making context tracking difficult.

Modern NLP uses **Subword Tokenization** to bridge this gap.

Byte-Pair Encoding (BPE) Algorithm

Byte-Pair Encoding (used in models like GPT-2, GPT-3, and GPT-4) dynamically builds a subword vocabulary through iterative frequency merges:

1. **Initialization:** Treat every individual character as an initial token in the vocabulary.
2. **Frequency Scan:** Count all adjacent token pairs across the training corpus.
3. **Iterative Merge:** Identify the most frequent adjacent token pair (e.g., "e" and "r") and merge them into a single new subword token ("er").
4. **Termination:** Repeat the scan and merge steps iteratively until reaching a target vocabulary size.

Result: High-frequency words (e.g., "the", "cat") remain whole single tokens, while rare words (e.g., "unaffordability") are decomposed into meaningful subword units ("un", "afford", "ability").

6.7 Where is Our Model ? Autocomplete using ANN's

The Missing Function Finder

So far, we have successfully mapped discrete text tokens into continuous, dense embedding vectors $v \in \mathbb{R}^d$. But tokenization and embeddings only give us a way to convert words into numbers in a sensible way. They do not predict the next token.

We return to our core objective: **where is the function finder?** We need a neural architecture $f(\text{Context}) = P(\text{Next Token})$ that takes sequence representations as input and outputs probability distributions over the vocabulary.

Our first instinct is to use standard **Artificial Neural Networks (Feedforward ANNs)**. Let us explore what happens when we try to model language with standard ANNs.

6.7.1 Naive Attempt: Feedforward Neural Networks

In a standard Feedforward Neural Network, input layers require a fixed-dimensional vector $x \in \mathbb{R}^M$. Consider trying to feed an entire context sentence into an ANN to predict the next token:

"This morning I took my cat for a _____" $\implies$ Predict: **"walk"**

Failure Mode 1: Rigid Input Dimensions

Standard ANNs have fixed network architectures. I.E., at the start of the code I decide my architecture eg: 10-12-12-10. So, the input layer dimension M is permanently locked upon initialization.

- Context A: *"This morning I took my cat for a _____"* (8 input tokens)
- Context B: *"This morning I _____"* (3 input tokens)

Because natural language inputs vary dynamically in length from sentence to sentence, a standard feedforward transformation $y = \sigma(Wx + b)$ cannot natively accommodate variable-length sequences without artificial padding or clipping.

6.7.2 Idea #1: The Fixed Context Window

To bypass the variable-length limitation of standard ANNs, we can establish a **Fixed Context Window** of size k . Instead of passing the entire sentence, we only pass the last k words as input to the network.

*"This morning I took my cat **for a** _____"*
(Given the fixed two-word context ["for", "a"], predict **"walk"**)

While this ensures fixed input dimensionality ($x \in \mathbb{R}^{k \times d}$), it introduces three severe structural flaws:

Failure Modes of Fixed-Window ANNs

1. **Inability to Model Long-Term Dependencies:** Consider the following sentence:

*"**France** is where I grew up, but I now live in Boston. I speak fluent _____."*

The model here, predicts english rather than french for a context = 2 because, speak fluent is usually followed by english because english is the most spoken language. To correctly predict *"French"*, the model must retain information from the token *"France"*, which occurred 12 tokens prior. If our fixed window size is $k = 2$ or $k = 5$, the critical context is completely truncated and unreachable.

2. **Context Explosion and Memory Inefficiency:** To fix long-range dependencies, one might suggest expanding k to a massive window size (e.g., $k = 500$). However, if a user writes a 300-word paragraph about living in India with no mention of France, the model gets overwhelmed by uninformative filler text while the weight matrix $W \in \mathbb{R}^{H \times (k \cdot d)}$ explodes in memory footprint.
3. **No Parameter Sharing Across Time:** Standard ANNs assign distinct parameters to each input position. The weights connected to word position 1 are entirely separate from word position 5. Learning the meaning of a word at the beginning of a sentence does not help the network recognize the same word at the end of a sentence.

6.7.3 Design Requirements for a True Sequence Model

To overcome the architectural rigidity of standard feedforward networks, any viable sequence-modeling architecture must fulfill four core pillars:

The Four Pillars of Sequence Modeling

1. **Handle Variable-Length Sequences:** Accept inputs of arbitrary temporal length without structural modification.
2. **Track Long-Term Dependencies:** Retain crucial historical context across extended sequences.
3. **Preserve Sequential Order:** Explicitly model token order ($v_{\text{dog bites man}} \neq v_{\text{man bites dog}}$).
4. **Share Parameters Across Time:** Reuse the same set of learned weight parameters across every position in the sequence.

6.7.4 Introducing Recurrent Neural Networks (RNNs)

The solution to these requirements is the **Recurrent Neural Network (RNN)**. Instead of processing entire text sequences through a single static forward pass, an RNN processes tokens sequentially, maintaining an internal **Hidden State Vector** h_t that acts as a recurrent memory.

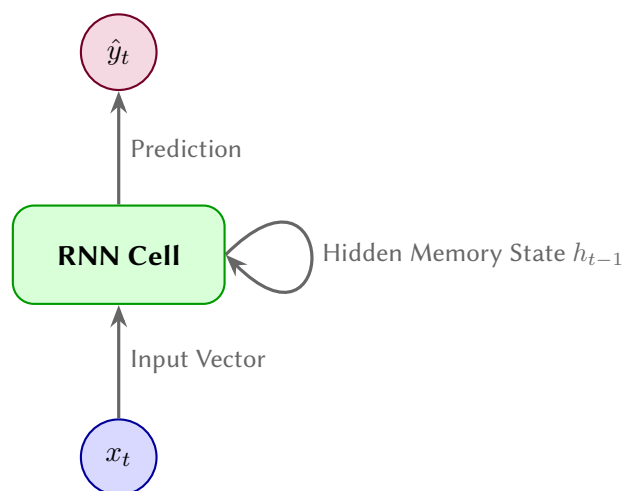


Figure 6.2: The Recurrent Neural Network architecture featuring a feedback loop to preserve sequence history across time steps.

Looking Ahead: The Recurrent Mechanism

Coming up with an AI architecture out of nowhere makes no sense. The RNN architecture was inspired by how we humans think. Let's look at this in a bit more detail and understand why researchers chose this particular architecture for RNN's

6.8 The Intuition : How we Autocomplete sentences

Lets have a look at the conversation between myself and John

Conversation

John: "My name is John, I grew up in France."

Rithvik: "ok"

John: I fluently speak _____

How do I answer this question? When John told me that his name is John and he grew up in france, I didn't remember word to word what he told me instead, I stored something like a mental summary as shown below.

$$\text{Mental Summary} \approx \begin{cases} \text{Name:} & \text{John} \\ \text{Country:} & \text{France} \end{cases}$$

Now, we get an idea! what if our model can store **relevant** Information upto that point and make informed decisions using that rather than storing a fixed size window. This is the intuition behind the **Hidden State Vector** and the Genius of RNN's.

6.8.1 The Hidden State Vector ($\mathbf{h}_t$)

Recurrent Neural Networks operationalize this human intuition through the **Hidden State Vector** $\mathbf{h}_t \in \mathbb{R}^d$. Rather than storing an expanding list of past word vectors, an RNN compresses the entire history of the sequence up to time step t into a single, fixed-dimensional vector.

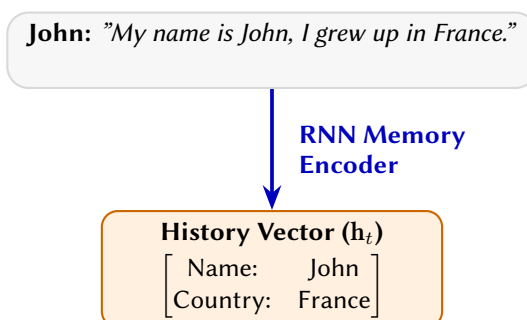


Figure 6.3: Compressing conversational context into a fixed-size recurrent state vector.

6.8.2 What is Relevant: The Computer Vision Parallel

A fundamental question arises:

How does the network decide what information to keep in $\mathbf{h}_t$ and what to throw away?. How does the network know what is relevant. After each input, I can't tell the network that

"Hey look France is important lets store it"

We need a mathematical way to let the network learn what is relevant.

We do not hardcode manual linguistic rules to specify which facts to extract. Instead, we let the model learn what is relevant end-to-end using gradient descent.

Parallel to Computer Vision

This paradigm shift mirrors the evolution of feature extraction in Computer Vision:

- **Classical CV:** Engineers hand-crafted fixed edge-detection kernels (e.g., Sobel filters) to detect features manually.
- **Deep CV (CNNs):** Networks learn optimal filter weights automatically through backpropagation based on the task objective.
- **Deep NLP (RNNs):** Rather than hand-crafting grammar rules or memory slot lookups, backpropagation trains weight matrices (W_{hh} and W_{xh}) to automatically extract and retain task-relevant historical context while discarding noise.

6.9 Recurrent Neural Networks : RNNs

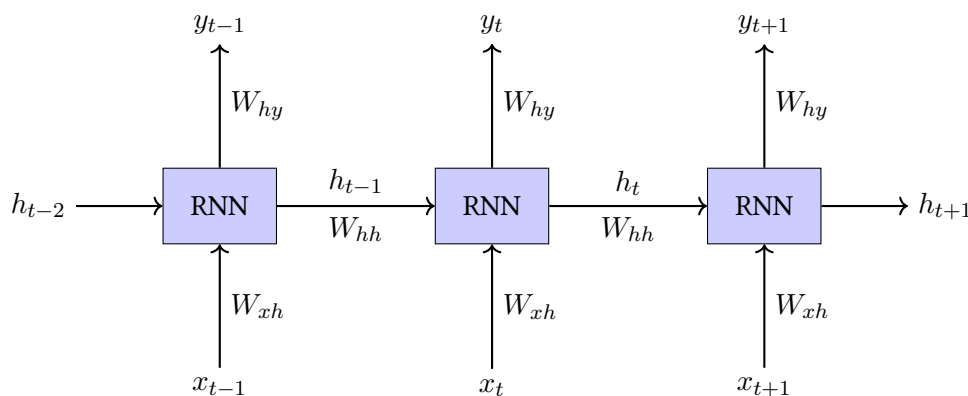


Figure 6.4: An unrolled Recurrent Neural Network across time.

6.9.1 How an RNN works

The key idea.

x_t = new information the model receives.

h_{t-1} = the relevant information the model has until t^{th} instant(1,2,3...,t-1).

$h_t = f(h_{t-1}, x_t)$

Relevant information after t^{th} instance = a function of relevant information upto the t^{th} instant and the new information it receives at the t^{th} instant

Firstly, we need to understand that an RNN processes the context one by one. I.e. say the context is "I am from France, I fluently speak _____".

How a RNN does this

1. Initial context vector $h_0 = \text{empty}$. Takes in input $x_1 = "I"$. Based on these two, makes a decision about the context it should store i.e. $h_1 = f(x_1, h_0)$
2. Takes in input $x_2 = "am"$. Based on the context it received(h_1). It takes a decision on what to store i.e. $h_2 = f(h_1, x_2)$. Repeats this and gives the output whenever we want

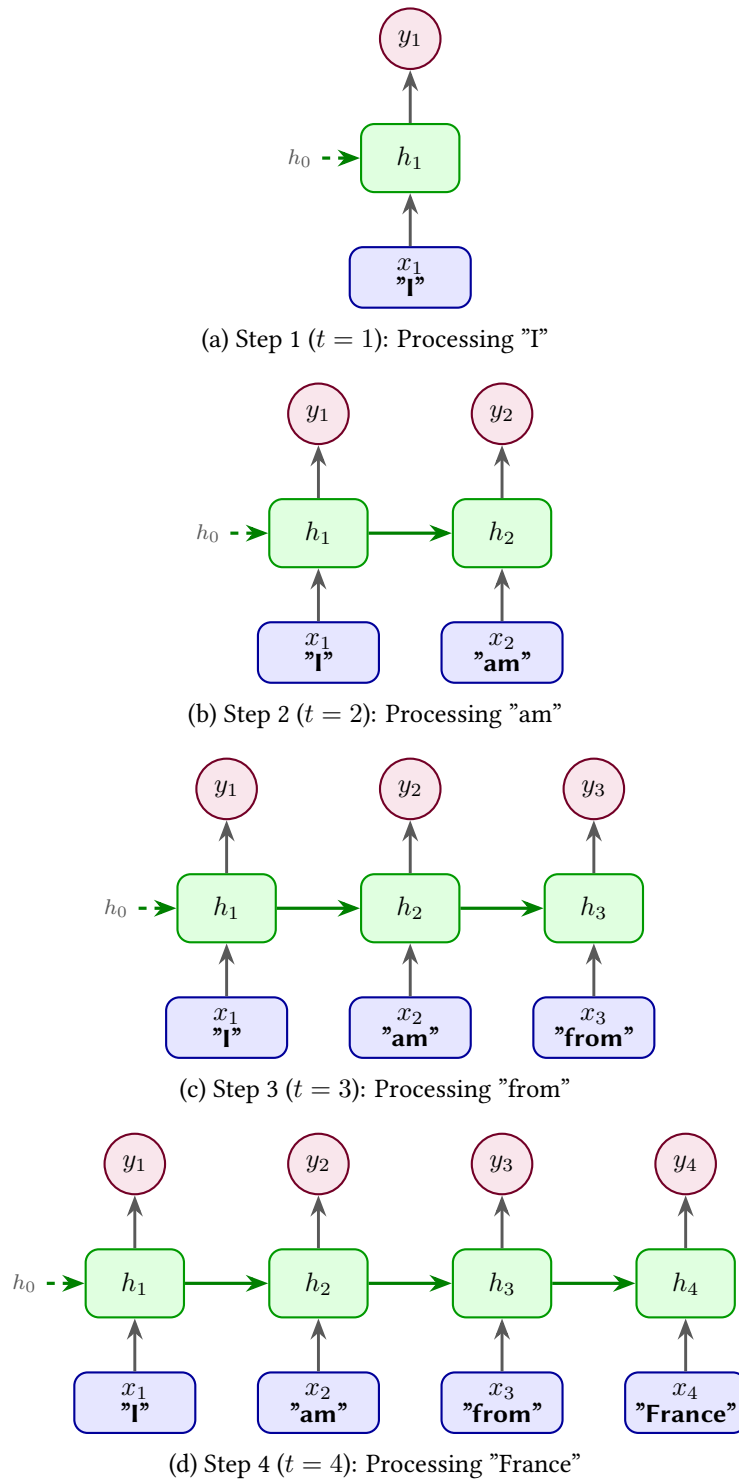


Figure 6.5: Vertical step-by-step unrolling of an RNN processing sequence tokens sequentially.

Symbol	Name	Description
x_t	Input Vector	Input token embedding at step t
h_t	Hidden State	Memory vector at step t
h_{t-1}	Previous Hidden State	Accumulated memory vector from step $t - 1$
y_t	Output Logits	Unnormalized prediction scores at step t
$\mathbf{W}_{xh}$	Input-to-Hidden Weight	Transformation matrix for current input x_t
$\mathbf{W}_{hh}$	Hidden-to-Hidden Weight	Recurrent matrix for prior memory h_{t-1}
$\mathbf{W}_{hy}$	Hidden-to-Output Weight	Projection matrix from hidden state to output space
b_h, b_y	Bias Vectors	Bias terms for hidden update and output layer

Table 6.1: RNN Notation Summary

$$h_t = \tanh(\mathbf{W}_{xh}x_t + \mathbf{W}_{hh}h_{t-1} + b_h) \quad (6.10)$$

$$y_t = \mathbf{W}_{hy}h_t + b_y \quad (6.11)$$

```

1 import numpy as np
2
3 class VanillaRNN:
4     def __init__(self, vocab_size, hidden_size):
5         # Initialize shared weights
6         self.W_xh = np.random.randn(hidden_size, vocab_size)
7         self.W_hh = np.random.randn(hidden_size, hidden_size)
8         self.W_hy = np.random.randn(vocab_size, hidden_size)
9         self.b_h = np.zeros((hidden_size, 1))
10        self.b_y = np.zeros((vocab_size, 1))
11
12    def forward(self, inputs, h_prev):
13        # inputs is a list of one-hot encoded tokens
14        hidden_states = []
15        outputs = []
16        h_t = h_prev
17
18        for x_t in inputs:
19            # Update hidden state
20            h_t = np.tanh(self.W_xh @ x_t + self.W_hh @ h_t + self.b_h)
21            # Calculate output logits
22            y_t = self.W_hy @ h_t + self.b_y
23
24            hidden_states.append(h_t)
25            outputs.append(y_t)
26
27        return outputs, h_t

```

Listing 6.1: Basic Vanilla RNN Forward Pass

6.9.2 Backpropagation Through Time (BPTT) & Limitations

To train an RNN, we calculate the loss at each time step, and then backpropagate the error backwards through time.

However, because the hidden state is recursively multiplied by the same weight matrix $\mathbf{W}_{hh}$ at every time step, we encounter the **Vanishing Gradient Problem**. If the eigenvalues of $\mathbf{W}_{hh}$ are less than 1, the gradient shrinks exponentially as we propagate backwards. The network completely forgets early words in a long sequence!

Furthermore, the entire history of the sequence is crushed into a single, fixed-size vector h_t . This **Hidden State Bottleneck** means the network will inevitably overwrite crucial early information with new, irrelevant tokens.

6.10 LSTMs and GRUs: Gating the Memory

To fix the vanishing gradient and bottleneck problems, researchers introduced the **Long Short-Term Memory (LSTM)** network.

Instead of aggressively overwriting the hidden state at every step, the LSTM introduces a second memory line called the **Cell State (C_t)**. The Cell State acts as a highway that runs straight down the entire chain, with only minor linear interactions. Information can flow along it almost completely unchanged!

The LSTM carefully controls what gets added or removed from this highway using **Gates** (sigmoid neural net layers that output values between 0 and 1).

1. **Forget Gate (f_t)**: Decides what existing information in the cell state is no longer relevant and should be thrown away.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (6.12)$$

2. **Input Gate (i_t) & Candidate State ($\tilde{C}_t$)**: The input gate decides which values to update, while the tanh layer creates a vector of new candidate values to be added.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (6.13)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (6.14)$$

3. **Update the Cell State (C_t)**: We multiply the old state by the forget gate, and add the scaled new candidate values.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (6.15)$$

4. **Output Gate (o_t)**: Decides what part of the internal cell state should be exposed to the outside world as the new hidden state h_t .

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (6.16)$$

$$h_t = o_t \odot \tanh(C_t) \quad (6.17)$$

6.10.1 Gated Recurrent Units (GRUs)

A GRU is a streamlined variant of the LSTM. It combines the forget and input gates into a single **Update Gate (z_t)** and merges the cell state and hidden state together. It also features a **Reset Gate (r_t)** to decide how much past information to ignore.

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (6.18)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (6.19)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t]) \quad (6.20)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (6.21)$$

GRUs are computationally cheaper than LSTMs while achieving highly comparable performance on most sequence tasks.

6.11 Sampling: Making the Model Creative

Once an RNN or LSTM computes its final logits, we use the Softmax function to get a probability distribution over the vocabulary. But if we always pick the word with the absolute highest probability (Greedy Search), our generated text becomes repetitive, boring, and robotic.

We can inject controlled randomness using sampling strategies:

- **Temperature (T):** Scales the logits before softmax: $P_i = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)}$. A temperature < 1 makes the model highly confident and deterministic. A temperature > 1 flattens the distribution, increasing creativity and randomness.
- **Top- k Sampling:** The model sorts the vocabulary by probability and only samples from the top k highest choices (e.g., $k = 50$). The rest are truncated to 0.
- **Top- p (Nucleus) Sampling:** The model samples from the smallest set of top words whose cumulative probability exceeds a threshold p (e.g., 0.90). This dynamically adjusts the candidate pool size based on the model's confidence.

Conclusion of Chapter 6: We have successfully given our neural networks a memory. LSTMs and GRUs conquered the vanishing gradient problem by elegantly gating their information flow. However, they share one fatal, inescapable flaw: **Sequential Computation**. Because h_t absolutely requires h_{t-1} to be computed first, we cannot parallelize the training process across modern GPUs. To scale to the massive datasets of today, we need an architecture that drops recurrence entirely. In the next chapter, we will enter the era of Attention and Transformers!

Chapter 7

Transformers & State Space Models

Shloka & Atharva

[Link to session recording](#)

7.1 Warmup: The Sequential Bottleneck

Imagine trying to read a 1,000-page novel through a tiny peephole, one word at a time. To understand the ending, you must compress the entire plot, every character arc, and every subtle foreshadowing into a single, tiny notebook (your hidden state h_t). By the time you reach page 1,000, the details of page 1 are inevitably crushed, overwritten, or forgotten.

This was the tragedy of Recurrent Neural Networks (RNNs) and LSTMs. No matter how clever the gating mechanisms were, information from the first token still had to pass through $O(N)$ sequential transformations to influence the N -th token. Over long distances, the signal degraded.

Even worse, RNNs enforce a **Sequential Dependency**. To process word 100, the GPU must sit idle waiting for the calculation of word 99 to finish. You cannot parallelize this across the thousands of cores available on modern hardware.

If we want to train models on the entire internet, we need an architecture that can look at *every single word in a book simultaneously*. We need a mechanism that provides a direct, $O(1)$ constant-length path between any two words, regardless of their physical distance. This mechanism is **Attention**.

7.2 Attention is All You Need

The core idea of Self-Attention is simple: when the model reads a word, it simultaneously looks at every other word in the sequence to gather context, dynamically deciding which words are most relevant to understanding the current word.

Take the sentence: *"The animal didn't cross the street because it was too tired."* What does the word *"it"* refer to? The street, or the animal? To a human, it's obvious. To a machine, it requires forming a strong mathematical connection between the vector for *"it"* and the vector for *"animal"*.

7.2.1 Queries, Keys, and Values

Attention achieves this using three distinct matrices, inspired by database retrieval. Imagine you are standing at a workbench building a clock, facing a massive wall of tiny component drawers.

- **Query (Q):** The token you are actively processing. It's the thought in your head: *"I need a gear."*
- **Key (K):** The labels taped to the outside of every single drawer: *"Screws"*, *"Gears"*, *"Springs"*.

- **Value (V):** The actual physical component inside the drawer.

Every token in our input matrix X is multiplied by three separate, learned weight matrices (W^Q, W^K, W^V) to generate its specific Query, Key, and Value vectors:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (7.1)$$

7.2.2 The Scaled Dot-Product Math

How do we mathematically measure how well a Query matches a Key? We take the **Dot Product**. By computing QK^T , we calculate the similarity between every single query and every single key in the sentence simultaneously.

However, in high-dimensional spaces (e.g., $d_k = 512$), dot products can yield extremely large numbers. If we feed massive numbers into a Softmax function, it gets pushed into its extreme flat regions where the gradient is exactly zero, bringing backpropagation to a halt. To fix this, we scale the dot product by dividing by the square root of the dimension size ($\sqrt{d_k}$).

Finally, we pass these scaled scores through a Softmax function to turn them into a valid probability distribution (percentages that sum to 1). We use these percentages to take a weighted sum of the Values.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (7.2)$$

If the softmax determines that the word "bank" should pay 90% attention to "river" and 10% to "the", the math computes exactly $0.90 \times V_{\text{river}} + 0.10 \times V_{\text{the}}$, producing a newly updated, context-aware vector for "bank".

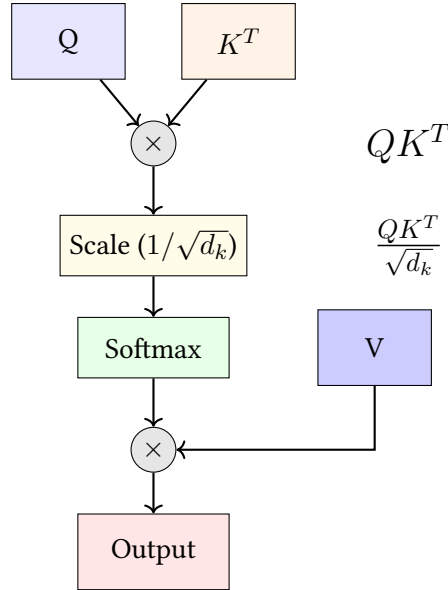


Figure 7.1: Scaled Dot-Product Attention mechanism.

7.2.3 Masked Self-Attention (Preventing Cheating)

During training, a Transformer processes massive batches of entire sentences at once. If its job is to predict the *next* word, what stops word #3 from simply querying word #4 to figure out what it is?

We must mathematically blindfold the model to the future. We do this by adding a **Mask Matrix** (M) right before the softmax. M is a lower-triangular matrix. For all "legal" connections (past words), it holds 0. For all "illegal" connections (future words), it holds $-\infty$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \quad (7.3)$$

Because $e^{-\infty} = 0$, the softmax function assigns exactly 0% attention to all future tokens, forcing the model to predict the future using only the past.

7.3 The Transformer Architecture

7.3.1 Positional Encoding

Because Attention processes all words simultaneously via matrix multiplication, it has absolutely zero concept of sequence order. The sentence "The dog bit the man" is mathematically identical to "The man bit the dog".

We must explicitly inject a "location tag" into every word's embedding. We do this using **Positional Encodings**, which generate unique combinations of sine and cosine waves for every absolute position in the sequence.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (7.4)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (7.5)$$

This vector of frequencies is simply added element-wise to the raw word embedding. No two positions will ever have the exact same frequency signature.

7.3.2 Multi-Head Attention

A single attention mechanism struggles to track multiple distinct relationships (e.g., spatial layout, grammar, verb-subject) perfectly at the same time.

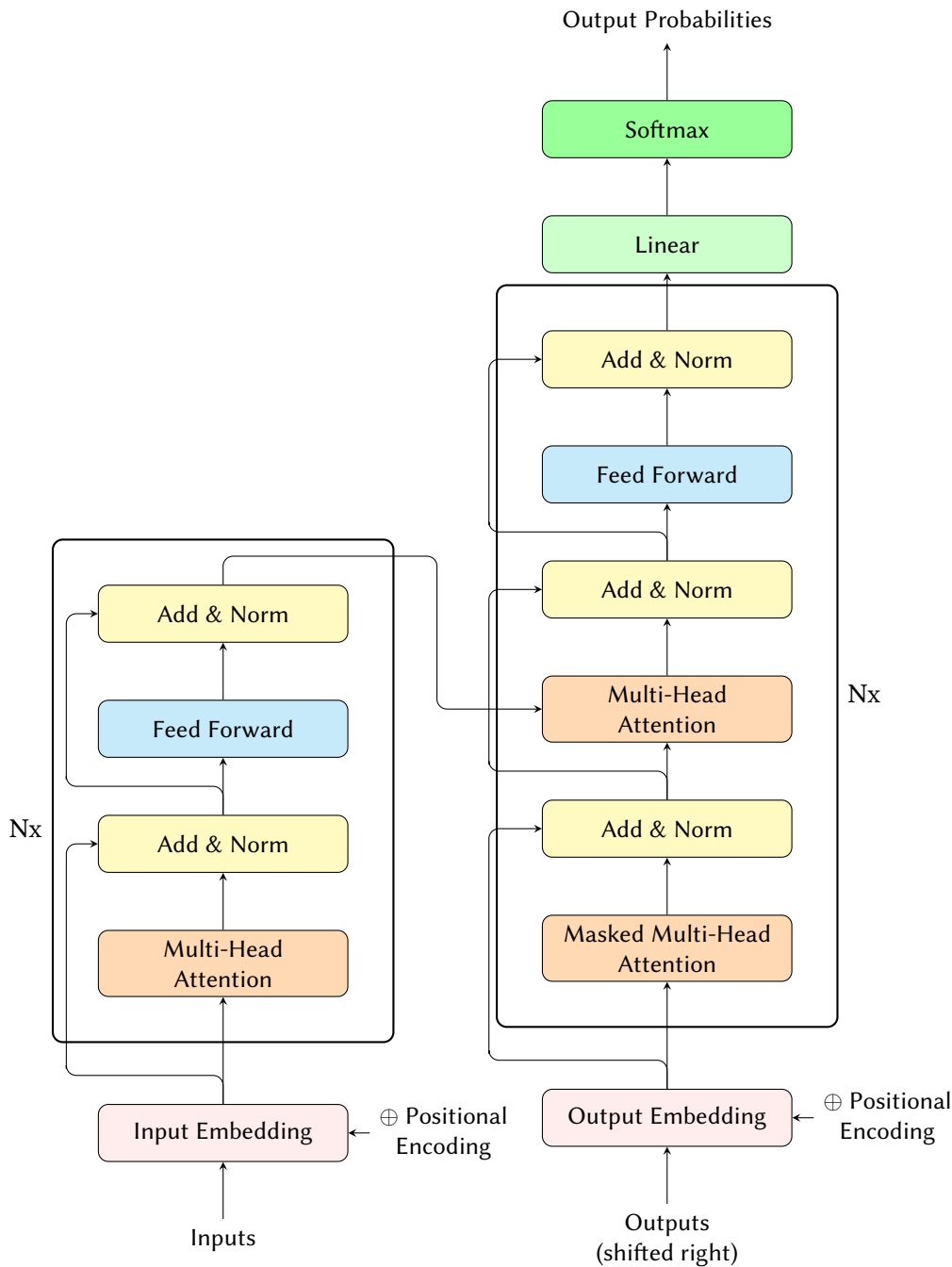
Multi-Head Attention splits the embedding dimension across h independent heads (e.g., 512 dimensions $\div$ 8 heads = 64 dimensions per head). Each head gets its own random weight matrices, allowing Head 1 to learn grammar, while Head 2 learns spatial relationships. Because the dimensions are smaller, calculating 8 heads in parallel takes roughly the same compute as one massive head. The outputs are concatenated and multiplied by a final projection matrix W^O .

7.3.3 The Transformer Block: Adding the MLP

Attention moves information *between* tokens. But we also need to process that accumulated information *within* each token individually.

After Attention, the vector is passed through a **Feed Forward Network (MLP)**. This consists of a linear expansion layer (usually expanding dimensions by 4x), a non-linear activation (like ReLU or GELU), and a linear reduction layer back to the original dimension. This allows the model to project the data into a high-dimensional space to uncover complex, non-linear representations before compressing it back down.

Every sub-layer is wrapped in a **Residual Connection** (adding the input to the output) and **Layer Normalization** to guarantee stable gradients across dozens of stacked blocks.



7.4 The Sequence Modeling Crisis: Enter SSMs

Transformers dominated the world (GPT, BERT, Claude). However, they have a massive, fatal flaw: **The Attention Matrix is $O(L^2)$** . If you double the sequence length L , the number of dot-products you must compute quadruples. A 100K token context window requires generating a 10 Billion element attention matrix for *every single head, in every single layer*.

Furthermore, during autoregressive generation (inference), the Transformer must keep the Keys and Values for every single past token in GPU memory (The **KV-Cache**). This memory grows continuously with every generated word, severely limiting batch sizes.

The Wish List: We want an architecture that has:

1. Parallelizable training (like Transformers, $O(L)$ time).
2. $O(1)$ constant compute and memory per token during inference (like RNNs, no KV-cache).

3. A principled mathematical way to remember very long contexts without vanishing gradients.

7.5 State Space Models (SSMs)

To achieve this, researchers looked to control theory and differential equations. A continuous State Space Model is defined by two simple equations mapping an input signal $x(t)$ to a hidden state $h(t)$ and an output $y(t)$:

$$h'(t) = Ah(t) + Bx(t) \quad (7.6)$$

$$y(t) = Ch(t) + Dx(t) \quad (7.7)$$

Here, A dictates how the memory drifts and forgets over time, B dictates how new inputs are written to memory, and C dictates how memory is read to the output.

7.5.1 Discretization: Zero-Order Hold

Language is discrete (tokens), not continuous. We must discretize the continuous ODE using a step size Δ (which the model learns). Using the Zero-Order Hold (ZOH) method, we convert the continuous matrices (A, B) into discrete, per-step matrices $(\bar{A}, \bar{B})$:

$$\bar{A} = \exp(\Delta A) \quad (7.8)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B \quad (7.9)$$

This gives us a highly rigorous **Recurrent Update Rule**:

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k \quad (7.10)$$

7.5.2 The Duality: Recurrence vs. Convolution

If we assume the system is **Linear and Time-Invariant (LTI)**—meaning A, B, C are fixed constants that never change based on the data—a magical mathematical duality emerges.

If we unroll the recurrence equation starting from $h_0 = 0$:

$$h_1 = \bar{B}x_1$$

$$h_2 = \bar{A}\bar{B}x_1 + \bar{B}x_2$$

$$y_3 = C\bar{A}^2\bar{B}x_1 + C\bar{A}\bar{B}x_2 + C\bar{B}x_3$$

This is exactly a **Convolution**! The output is simply the input sequence convolved with a fixed, pre-computable Kernel K :

$$K = (C\bar{B}, C\bar{A}\bar{B}, C\bar{A}^2\bar{B}, \dots) \quad (7.11)$$

The S4 Engine: This solves the wish list! During training, we use the Fast Fourier Transform (FFT) to compute the convolution over the whole sequence in parallel. During inference, we discard the convolution, switch to the h_k recurrence, and generate tokens with $O(1)$ constant memory!

Note: S4 models initialize the A matrix using HiPPO (High-order Polynomial Projection Operators), specifically Legendre polynomials, which mathematically guarantees the state vector retains an optimal compression of the entire history.

7.6 Mamba: The Selective SSM

LTI models (like S4) are beautiful, but they fail at complex NLP tasks like **Selective Copying** or **Induction Heads**. Why? Because $\bar{A}$ and $\bar{B}$ are constants. The model applies the exact same memory-updating logic to a crucial noun as it does to a useless comma. It cannot dynamically choose to ignore irrelevant tokens.

Mamba (S6) makes a radical leap: It destroys the Time-Invariant assumption. It makes the parameters B , C , and crucially the step size Δ , **Input-Dependent**.

$$B_k = \text{Linear}(x_k) \quad (7.12)$$

$$C_k = \text{Linear}(x_k) \quad (7.13)$$

$$\Delta_k = \text{Softplus}(\text{Linear}(x_k)) \quad (7.14)$$

Now, $\bar{A}_k = \exp(\Delta_k A)$. The matrix $\bar{A}$ changes at every single time step! Δ_k acts as a powerful gate. If the model sees a useless word, it predicts a tiny Δ , making $\bar{A} \approx I$, perfectly preserving the old memory. If it sees a crucial word, it predicts a large Δ , forcing the model to absorb it.

7.6.1 Hardware-Aware Optimization

By making the matrices input-dependent, **Mamba breaks the convolution trick**. You can no longer pre-compute a fixed Kernel K . Naively, we are forced back to slow, sequential RNN computation.

To train Mamba in parallel, the authors used hardcore GPU engineering:

1. **Parallel Scan (Prefix Sum)**: Because the recurrent update is an associative operator, we can compute partial state updates in a binary tree structure across GPU threads, dropping sequential time to $O(L/T)$.
2. **Kernel Fusion & SRAM**: GPU memory transfer (HBM to SRAM) is the true bottleneck. Mamba fuses the discretization and scan operations into a single custom CUDA kernel. It loads the parameters into ultra-fast SRAM, computes the entire state sequence, and only writes the final outputs back to HBM.
3. **Recomputation**: Instead of saving massive hidden states to memory for backpropagation, Mamba simply recomputes them on the fly during the backward pass inside SRAM. Redoing the math is actually faster than reading from slow GPU memory!

```

1 import numpy as np
2
3 # 1. Simplified Scaled Dot-Product Attention
4 def self_attention(Q, K, V, mask=None):
5     d_k = Q.shape[-1]
6     scores = np.dot(Q, K.T) / np.sqrt(d_k)
7
8     if mask is not None:
9         scores = scores + mask # mask contains 0s and -inf
10
11     # Softmax row-wise
12     probs = np.exp(scores) / np.sum(np.exp(scores), axis=-1, keepdims=True)
13     return np.dot(probs, V)
14
15 # 2. Conceptual Sequential Scan for Mamba (Inference Mode)
16 def mamba_step(x_k, h_prev, A, B_proj, C_proj, Delta_proj):
17     # Parameters become input dependent!
18     B_k = B_proj(x_k)
19     C_k = C_proj(x_k)
20     delta_k = np.log(1 + np.exp(Delta_proj(x_k))) # Softplus
21
22     # Discretize (Zero-Order Hold approximation)
23     A_bar = np.exp(delta_k * A)
24     B_bar = delta_k * B_k # Simplified approximation
25
26     # Recurrent Update
27     h_k = A_bar * h_prev + B_bar * x_k
28     y_k = C_k * h_k
29
30     return y_k, h_k

```

Listing 7.1: Conceptual Code for Attention and Mamba Scan

Conclusion of Chapter 7: The journey from Attention to Mamba illustrates the eternal battle in Deep Learning: expressivity versus computational cost. Transformers gave us perfect routing and parallel training at the cost of quadratic complexity. State Space Models utilized differential equations to bring us back to linear time and constant memory. In the next chapters, we will pivot from text to pixels, and explore how these architectures are adapted to see the world!

Chapter 8

Computer Vision Fundamentals

Madhura & Pranaav

[Link to session recording](#)

8.1 Warmup: The Visual World and the Flattening Problem

We have successfully taught our models to understand the sequential nature of text. Now, we face a new frontier: Vision. How do we teach a neural network to "see" a photograph of a cat and distinguish it from a dog?

To a computer, an image is not a picture; it is a massive 3D tensor of numbers. A standard RGB image has three dimensions: Height, Width, and Channels (Red, Green, Blue). A small 64×64 pixel image is actually a matrix of $64 \times 64 \times 3 = 12,288$ individual numbers.

If we naively apply the Multi-Layer Perceptrons (MLPs) we learned about in Chapter 5, we immediately run into a catastrophic problem. MLPs require 1D vector inputs. To feed our image into an MLP, we must **flatten** it—taking the rows of pixels and stacking them end-to-end into a single 12,288-length column.

Why is Flattening a Terrible Idea?

1. **Loss of Spatial Hierarchy:** In the real world, a pixel's meaning is heavily defined by its neighbors. An "edge" is just a sharp contrast between adjacent pixels. By flattening the image, we destroy the 2D spatial relationships. Pixel (0, 63) and pixel (1, 0) are placed right next to each other in the flattened array, even though they are on opposite sides of the image!
2. **Parameter Explosion:** If our first hidden layer has 1,000 neurons, the weight matrix connecting the flattened input to this layer will have $12,288 \times 1,000 \approx 12.2$ Million parameters. For a single layer! This massive parameter count guarantees severe overfitting and requires an impossible amount of compute.
3. **Lack of Translation Invariance:** If the cat is in the top-left of the image, the network learns weights specific to the top-left input nodes. If we feed it a new image where the exact same cat is in the bottom-right, the network will completely fail to recognize it because the pixel data is entering entirely different input nodes.

To solve vision, we need an architecture that preserves 2D structure, reuses parameters, and looks for local patterns regardless of where they appear. Enter the **Convolutional Neural Network (CNN)**.

8.2 The Convolutional Operation

Instead of connecting every input pixel to a neuron, CNNs use a mathematical operation called a **Convolution**.

Imagine holding a tiny magnifying glass—perhaps 3×3 pixels wide—and sliding it slowly across the image. Mathematically, for a 2D image I and a 2D kernel K , the discrete convolution operation produces a feature map S :

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (8.1)$$

8.2.1 Kernels and Feature Maps

A kernel is simply a small matrix of weights. As we slide (convolve) the kernel over the input image, we perform an element-wise multiplication between the kernel weights and the image pixels underneath, sum them up, and add a bias. The result is a single number.

As the kernel slides across the entire image, it generates a brand new 2D matrix called a **Feature Map**.

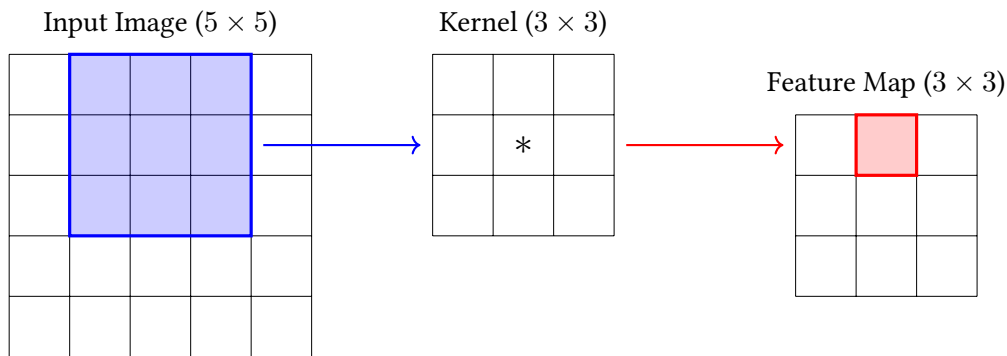


Figure 8.1: The Convolution Operation: A 3×3 kernel convolves over a local patch (receptive field) of the input to produce a single value in the feature map.

The genius of the CNN is that **the weights of the kernel are the parameters the network learns**. If the network needs to find vertical edges, gradient descent will naturally adjust the 9 weights in the 3×3 kernel to form a vertical edge detector (e.g., a Sobel filter). Because the *exact same* 3×3 kernel slides over the entire image, the network achieves **Parameter Sharing**. It only needs to learn 9 parameters to detect a vertical edge anywhere in the image!

8.2.2 Stride and Padding

Two crucial hyperparameters control the size of our output feature map:

- **Stride (s):** The step size the kernel takes as it slides. A stride of 1 means moving one pixel at a time. A stride of 2 skips every other pixel, aggressively downsampling the spatial dimensions of the feature map.
- **Padding (p):** Notice in the diagram above, a 5×5 input convolved with a 3×3 kernel (stride 1) produces a 3×3 output. We lost the edges! To preserve spatial dimensions, we can add a border of fake pixels (usually zeros) around the outside of the original image. This is called "Same" Padding, so the dimensionality is preserved. "Valid" Padding means no padding is applied (and hence dimensions reduce from one step to the next).

The exact mathematical formula for the dimension of the output feature map (n_{out}) given an input dimension (n_{in}), padding (p), kernel size (f), and stride (s) is:

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - f}{s} + 1 \right\rfloor \quad (8.2)$$

8.3 The Anatomy of a CNN Architecture

A full CNN is not just convolutions. It is a highly structured pipeline designed to extract hierarchical features. The classic pipeline is: Conv $\rightarrow$ Activation $\rightarrow$ Pool $\rightarrow$ (Repeat) $\rightarrow$ Flatten $\rightarrow$ Fully Connected.

8.3.1 The Activation Layer (ReLU)

Just like standard neural networks, the output of a convolution is strictly linear. We must pass the feature map through an activation function to learn non-linear concepts.

In modern Vision architectures, **ReLU (Rectified Linear Unit)** is almost universally used. $ReLU(x) = \max(0, x)$. It simply turns all negative pixel values in the feature map to exactly zero. It is computationally blazingly fast and helps preserve strong gradients during backpropagation.

8.3.2 The Pooling Layer

Pooling is a purely structural layer used to aggressively reduce spatial dimensions, lower memory constraints, and prevent overfitting. It requires no trainable parameters.

Similar to convolution, a window (e.g., 2×2) slides over the feature map. But instead of a dot product, it performs a fixed statistical operation:

- **Max Pooling:** Takes the maximum value in the window. This is the most common technique. It acts as an "active feature detector," identifying if a specific feature (like an edge or a color) exists anywhere in that 2×2 patch, granting the network robust translation invariance.
- **Average Pooling:** Takes the arithmetic mean. This provides a more generalized, smoothed representation, often used at the very end of deep networks (Global Average Pooling).
- **Min Pooling:** Takes the minimum value in the window. Not very commonly used, except for images with black background and a white object

8.3.3 The Flattening and Fully Connected Layers

After several blocks of Convolution and Pooling, our image has been transformed from a large spatial grid of raw colors ($64 \times 64 \times 3$) into a very deep, narrow block of highly abstract features (e.g., $4 \times 4 \times 512$).

At this point, we have successfully extracted the "meaning" of the image. We **flatten** this $4 \times 4 \times 512$ block into a 1D vector (length 8,192) and pass it into standard Fully Connected (Dense) layers. These layers act as the final classifier, looking at the presence of abstract concepts (like "has fur," "has pointy ears") to output logits for our final classes, which are normalized using Softmax, which are then used to classify the input into pre-defined labels.

8.4 Data Augmentation

Deep CNNs require an immense amount of data to avoid overfitting. If we only have 1,000 pictures of cats, the network might memorize the specific backgrounds instead of learning what a cat looks like.

Data Augmentation artificially expands our dataset during training by applying random, dynamic transformations to the images before they hit the network.

These measures help to make the model generalize better and both prevent overfitting and mitigate the lack of input training data

- **Geometric Transformations:** Randomly rotating, horizontally flipping, cropping, or zooming the image. A cat is still a cat if it's mirrored!
- **Color Space Transformations:** Randomly tweaking brightness, contrast, hue, or saturation. This teaches the model to ignore lighting conditions.

- **Kernel Filters & Erasing:** Applying slight Gaussian blur, or randomly cutting out (erasing) a black square in the middle of the image, forcing the model to learn the whole object rather than relying on a single defining feature (like just the nose).

8.5 The Deep Training Crisis: A Gradients Recap

As vision tasks became more complex, researchers realized they needed deeper architectures to capture complex hierarchies. However, as they tried to train networks with 30, 50, or 100 layers, the models completely failed to learn.

This brings us back to the fatal flaw of Backpropagation discussed in Chapter 5: **The Vanishing and Exploding Gradient Problem**.

Recall the Chain Rule. To find how much a weight in the very first convolutional layer contributed to the final error, backpropagation must multiply the local derivatives of every single layer that came after it.

- If the network uses Sigmoid activations (max derivative of 0.25) or if the initialized weights are less than 1, multiplying dozens of small fractions together causes the gradient to **vanish** to absolute zero. The early layers—which are supposed to learn foundational edges and textures—never receive the error signal and never update. The model underfits.
- Conversely, if the weights are initialized too large, repeated multiplication causes the gradient to **explode** to infinity. The weight updates become so massive that the loss oscillates wildly, resulting in NaN (Not a Number) failures.

While techniques like ReLU activations, He Initialization, and Batch Normalization (which normalizes the inputs of each layer to stabilize gradients) successfully allow us to train these massive 100-layer networks, a new problem emerged: **Computational Cost**.

Training a deep CNN from scratch requires millions of labeled images (like the ImageNet dataset) and hundreds of hours on massive GPU clusters. For a medical researcher trying to classify rare lung x-rays with a dataset of only 500 images, training from scratch is mathematically impossible; the model will instantly overfit to the 500 images.

The solution to this data and compute crisis is the most powerful paradigm in modern AI.

8.6 Transfer Learning: Standing on the Shoulders of Giants

Transfer Learning is the practice of taking a massive, deep neural network that has already been trained on a massive general dataset (the Source Task) and repurposing its learned knowledge for a specific, often smaller problem (the Target Task). Rather than training a neural network from randomly initialized weights, we begin with a model that has already learned useful representations from a large dataset and adapt it to the new problem.

8.6.1 Why does this work in Vision?

In deep CNNs, the layers learn a strict hierarchy of features:

- **Early Layers:** Learn universal, low-level features—basic geometry like diagonal edges, color gradients, and simple textures. These features are identically useful whether you are looking at a dog, a car, or an MRI scan.
- **Middle Layers:** Combine edges into simple shapes and motifs (wheels, eyes, leaves).
- **Late Layers:** Learn highly abstract, domain-specific combinations (a full dog face, a car bumper).

Because early layers learn universal truths about how pixels form geometry, we can reuse them! There is absolutely no reason to spend millions of dollars forcing a GPU to relearn what a “straight line” looks like. Transfer learning allows us to preserve this general visual knowledge while adapting only the higher-level semantic features to the new task.

8.6.2 Core Approaches: Extraction vs. Fine-Tuning

When we download a pre-trained model (like a ResNet trained on 1.4 million ImageNet photos), we chop off its final classification head (which predicts 1000 categories) and attach a brand new, randomly initialized head for our specific task (e.g., a 2-node head for Tumor vs. No Tumor).

How we train this modified network depends on our resources:

1. Feature Extraction (Freezing Layers) We “freeze” the weights of all the pre-trained layers. During backpropagation, we only calculate gradients and update weights for our newly attached head. The massive CNN acts purely as a fixed mathematical function that transforms images into rich feature vectors.

- *Pros:* Blazingly fast to train. Mathematically impossible to catastrophic-forget the pre-trained knowledge. Ideal when you have very little data.
- *Cons:* Might underfit if your target images look nothing like the source images (e.g., using a dog classifier for satellite imagery).

2. Fine-Tuning (Unfreezing Layers) We unfreeze some (or all) of the pre-trained layers and continue training the entire network on our new dataset, usually at a much lower learning rate.

- *Pros:* Allows the deeper, abstract layers to adapt their specific feature detectors to your exact domain. Yields the highest possible accuracy.
- *Cons:* Requires more compute and a larger dataset to avoid overfitting.

8.6.3 The Strategy Matrix

How do you choose your strategy? It depends on Domain Similarity and Dataset Size:

- **Small Data, Similar Domain:** Freeze the entire base network. Train only the head. (Highest risk of overfitting if you unfreeze).
- **Large Data, Similar Domain:** Fine-tune the entire network end-to-end with a low learning rate.
- **Small Data, Different Domain:** Hardest scenario. Freeze early layers, unfreeze middle layers, and use aggressive data augmentation.
- **Large Data, Different Domain:** Use the pre-trained weights merely as a highly optimized initialization, and fine-tune the whole network heavily.

8.6.4 Types of Transfer Learning

- **Inductive:** Source and target tasks differ, but labeled data exists in the target domain (e.g., taking an ImageNet classifier and fine-tuning it to classify 10 specific dog breeds).
- **Transductive:** Tasks are the same, but domains differ, and labels only exist in the source (e.g., adapting a self-driving model trained in sunny California to drive in snowy conditions using unlabelled snowy data).
- **Unsupervised:** Using a model trained on massive unlabelled data to extract features for a downstream clustering task.

```

1 import torch
2 import torch.nn as nn
3 import torchvision.models as models
4 import torchvision.transforms as transforms
5
6 # 1. Data Augmentation Pipeline
7 train_transform = transforms.Compose([

```

```
8     transforms.RandomResizedCrop(224),
9     transforms.RandomHorizontalFlip(),
10    transforms.ColorJitter(brightness=0.2, contrast=0.2),
11    transforms.ToTensor(),
12    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]))
13
14
15 # 2. Loading a Pre-trained Model for Transfer Learning
16 # Load a ResNet18 model pre-trained on ImageNet
17 base_model = models.resnet18(pretrained=True)
18
19 # 3. Feature Extraction Strategy (Freeze the base layers)
20 for param in base_model.parameters():
21     param.requires_grad = False
22
23 # 4. Replace the classification head for our specific task (e.g., 2 classes)
24 num_features = base_model.fc.in_features
25 base_model.fc = nn.Linear(num_features, 2)
26 # Note: By default, new layers have requires_grad=True
27
28 # Now, when we pass this to an optimizer, ONLY the new base_model.fc weights will update!
```

Listing 8.1: Basic CNN Pipeline with Augmentation and Transfer Learning in PyTorch

Conclusion of Chapter 8: We have successfully learned how to preserve the spatial hierarchy of images using convolutions, how to compress that data using pooling, and how to artificially expand our datasets. Most importantly, we recognized the computational impossibility of training deep networks from scratch and solved it using Transfer Learning. In the next chapter, we will look at the specific architectural breakthroughs (like VGG and ResNet) that made deep vision possible, and explore how to not just classify images, but locate multiple objects within them!

Chapter 9

Advanced Vision & Object Detection

Abhinav & Krish

[Link to session recording](#)

9.1 Warmup: The Depth Conundrum

In the previous chapter, we established the fundamental building blocks of Computer Vision: Convolution, Pooling, and Fully Connected Layers. We also learned how Transfer Learning allows us to recycle features learned from massive datasets.

But how did those massive models get so powerful in the first place?

In 2012, a model named **AlexNet** shattered the world's expectations in the ImageNet classification challenge. By leveraging GPU training and the ReLU activation function, AlexNet proved that Deep Convolutional Neural Networks were the undisputed kings of vision. AlexNet was considered incredibly deep for its time, possessing 8 trainable layers.

Immediately, the research community asked a natural question: *If 8 layers are good, why not 16? Why not 50? Why not 150?*

The intuition was simple: deeper networks can learn more complex, abstract features. However, as researchers eagerly stacked dozens of convolutional layers together, they hit a brutal, mathematical wall. The models didn't just overfit; they completely collapsed, performing worse on both the training and testing data. This chapter explores how we solved the physics of training ultra-deep networks, and how we evolved them from simply answering "What is in this image?" to "Where exactly is it?"

9.2 VGG Networks: The Power of Tiny Kernels

Before we conquered ultra-deep networks, researchers first had to optimize how convolutions were performed. AlexNet used massive kernels in its early layers (e.g., 11×11 and 5×5 filters).

In 2014, the Visual Geometry Group (VGG) at Oxford proposed a radical simplification: **Abandon large filters entirely. Only use 3×3 filters.**

9.2.1 The Receptive Field Mathematics

Why replace one 5×5 filter with two stacked 3×3 filters? Let's analyze the **Receptive Field**—the area of the original image that a neuron "sees."

- A single 5×5 convolution looks at a 5×5 pixel patch.

- If you apply a 3×3 convolution, the resulting feature map represents 3×3 patches. If you apply *another* 3×3 convolution on top of that, the second layer's neurons are mathematically observing a 5×5 patch of the original image!

Therefore, two 3×3 convolutions have the exact same effective receptive field as one 5×5 convolution. Three 3×3 convolutions equal one 7×7 convolution.

9.2.2 The Parameter Efficiency Advantage

Let's do the math on the number of learnable parameters. Assume our input has C channels, and we want an output of C channels.

- **One 5×5 layer:** The number of weights is $(5 \times 5 \times C_{in}) \times C_{out} = 25C^2$.
- **Two 3×3 layers:** The number of weights is $2 \times (3 \times 3 \times C_{in} \times C_{out}) = 18C^2$.

By using stacked 3×3 filters, VGG achieved the exact same visual coverage while reducing the number of parameters by nearly 30%! Furthermore, because we now have two convolutional layers instead of one, we get to insert *two* ReLU activation functions. This injects more non-linearity into the network, allowing it to learn more complex decision boundaries.

VGG16 (16 layers) and VGG19 became legendary architectures, proving that uniform, simple depth vastly outperforms shallow networks with large, clumsy filters.

9.3 ResNets and The Degradation Problem

VGG pushed networks to 19 layers. But what happened when researchers tried 30 or 50 layers?

As depth increased, accuracy saturated and then rapidly degraded. Surprisingly, this was **not overfitting**. The error on the *training* data itself was higher for a 56-layer network than a 20-layer network.

Theoretically, a deeper network should never perform worse than a shallower one. If a 20-layer network finds the perfect solution, a 56-layer network could simply copy those 20 layers and turn the remaining 36 layers into **Identity Mappings** (layers that do nothing but pass the data forward unchanged: $f(x) = x$).

However, Gradient Descent fundamentally struggles to learn an identity mapping through multiple layers of non-linear activations. Furthermore, as the error backpropagates through 50 layers of multiplications, the gradients **vanish to zero** (as discussed in Chapter 5). The early layers freeze.

9.3.1 The Residual Block Highway

In 2015, Microsoft Research solved this with the **Residual Network (ResNet)**. Instead of hoping the network would figure out how to pass data cleanly through its layers, they built physical bypasses.

They introduced the **Skip Connection**. The input x is explicitly routed around the convolutional layers and added directly to their output.

Let the non-linear convolutional layers represent a function $F(x)$. In a standard network, the output is simply $F(x)$. In a ResNet block, the output becomes:

$$H(x) = F(x) + x \tag{9.1}$$

9.3.2 The Gradient Miracle

Why is $F(x) + x$ so brilliant? Look at what happens during backpropagation. We take the derivative of the output with respect to the input:

$$\frac{\partial H(x)}{\partial x} = \frac{\partial F(x)}{\partial x} + 1 \tag{9.2}$$

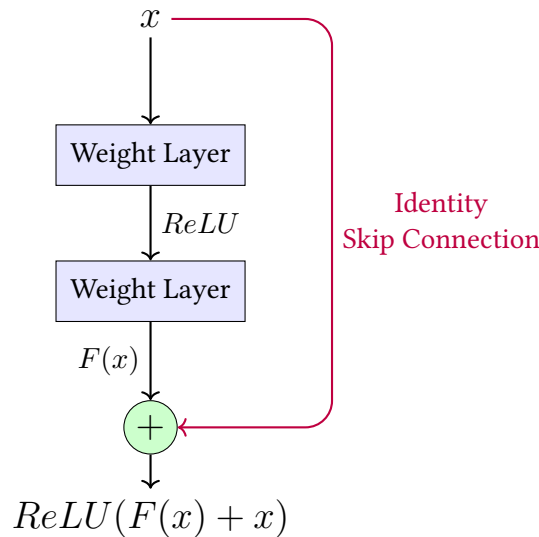


Figure 9.1: A Residual Block. The gradient can flow backward flawlessly through the identity shortcut.

That $+1$ is a miracle. It acts as an absolute gradient highway! Even if the gradient inside the heavy convolutional layers $\frac{\partial F(x)}{\partial x}$ vanishes completely to 0, the $+1$ guarantees that the gradient flowing back to the earlier layers will *never* be less than 1.

Additionally, the block no longer has to learn the full mapping $H(x)$. It only has to learn the **Residual** difference $F(x) = H(x) - x$. If the optimal state is the identity mapping, the network simply drives the weights of $F(x)$ to zero, which is mathematically incredibly easy for Gradient Descent.

With this simple addition, researchers successfully trained ResNet-152 (152 layers!), obliterating previous records and making ResNet the universal backbone for modern Computer Vision.

```

1 import torch
2 import torch.nn as nn
3
4 class ResidualBlock(nn.Module):
5     def __init__(self, in_channels, out_channels):
6         super(ResidualBlock, self).__init__()
7         # F(x) path: Two 3x3 convolutions
8         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
9         self.relu = nn.ReLU()
10        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
11
12        # Shortcut path: Identity (if dimensions match) or a 1x1 conv to match them
13        self.shortcut = nn.Sequential()
14        if in_channels != out_channels:
15            self.shortcut = nn.Conv2d(in_channels, out_channels, kernel_size=1)
16
17    def forward(self, x):
18        # Calculate F(x)
19        out = self.relu(self.conv1(x))
20        out = self.conv2(out)
21
22        # Add x to F(x)
23        out += self.shortcut(x)
24
25        # Final activation
26        out = self.relu(out)
27        return out

```

Listing 9.1: Basic Implementation of a Residual Block in PyTorch

9.4 From Classification to Object Detection

Up to this point, our networks (like VGG and ResNet) end in a flattening layer and a Softmax function, outputting a single label like "Dog." But what if an image contains a dog, a cat, and three cars?

Object Detection requires the model to output two things for every object:

1. **Classification:** What is the object? (Label probabilities).
2. **Localization:** Where is it? A **Bounding Box** parameterized by (x, y, w, h) , where (x, y) is the center coordinate, and (w, h) are the width and height.

9.4.1 Two-Stage Detectors (R-CNN Family)

Early deep learning detectors (R-CNN, Fast R-CNN, Faster R-CNN) split this problem into two distinct stages:

- **Stage 1 (Region Proposal):** An algorithm scans the image and generates thousands of "proposals" (boxes that *might* contain an object based on edges or textures).
- **Stage 2 (Classification):** Each proposed box is cropped, warped, and fed through a heavy CNN (like ResNet) to determine if it is actually an object, and to refine the box boundaries.

The Verdict: Highly accurate, but incredibly slow. R-CNN took 47 seconds to process a *single image*. Even Faster R-CNN struggled to hit real-time video framerates because evaluating thousands of proposed boxes is computationally suffocating.

9.5 YOLO: You Only Look Once

In 2015, Joseph Redmon revolutionized the field by framing object detection not as a two-stage cropping problem, but as a **Single Regression Problem**.

YOLO (You Only Look Once) passes the entire image through a single convolutional network exactly one time.

9.5.1 The Grid System

YOLO divides the input image into an $S \times S$ grid (e.g., 7×7). The fundamental rule of YOLO is: **If the center of an object falls into a grid cell, that specific grid cell is solely responsible for detecting that object.**

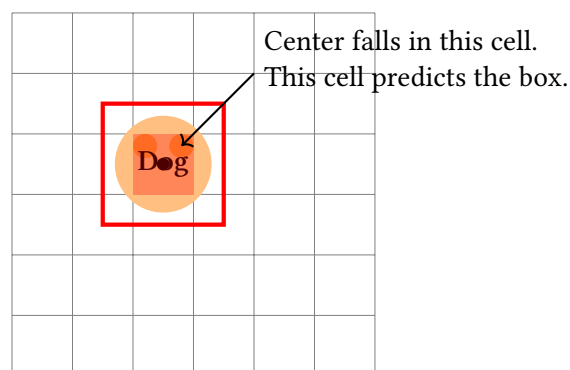


Figure 9.2: YOLO divides the image into a grid. The cell containing the object's center point is responsible for it.

9.5.2 The Prediction Tensor

Each individual grid cell predicts exactly B bounding boxes. For each bounding box, it outputs 5 numbers: $[x, y, w, h, \text{Confidence}]$.

- x, y : The center coordinates of the box relative to the bounds of the grid cell.
- w, h : The width and height of the box relative to the entire image.
- **Confidence**: The probability that an object actually exists in this box, multiplied by the Intersection Over Union (IoU) of the predicted box and the ground truth.

Simultaneously, each grid cell predicts C conditional class probabilities (e.g., if there is an object, what is the probability it is a Dog vs. a Cat).

Therefore, the final output of the YOLO CNN is a massive 3D tensor of shape:

$$S \times S \times (B \times 5 + C) \quad (9.3)$$

By formatting localization as a pure tensor regression problem, YOLO eliminates the need for region proposals. It processes the whole image concurrently, achieving speeds of 45 to 155 Frames Per Second (FPS). It effectively bridged the gap between deep learning and real-time robotics.

9.6 The Edge of Vision: UNet and ViT

YOLO answers "where is the object" with a rectangle. That is enough for most detection tasks, but it is not enough for a radiologist tracing the exact boundary of a tumor, or a self-driving car deciding precisely which pixels belong to the road versus the sidewalk. This is **Semantic Segmentation**: instead of one label per image, or one box per object, the model must output one class label for *every single pixel*.

The obstacle is the same one we ran into with classifiers throughout this chapter. A CNN encoder is built to recognize *what* an object is, and it does this by repeatedly pooling the image down into smaller, deeper feature maps. Each pooling step throws away spatial resolution in exchange for stronger semantic features. But segmentation needs exactly the opposite: a dense, pixel-perfect output that matches the original image resolution. A decoder sitting on top of a heavily compressed bottleneck has no way to recover fine object boundaries that the encoder already destroyed.

9.6.1 The Encoder-Decoder Architecture

In 2015, Ronneberger, Fischer, and Brox proposed the UNet, originally designed for segmenting cell boundaries in biomedical microscopy images. The architecture is named after its shape when drawn out: a symmetric U.

1. **The Contracting Path (Encoder)**: A standard stack of convolution and max-pooling blocks. Spatial resolution is halved and channel depth is doubled at every stage, exactly like a VGG-style backbone.
2. **The Bottleneck**: The most compressed representation in the network – lowest resolution, highest channel depth.
3. **The Expanding Path (Decoder)**: A mirror image of the encoder. At every stage, the feature map is upsampled back toward the original resolution while channel depth is halved.
4. **Skip Connections**: At each resolution level, the raw encoder feature map is copied directly across the "U" and concatenated onto the matching decoder stage, before that stage's convolutions run.

The skip connections are the reason UNet actually works. They hand the decoder a second, uncorrupted copy of the fine spatial detail that pooling destroyed, at every scale, so the decoder is never forced to reconstruct boundaries from the bottleneck alone.

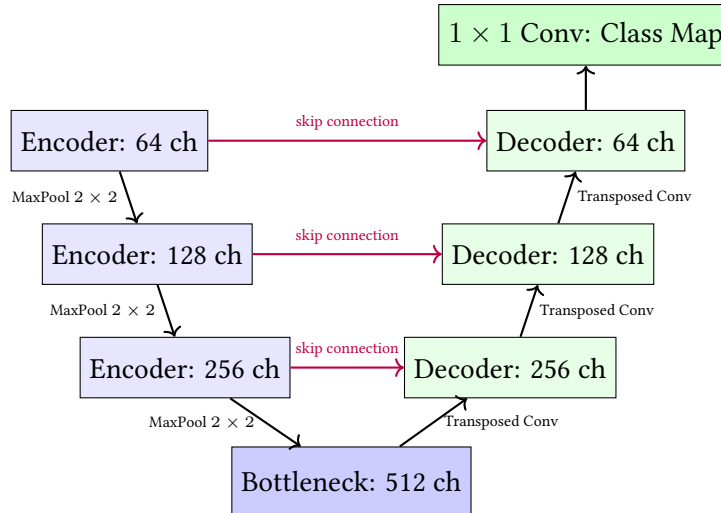


Figure 9.3: The UNet architecture. The contracting path (left) halves spatial resolution and doubles channels at every stage. The expanding path (right) mirrors this in reverse. The purple skip connections concatenate the encoder feature map directly onto the matching decoder stage.

9.6.2 Transposed Convolution

The decoder needs a way to increase spatial resolution at each stage. Fixed interpolation methods, such as nearest-neighbor or bilinear upsampling, have no learnable parameters at all. UNet instead typically uses a **Transposed Convolution**, which runs a standard convolution “in reverse”: each input value is multiplied by a learnable kernel and the result is stamped onto a larger output grid, with overlapping regions summed together. Because the kernel weights are learned through backpropagation, the network can actively learn how to reconstruct sharp boundaries, at the cost of occasionally introducing regular checkerboard artifacts when the kernel size and stride do not divide evenly.

The output spatial size of a transposed convolution is given by:

$$H_{out} = (H_{in} - 1) \cdot S - 2P + K \quad (9.4)$$

where S is the stride, P is the padding, and K is the kernel size. This is simply the standard convolution size formula solved for H_{in} in terms of H_{out} , which is why the operation reverses the *shape* of a convolution.

9.6.3 Skip Connections: Concatenation versus Addition

We already met skip connections in ResNet, where they solved the vanishing gradient problem. UNet’s skip connections look similar but combine information in a fundamentally different way.

Property	ResNet (Additive)	UNet (Concatenation)
Combination rule	$y = F(x) + x$	$y = [F(x); x]$
Channel count	Unchanged	Doubled at each level
Preserves	Gradient signal	Raw spatial detail
Requires	$F(x)$ and x same shape	Only spatial dimensions must match

Table 9.1: Additive skip connections force the network to learn only a residual correction, keeping every feature map in the same vector space. Concatenative skip connections instead hand the next layer both representations side by side along the channel axis, letting subsequent convolutions learn how to fuse them.

Because concatenation doubles the channel count going into the next convolution block, that block must explicitly account for it – this is visible directly in the code below, where each decoder’s convolution block always expects twice the channels of its own upsampled input.

9.6.4 The 1×1 Convolution

UNet’s final layer is almost always a 1×1 convolution. A 1×1 kernel spans the entire channel depth of a feature map but looks at only a single spatial pixel at a time, so it performs a learned linear combination across channels without touching the spatial dimensions. Given an input of shape $H \times W \times C_{in}$, a 1×1 convolution with C_{out} filters produces an output of shape $H \times W \times C_{out}$.

This is exactly what is needed at the last step of UNet: the final decoder block outputs many channels of fused features per pixel, but the task requires exactly one score per segmentation class. A 1×1 convolution collapses the channel depth to the required number of classes while leaving the pixel-perfect spatial resolution untouched. A 3×3 convolution at this stage would unnecessarily blur neighboring pixels together, ruining crisp boundaries at the final step, while using roughly nine times as many parameters for no benefit.

UNet’s encoder-decoder-with-skip-connections template has since become one of the most reused patterns in deep learning: organ and tumor segmentation in medical imaging, land-cover classification from satellite imagery, cell segmentation in microscopy, and, more recently, the denoising network at the core of diffusion-based image generation models such as Stable Diffusion, which predicts noise at every pixel using the exact same architecture.

```

1 import torch.nn as nn
2
3 class DoubleConv(nn.Module):
4     """Two 3x3 conv + BatchNorm + ReLU blocks, the basic UNet unit."""
5     def __init__(self, in_channels, out_channels):
6         super().__init__()
7         self.double_conv = nn.Sequential(
8             nn.Conv2d(in_channels, out_channels, 3, padding=1, bias=False),
9             nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True),
10            nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False),
11            nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True),
12        )
13    def forward(self, x):
14        return self.double_conv(x)
15
16 class UNet(nn.Module):
17    def __init__(self, in_channels=3, num_classes=2):
18        super().__init__()
19        # Contracting path
20        self.inc = DoubleConv(in_channels, 64)
21        self.down1 = nn.Sequential(nn.MaxPool2d(2), DoubleConv(64, 128))
22        self.down2 = nn.Sequential(nn.MaxPool2d(2), DoubleConv(128, 256))
23
24        # Expanding path -- note the doubled channel counts after concat
25        self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
26        self.conv_up1 = DoubleConv(256, 128) # 128 (skip) + 128 (up) = 256
27        self.up2 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
28        self.conv_up2 = DoubleConv(128, 64) # 64 (skip) + 64 (up) = 128
29
30        # 1x1 conv: collapse channels to per-pixel class scores
31        self.head = nn.Conv2d(64, num_classes, kernel_size=1)
32
33    def forward(self, x):
34        x1 = self.inc(x) # kept for the first skip connection
35        x2 = self.down1(x1) # kept for the second skip connection
36        x3 = self.down2(x2) # bottleneck
37        x = self.up1(x3)
38        x = torch.cat([x, x2], dim=1) # concatenative skip connection
39        x = self.conv_up1(x)
40        x = self.up2(x)
41        x = torch.cat([x, x1], dim=1) # concatenative skip connection
42        x = self.conv_up2(x)
43        return self.head(x) # (B, num_classes, H, W) -- dense prediction

```

Listing 9.2: A Minimal UNet with Concatenative Skip Connections (PyTorch)

9.7 Vision Transformers (ViT)

UNet still relies entirely on convolution – it changed what we predict, not how we look at pixels. In 2020, a Google Brain paper titled “An Image is Worth 16x16 Words” asked a more direct question: if the Transformer (Chapter 7) handled language by treating a sentence as a sequence of token embeddings passed through self-attention, why not treat an image as a sequence of patch embeddings and process it the exact same way?

The **Vision Transformer** drops convolution and its built-in assumptions of locality and translation invariance entirely. Every patch attends to every other patch through self-attention from the very first layer, and the network is left to learn spatial relationships purely from data rather than from a hand-designed kernel structure.

9.7.1 Patch Embeddings

Before any attention can happen, the image has to be converted into a sequence, the same way a sentence is tokenized into words. Table 9.2 walks through this for the standard ViT configuration on a 224×224 RGB image with 16×16 patches.

Step	Input Shape	Operation	Output Shape
1. Patch division	$224 \times 224 \times 3$	Split into non-overlapping 16×16 patches	196 patches
2. Flattening	$196 \times (16 \times 16 \times 3)$	Flatten each patch into a 1D vector	196×768
3. Linear projection	196×768	Multiply by learned matrix E into embedding dimension D	$196 \times D$
4. Prepend CLS token	$196 \times D$	Prepend a learnable classification token	$197 \times D$
5. Position addition	$197 \times D$	Add learnable 1D positional embeddings	$197 \times D$

Table 9.2: The patch embedding pipeline. 16×16 patches of a 224×224 image tile a 14×14 grid, giving $14^2 = 196$ patches, each flattened from $16 \times 16 \times 3 = 768$ raw pixel values.

In practice, steps 1 through 3 are implemented as a single strided convolution: a 16×16 kernel with stride 16 and D output channels applied once is mathematically identical to slicing the image into non-overlapping patches and applying the same linear map to each one.

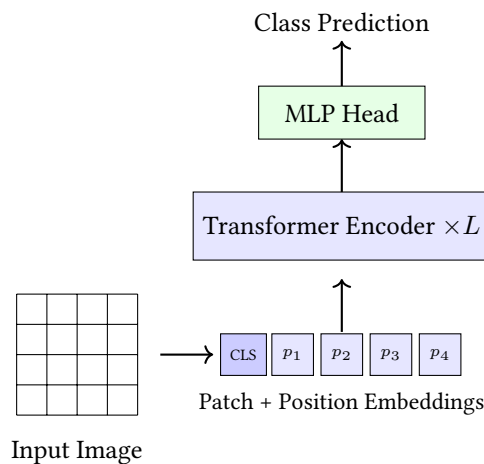


Figure 9.4: The Vision Transformer pipeline. Non-overlapping patches are linearly embedded, a learnable CLS token is prepended, position embeddings are added, and the sequence is passed through a standard Transformer encoder. The output CLS token is fed to an MLP head for classification.

```

1 import torch
2 import torch.nn as nn
3
4 class PatchEmbedding(nn.Module):
5     def __init__(self, image_size=224, patch_size=16, in_channels=3, embed_dim=768):
6         super().__init__()
7         self.num_patches = (image_size // patch_size) ** 2
8         # A single strided conv == "slice into patches + flatten + linear project"
9         self.proj = nn.Conv2d(in_channels, embed_dim,
10                                kernel_size=patch_size, stride=patch_size)
11         self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
12         self.pos_embed = nn.Parameter(torch.zeros(1, 1 + self.num_patches, embed_dim))
13
14     def forward(self, x):
15         B = x.shape[0]
16         x = self.proj(x) # (B, D, H/16, W/16)
17         x = x.flatten(2).transpose(1, 2) # (B, num_patches, D)
18         cls = self.cls_token.expand(B, -1, -1)
19         x = torch.cat([cls, x], dim=1) # prepend CLS token
20         x = x + self.pos_embed # add positional information
21         return x

```

Listing 9.3: Patch Embedding via a Strided Convolution

9.7.2 Self-Attention Over Patches

Chapter 7 already introduced the Query, Key, and Value mechanism for text. The mechanics here are identical – only the tokens have changed, from words to image patches:

- **Query (Q):** what features this patch is currently looking for elsewhere in the image.
- **Key (K):** what content this patch offers to any patch that queries it.
- **Value (V):** the actual content of this patch, passed forward if it is matched.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (9.5)$$

The key architectural difference from a CNN is the receptive field. In a convolution, a neuron only sees a small local window, and information from distant pixels can only combine after many stacked layers. In self-attention, $QK^\top$ computes a similarity score between every pair of patches at once – patch 1 and patch 196 can influence each other directly, in a single layer, from the very start of the network.

9.7.3 CNN versus Vision Transformer

This global receptive field is not free. It comes at the cost of the assumptions that make CNNs efficient on modest amounts of data.

Metric	CNN	Vision Transformer
Primary operator	Local windowed convolution kernels	Global query-key-value self-attention
Receptive field	Grows gradually with depth	Fully global from the first layer
Inductive bias	Strong: locality, translation equivariance	Minimal: spatial structure learned from data
Data requirement	Sample-efficient, works on small datasets	Needs large-scale pre-training (e.g. ImageNet-21k, JFT-5B)

Table 9.3: CNNs bake in useful assumptions about images – nearby pixels are related, and an object is the same object wherever it appears – that let them generalize from comparatively little data. ViT assumes nothing about spatial structure and must learn it from data volume alone.

In the original ViT paper, this trade-off shows up directly in the pre-training results: on small and medium datasets, ResNets outperform ViT, but once pre-training data scales into the hundreds of millions of images, ViT overtakes every convolutional baseline. This reframes the CNN-versus-Transformer question from “which architecture is better” to “how much data do you have,” and is a large part of why modern large-scale vision and multimodal systems now standardize on the Transformer as a shared backbone.

```

1 import torch
2 import torch.nn as nn
3
4 class SimpleViT(nn.Module):
5     def __init__(self, image_size=64, patch_size=8, num_classes=10,
6                   embed_dim=128, heads=4, num_layers=4):
7         super().__init__()
8         self.num_patches = (image_size // patch_size) ** 2
9         self.patch_embed = nn.Conv2d(3, embed_dim, kernel_size=patch_size, stride=
10 patch_size)
11         self.cls_token = nn.Parameter(torch.randn(1, 1, embed_dim))
12         self.pos_embed = nn.Parameter(torch.randn(1, self.num_patches + 1, embed_dim))
13
14         encoder_layer = nn.TransformerEncoderLayer(
15             d_model=embed_dim, nhead=heads, dim_feedforward=embed_dim * 4,
16             batch_first=True, activation='gelu', dropout=0.1)
17         self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
18         self.head = nn.Sequential(nn.LayerNorm(embed_dim), nn.Linear(embed_dim,
19 num_classes))
20
21     def forward(self, x):
22         B = x.shape[0]
23         x = self.patch_embed(x).flatten(2).transpose(1, 2)  # (B, num_patches, D)
24         cls = self.cls_token.expand(B, -1, -1)
25         x = torch.cat((cls, x), dim=1) + self.pos_embed
26         x = self.transformer(x)
27         return self.head(x[:, 0])  # classify using the CLS token only

```

Listing 9.4: A Minimal Vision Transformer for Image Classification

A quick empirical check with these exact two architectures, trained for only a few epochs on a small dataset, confirms the theory: the convolutional UNet-style backbone converges faster and reaches higher accuracy than the Transformer, which needs to see far more examples before its self-attention patterns become useful. This small-scale result is the same data-scaling trade-off shown in the original ViT paper’s benchmarks, just at a much smaller scale.

Conclusion of Chapter 9: We have now traced the full arc of modern computer vision: unstable, ultra-deep VGG stacks were rescued by ResNet’s gradient highways; bounding-box detection was compressed into a single regression by YOLO; UNet showed that skip connections can restore lost spatial detail for pixel-perfect segmentation, a pattern that now also powers the denoising step inside modern image-generation diffusion models; and ViT showed that with enough data, even the local convolutional kernel is not a law of nature but a useful inductive bias that global self-attention can outgrow. Every architecture in this chapter has still shared one assumption, however: an agent that only ever looks at a fixed dataset and predicts a label. What happens when an agent must instead act, receive feedback, and learn from the consequences of its own decisions in a changing environment? In our final chapter, we leave supervised perception behind and enter the dynamic, feedback-driven world of Reinforcement Learning.

Chapter 10

Reinforcement Learning

Bhavana & Atharva

[Link to session recording](#)

10.1 Warmup: The RL Loop and Foresight

Up to this point, our models have been passive observers. They ingest a dataset of emails, housing prices, or images, and output a prediction. They do not interact with their environment, and their predictions do not change the state of the world.

Reinforcement Learning (RL) shifts this paradigm entirely. It is a framework for **Sequential Decision Making**. In RL, an **Agent** actively interacts with a dynamic **Environment**. It observes the current **State**, takes an **Action**, and receives numerical feedback in the form of a **Reward**. The environment then transitions to a new state based on that action.

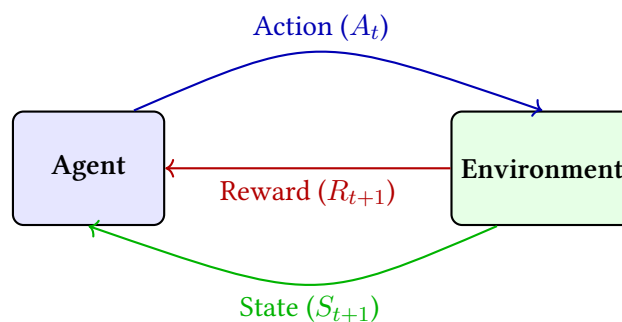


Figure 10.1: The Reinforcement Learning Loop.

Why do we need this? Consider an autonomous cooling system in a massive data center. We cannot simply provide a labeled dataset of "perfect cooling actions" because the optimal action depends entirely on the current, ever-changing temperature load. The agent must continuously adapt. Furthermore, it must possess **Foresight**. It might need to take an action that yields a negative reward right now (like turning on an expensive pump) to prevent a catastrophic, highly-penalized meltdown 10 minutes in the future.

10.2 Markov Decision Processes (MDP)

To solve RL problems mathematically, we must formalize the environment. We do this using a **Markov Decision Process (MDP)**.

10.2.1 The Markov Property

The absolute foundation of an MDP is the **Markov Property**: *The future state depends solely on the current state and action, not on the sequence of events that preceded it.*

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, S_2, \dots, S_t] \quad (10.1)$$

If the current state is truly Markov, it contains all the relevant information from the past needed to predict the future. (E.g., In chess, the current board configuration is a Markov state. You do not need to know the entire history of moves to decide your next best move).

An MDP is defined by a tuple $\langle S, A, T, R \rangle$:

- **S (States)**: The finite set of situations the agent can find itself in.
- **A (Actions)**: The finite set of choices available to the agent.
- **T (Transition Dynamics)**: The probability $\mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$. It defines how the environment changes when an action is taken.
- **R (Reward Function)**: The immediate numerical feedback $R(s, a, s')$.

10.2.2 State Transition Matrix

When the action policy is fixed or unconditioned, transitions between n finite states can be represented as an $n \times n$ **State Transition Probability Matrix P**:

$$\mathbf{P} = \begin{bmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{bmatrix} \quad \text{where } P_{ij} = \mathbb{P}[S_{t+1} = s_j | S_t = s_i] \quad (10.2)$$

where each row sums to 1 ($\sum_{j=1}^n P_{ij} = 1$).

10.2.3 Policies and Returns

The agent's behavior is dictated by a **Policy** (π). A deterministic policy always selects the exact same action in a given state. A stochastic policy $\pi(a|s)$ assigns a probability distribution over possible actions.

The agent's goal is not to maximize immediate reward, but to maximize the **Return** (G_t), which is the total accumulated reward from time t onward. Because infinite tasks would yield infinite returns, we introduce a **Discount Factor** ($\gamma \in [0, 1)$):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (10.3)$$

If $\gamma = 0$, the agent is incredibly short-sighted (only cares about immediate reward). As γ approaches 1, the agent becomes far-sighted, caring heavily about the distant future.

10.3 Value Functions and The Bellman Equation

How do we measure how "good" a state is? We define the **State Value Function** $V^\pi(s)$, which is the expected return starting from state s and strictly following policy π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (10.4)$$

Similarly, the **Action Value Function** $Q^\pi(s, a)$ measures how good it is to take a specific action a in state s , and then follow policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \quad (10.5)$$

10.3.1 The Bellman Expectation Equation

Predicting an infinite sum of future rewards seems mathematically impossible. Richard Bellman's genius was realizing that the Return can be broken down recursively. The value of a state is simply the immediate reward plus the discounted value of the *next* state!

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] \quad (10.6)$$

Expanding this using our transition probabilities yields the full Bellman Equation:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (10.7)$$

Worked Example: Policy Evaluation vs. Value Iteration

Consider a simple MDP consisting of two non-terminal states (*A* and *B*) and a Terminal state (*T*).

- **Actions:** {Left, Right}.
- **Transitions & Rewards:**
 - From *A*: Right $\rightarrow$ *B* ($R = 0$); Left $\rightarrow$ *T* ($R = 0$).
 - From *B*: Right $\rightarrow$ *T* ($R = +1$); Left $\rightarrow$ *A* ($R = 0$).
- **Discount:** $\gamma = 1.0$. $V(T) = 0$.

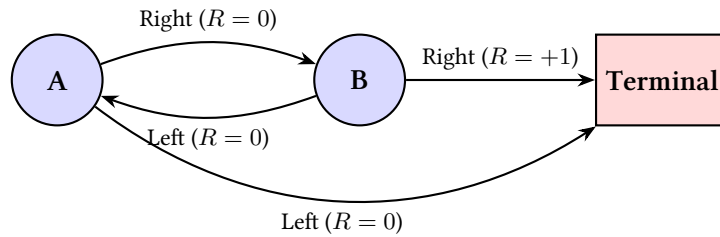


Figure 10.2: State Transition Diagram for the 2-State MDP.

Intuitive Explanation of the Worked Example

To understand the mathematical logic behind this example without getting lost in formal notation, picture the 2-state MDP as walking through three rooms connected in a line: **Room A**, **Room B**, and the **Exit (Terminal)**.

- **Room A:** Moving Left takes you straight to the Exit with 0 reward. Moving Right takes you to Room B with 0 reward.
- **Room B:** Moving Left takes you back to Room A with 0 reward. Moving Right takes you to the Exit and awards a +1 reward.
- **Objective:** Reach the exit while accumulating the maximum total reward possible.

1. Evaluating a Bad Strategy (Policy Evaluation)

Suppose the agent is given a rigid, fixed rule set π :

- If in **Room A**, always move Right to **B**.
- If in **Room B**, always move Left to **A**.

What happens? The agent gets trapped in an infinite loop, shuffling back and forth between Room A and Room B ($A \rightarrow B \rightarrow A \rightarrow B \dots$). Because it never chooses to move Right from Room B, it never reaches the exit to cash in on the +1 reward.

Since every move awards 0 points indefinitely, the expected total value of starting in either room under this strategy is zero:

$$V^\pi(A) = 0 \quad \text{and} \quad V^\pi(B) = 0 \quad (10.8)$$

2. Finding the Optimal Strategy (Value Iteration) Instead of following a fixed rule, we evaluate the best choices by reasoning backward from the payout:

1. Evaluate Room B first:

- Moving Left takes you to Room A (currently worth 0).
- Moving Right exits immediately and earns +1 point.
- *Decision:* The best move in Room B is **Right**. This makes Room B worth 1 point:

$$V^*(B) = 1 \quad (10.9)$$

2. Evaluate Room A next:

- Moving Left exits immediately for 0 points.
- Moving Right moves you into Room B (which we just calculated is worth 1 point).
- *Decision:* The best move in Room A is **Right**. This makes Room A also worth 1 point:

$$V^*(A) = 1 \quad (10.10)$$

Final Takeaway

The optimal policy π^* tells the agent:

- When in **Room A** $\rightarrow$ Move **Right**.
- When in **Room B** $\rightarrow$ Move **Right** to collect +1 and exit.

Under optimal play, starting in either Room A or Room B guarantees a total score of 1.

10.3.2 The Bellman Optimality Equation

We don't just want to evaluate a fixed policy; we want the *best possible* policy π^* . The optimal value function $V^*(s)$ must satisfy the Bellman Optimality Equation, replacing the expected policy average with an absolute max over actions:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (10.11)$$

Similarly, the optimal action-value function $Q^*(s, a)$ is defined as the expected immediate reward plus the discounted optimal value of the best action in the next state:

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (10.12)$$

Once we have the optimal Q-values, $Q^*(s, a)$, finding the optimal policy is trivial: we simply act greedily by taking the action with the highest Q-value!

Code Implementation: Value Iteration

Listing 10.1 demonstrates Value Iteration implemented in Python for the 2-State MDP example.

```

1 import numpy as np
2
3 # Define MDP Parameters
4 states = ['A', 'B']
5 gamma = 1.0
6 theta = 1e-6 # Convergence threshold
7
8 # Initialize values V(S) to zero
9 V = {'A': 0.0, 'B': 0.0, 'T': 0.0}
10
11 def bellman_backup(state, V):
12     """Computes max action value for a given state."""
13     if state == 'A':
14         # Action Left -> T (R=0), Action Right -> B (R=0)
15         q_left = 0.0 + gamma * V['T']
16         q_right = 0.0 + gamma * V['B']
17         return max(q_left, q_right)
18     elif state == 'B':
19         # Action Left -> A (R=0), Action Right -> T (R=1)
20         q_left = 0.0 + gamma * V['A']
21         q_right = 1.0 + gamma * V['T']
22         return max(q_left, q_right)
23
24 # Value Iteration Loop
25 iteration = 0
26 while True:
27     delta = 0
28     new_V = V.copy()
29     for s in states:
30         v_old = V[s]
31         new_V[s] = bellman_backup(s, V)
32         delta = max(delta, abs(v_old - new_V[s]))
33
34     V = new_V
35     iteration += 1
36     if delta < theta:
37         break
38
39 print(f"Converged in {iteration} iterations.")
40 print(f"Optimal Value Function: V*(A) = {V['A']:.2f}, V*(B) = {V['B']:.2f}")

```

Listing 10.1: Python Implementation of Value Iteration for a 2-State MDP

10.4 Dynamic Programming (DP)

If we have a **perfect model** of the environment (meaning we explicitly know the transition probabilities P and rewards R , such as the exact rules of a chess game), we can solve the Bellman equations using **Dynamic Programming**.

- **Iterative Policy Evaluation:** We sweep over all states, updating their values using the Bellman Expectation Equation until they converge.
- **Policy Improvement:** Once we know the true values of states under policy π , we act greedily with respect to those values to create a strictly better policy π' .
- **Value Iteration:** We fuse these steps by forcefully applying the Bellman Optimality update rule across the entire state space iteratively.

The Limit of DP: DP is planning, not learning. It requires complete knowledge of the environment dynamics. In the real world (like a self-driving car on a chaotic highway), we almost never have a perfect transition matrix. We must learn from experience.

10.5 Monte Carlo (MC) Methods

When we drop the perfect model, we enter true Reinforcement Learning. **Monte Carlo** methods learn values directly from raw experience (episodes of interaction).

The core idea is the Law of Large Numbers: the expected value (Value Function) is simply the average of the observed returns.

1. The agent plays an entire episode from start to finish (e.g., plays a full game of Blackjack).
2. It calculates the actual return G_t observed after visiting a state.
3. It updates the value of that state by averaging all the returns it has ever observed from it.

The Exploration Problem: If our agent acts purely greedily based on early, flawed MC estimates, it might find a suboptimal path and exploit it forever, never exploring other actions that might yield higher rewards. We solve this using an ϵ -greedy policy: with probability $1 - \epsilon$, the agent exploits the best known action; with probability ϵ , it explores a completely random action.

10.6 Temporal Difference (TD) Learning

Monte Carlo is unbiased, but it forces us to wait until the very end of an episode to update our values (because we need the final G_t). What if the episode is incredibly long, or continuous (never ends)?

Temporal Difference (TD) learning is the master synthesis of DP and MC. Like MC, it learns from raw experience without a model. Like DP, it **bootstraps**: it updates its estimates based on other learned estimates, without waiting for the final outcome!

Instead of updating toward the true return G_t , a TD(0) agent takes a single step, observes the immediate reward R_{t+1} , and updates its current state value based on its existing estimate of the *next* state:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (10.13)$$

The term inside the brackets is the **TD Error** (δ_t). α is the learning rate.

10.6.1 SARSA: On-Policy Control

SARSA updates the Q-values based on the exact path the agent is currently walking (On-Policy). It takes an action A_t , observes R_{t+1} and S_{t+1} , then uses its policy to pick the actual next action A_{t+1} to calculate the update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (10.14)$$

10.6.2 Q-Learning: Off-Policy Control

Q-Learning is legendary because it is **Off-Policy**. It learns the optimal policy even if the agent is acting randomly! Instead of using the actual next action the agent takes, it updates using the *maximum* possible Q-value of the next state:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \quad (10.15)$$

```
1 def q_learning_update(Q_table, state, action, reward, next_state, alpha=0.1, gamma=0.9):
2     # Get the maximum Q-value for the next state
3     best_next_action_value = max(Q_table[next_state].values())
4
5     # Calculate the TD Target
6     td_target = reward + gamma * best_next_action_value
7
8     # Calculate the TD Error
9     td_error = td_target - Q_table[state][action]
10
11    # Update the Q-value
12    Q_table[state][action] += alpha * td_error
13
14    return Q_table
```

Listing 10.2: Basic Q-Learning Update Step

Conclusion of Chapter 10: From the rigid mathematics of dynamic programming to the exploratory, model-free leaps of Q-Learning, Reinforcement Learning allows AI to conquer dynamic, complex environments. Modern Deep RL combines these exact TD concepts with Neural Networks (e.g., replacing the Q-table with a Deep Q-Network) to achieve superhuman performance in everything from robotics to AlphaGo.

Appendix A

Mathematical Toolkit

Welcome to the engine room. Machine Learning often feels like magic, but under the hood, it is entirely driven by three mathematical engines: **Linear Algebra** (how we represent and manipulate data), **Probability** (how we handle uncertainty), and **Calculus** (how we learn and optimize).

If you are new to these concepts, seeing a wall of equations can be intimidating. This appendix is designed to bridge the gap between abstract mathematical definitions and the physical intuition you need to understand Machine Learning.

A.1 Linear Algebra: The Language of Data

In programming, we use arrays and lists. In Machine Learning, we use vectors and matrices. Linear algebra allows us to process thousands of data points simultaneously without writing a single for loop.

A.1.1 Scalars, Vectors, and Matrices

- **Scalar** ($x \in \mathbb{R}$): A single number. It represents a single magnitude, like the temperature of a room or the price of a house.
- **Vector** ($x \in \mathbb{R}^d$): An ordered list of d numbers, arranged in a column. Geometrically, you can think of a vector as an arrow pointing from the origin $(0, 0)$ to a specific coordinate in a d -dimensional space. In ML, a vector usually represents a single *sample* or *data point*. For example, a house vector might be $x = [2000 \text{ sqft}, 3 \text{ beds}, 2 \text{ baths}]^T$, simplistically represented as $[2000, 3, 2]^T$.
- **Matrix** ($X \in \mathbb{R}^{n \times d}$): A 2D grid of numbers with n rows and d columns. In ML, a matrix almost always represents our *entire dataset*. Each row is a specific data point (a house), and each column is a feature (square footage, bedrooms, etc.).

A.1.2 The Dot Product (Inner Product)

The dot product takes two vectors of the same size and returns a single scalar number. It is calculated by multiplying matching elements and adding them up:

$$x \cdot y = x^T y = \sum_{i=1}^d x_i y_i \quad (\text{A.1})$$

The Intuition: The dot product measures *alignment* or *similarity*. If two vectors point in the exact same direction, their dot product is large and positive. If they point in opposite directions, it is negative. If they are exactly perpendicular (orthogonal), their dot product is zero.

Whenever you see a model trying to figure out how relevant a query is to a piece of data (like the Attention mechanism in Large Language Models), it is fundamentally relying on the dot product.

A.1.3 Matrix Multiplication

To multiply two matrices, $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the inner dimensions (n) absolutely must match. The output is a new matrix $C \in \mathbb{R}^{m \times p}$.

Do not think of matrix multiplication as just a tedious arithmetic algorithm. Think of it as computing many dot products all at once. When we write $\hat{y} = X\theta$ in Linear Regression, we are taking the dot product of our parameter vector θ with *every single row* (data point) in our dataset matrix X simultaneously.

A.1.4 Transpose and Inverse

- **Transpose (A^T):** Flipping a matrix over its main diagonal. Rows become columns, and columns become rows. A crucial property to remember when taking the transpose of a product is that the order reverses: $(AB)^T = B^T A^T$.
- **Inverse (A^{-1}):** In normal arithmetic, to undo multiplication by 5, we multiply by $1/5$. In linear algebra, to undo a matrix transformation A , we multiply by its inverse A^{-1} , such that $AA^{-1} = I$ (where I is the Identity matrix, the matrix equivalent of the number 1).

Note: Not all matrices have an inverse. A matrix must be square (same number of rows and columns) and have "full rank" (no column can be a perfect linear combination of other columns) to be invertible. We use the inverse extensively in the Normal Equation for Ordinary Least Squares (OLS) to solve for our weights.

A.1.5 Eigenvectors and Singular Value Decomposition (SVD)

Imagine a matrix A as a physical action that stretches and rotates vectors in space. Most vectors get knocked off their original line when A is applied to them. However, for any given matrix, there are a few special vectors that *only* get stretched or shrunk, but do not change their direction.

These stubborn vectors are called **Eigenvectors** (v), and the amount they are stretched is called the **Eigenvalue** (λ):

$$Av = \lambda v \tag{A.2}$$

While eigenvectors only exist for square matrices, **Singular Value Decomposition (SVD)** is the ultimate generalization. SVD proves that *any* matrix (even non-square ones) can be broken down into three distinct, simpler steps:

$$X = U\Sigma V^T \tag{A.3}$$

Where V^T rotates the space, Σ stretches it along the axes, and U rotates it again. (To be a bit more concrete, U and V are orthogonal matrices and Σ is a diagonal matrix.) SVD is the mathematical backbone of dimensionality reduction techniques like Principal Component Analysis (PCA).

A.2 Probability and Statistics: Handling Uncertainty

In ML, we never have perfect information. Our data is noisy, our sensors are flawed, and the universe is chaotic. Probability gives us a rigorous framework to make optimal decisions despite this uncertainty.

A.2.1 Random Variables and Vectors

Before we can model data, we need a mathematical language to describe uncertainty.

- **Random Variable X :** Unlike a traditional algebraic variable that holds a single fixed number, a **random variable** X is a variable whose exact value is governed by an unpredictable, random process (like the measured height of a randomly selected person, or the noise in a sensor).

- **Expectation** $\mathbb{E}[X]$: Also known as the mean. If you sampled X infinitely many times, this is the average value you would get. It represents the "center of mass" of the probability distribution.
- **Variance** $Var(X)$: Measures how wildly X fluctuates around its mean. A low variance means most samples are tightly packed around the average; a high variance means the values are heavily spread out.

$$Var(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] \quad (\text{A.4})$$

- **Covariance** $Cov(X, Y)$: Measures how two different random variables move together. If tall people tend to weigh more, height and weight have a positive covariance. If they are completely independent, their covariance is exactly zero.

In Machine Learning, a single data point is rarely just one number; it is an array of features. Thus, we must extend these concepts to **Random Vectors** $\mathbf{x} \in \mathbb{R}^d$.

- **Expectation Vector** $\mathbb{E}[\mathbf{x}]$: Simply a vector containing the expected value of each individual feature.
- **Covariance Matrix** Σ : When dealing with vectors, variance and covariance are elegantly combined into a symmetric $d \times d$ matrix. The Covariance Matrix Σ (or $Cov(\mathbf{x})$) captures how every single feature in the vector varies with every other feature:

$$\Sigma = \mathbb{E}[(\mathbf{x} - \mathbb{E}[\mathbf{x}])(\mathbf{x} - \mathbb{E}[\mathbf{x}])^T] \quad (\text{A.5})$$

The diagonal elements of Σ represent the individual variances of each feature (e.g., the variance of feature x_1). The off-diagonal elements represent the covariances between different features (e.g., how feature x_1 moves with feature x_2). If all features are completely independent, all off-diagonal elements become zero, and Σ reduces to a diagonal matrix.

A.2.2 Bayes' Theorem and Parameter Estimation

How do we find the mathematical rules that govern our data? We use statistics to estimate the parameters of our models. This process is deeply rooted in Bayes' Theorem, which tells us how to mathematically update our beliefs when we see new evidence.

Let D represent our observed Dataset (the evidence), and let θ represent the parameters of our model (our belief). Bayes' Theorem states:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \quad (\text{A.6})$$

- $P(\theta|D)$ is the **Posterior**: How probable are these parameters *after* seeing the data? (This is what we ultimately want to maximize).
- $P(D|\theta)$ is the **Likelihood**: If these parameters were true, how probable is it that we would have observed this exact dataset?
- $P(\theta)$ is the **Prior**: Before looking at any data, how probable did we think these parameters were?
- $P(D)$ is the **Evidence**: The total probability of observing the data under all possible parameters (usually treated as a normalizing constant that we can ignore during optimization).

From this theorem, two fundamental philosophies of machine learning emerge:

1. Maximum Likelihood Estimation (MLE)

MLE is the purely data-driven approach. It assumes we have no prior beliefs whatsoever (or that all possible parameters are equally likely). If $P(\theta)$ is a constant, maximizing the Posterior is identical to maximizing the Likelihood. We simply ask: "What parameters θ make the dataset I just observed the most likely to have occurred?"

Mathematically, we want to find $\theta_{MLE} = \arg \max_{\theta} P(D|\theta)$. Because datasets consist of many independent samples, their combined likelihood is a massive product of probabilities. To prevent numerical underflow and turn complex multiplication into simple addition, we almost always maximize the **Log-Likelihood** instead:

$$\theta_{MLE} = \arg \max_{\theta} \log P(D|\theta) \quad (\text{A.7})$$

By assuming a specific probability distribution for our data—or for the inherent noise in our measurements—this abstract log-likelihood equation will elegantly transform into the exact, practical objective functions we train our models to minimize.

2. Maximum A Posteriori (MAP) Estimation

MLE is powerful but vulnerable. If we have very little data, MLE will aggressively memorize the noise (overfitting). MAP estimation solves this by bringing back the Prior $P(\theta)$.

Instead of just maximizing the Likelihood, MAP maximizes the full numerator of Bayes' Theorem (Likelihood $\times$ Prior):

$$\theta_{MAP} = \arg \max_{\theta} [\log P(D|\theta) + \log P(\theta)] \quad (\text{A.8})$$

Here, the log-prior $\log P(\theta)$ acts as a built-in mathematical penalty. If we encode a prior belief that extreme or overly complex parameters are inherently improbable, the $\log P(\theta)$ term will mathematically penalize the model for relying too heavily on the training data. This mechanism allows MAP to seamlessly unify probability theory with the practical need to constrain models and prevent them from memorizing noise.

A.3 Calculus and Optimization: How Models Learn

If linear algebra holds our data and probability defines our goal, calculus is the engine that actually gets us there. Training a model means navigating a massive, hilly landscape of "loss" (error) to find the deepest valley (the minimum error).

A.3.1 Derivatives and Gradients

The derivative $f'(x)$ tells you the slope of a curve at a specific point. If the derivative is positive, the function is going up. If it is negative, the function is going down. If it is exactly zero, you are either at the bottom of a valley or the peak of a hill.

In Machine Learning, our loss function doesn't just have one variable; it has millions of weights. The **Gradient** (∇f) is simply a vector that packs all the individual partial derivatives together.

The Golden Rule of Gradients: The gradient vector always points in the direction of the *steepest uphill ascent*. Therefore, to minimize our error, we calculate the gradient, multiply it by a small step size (the learning rate), and walk in the *exact opposite* direction. This algorithm is known as **Gradient Descent**.

A.3.2 The Chain Rule (Backpropagation)

How do we calculate the gradient for a neural network with 100 stacked layers? We use the Chain Rule.

Imagine a machine with three gears: x turns u , and u turns y . If you want to know how much y moves when you tweak x , you just multiply the effect of the middle connections:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x} \quad (\text{A.9})$$

This is the exact mathematical mechanism behind **Backpropagation**. To find out how a weight in the very first layer of a Neural Network affects the final output error, we calculate the derivative at the final layer, and multiply it backward through the derivatives of every intermediate layer. It is a brilliant, highly efficient cascade of multiplication that allows deep networks to learn.